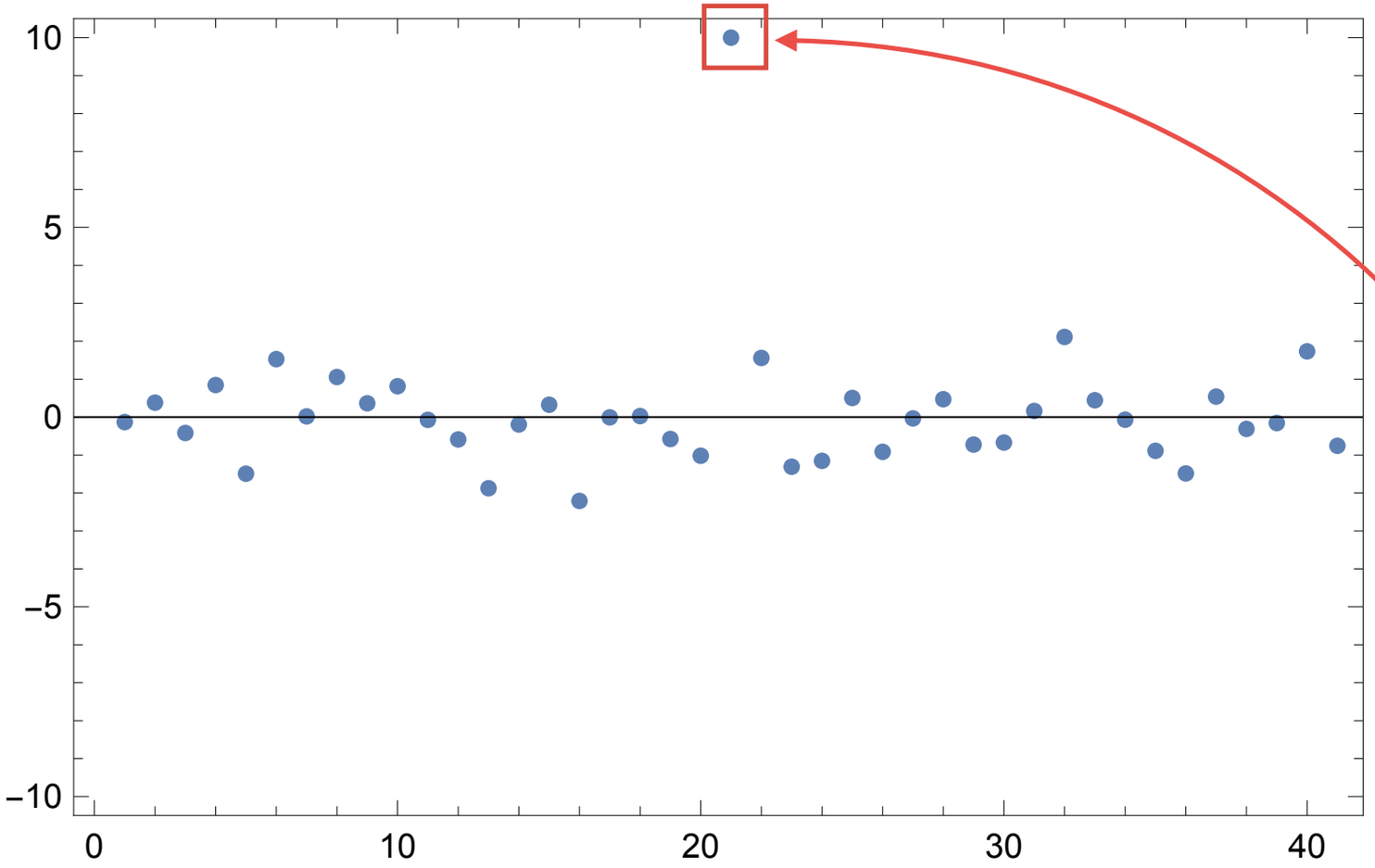
L7: Data Cleaning

ASTR3986: Astrostatistics

L7.1 Outliers

“Outliers” in astronomical data are usually interpreted as data to be removed/discarded



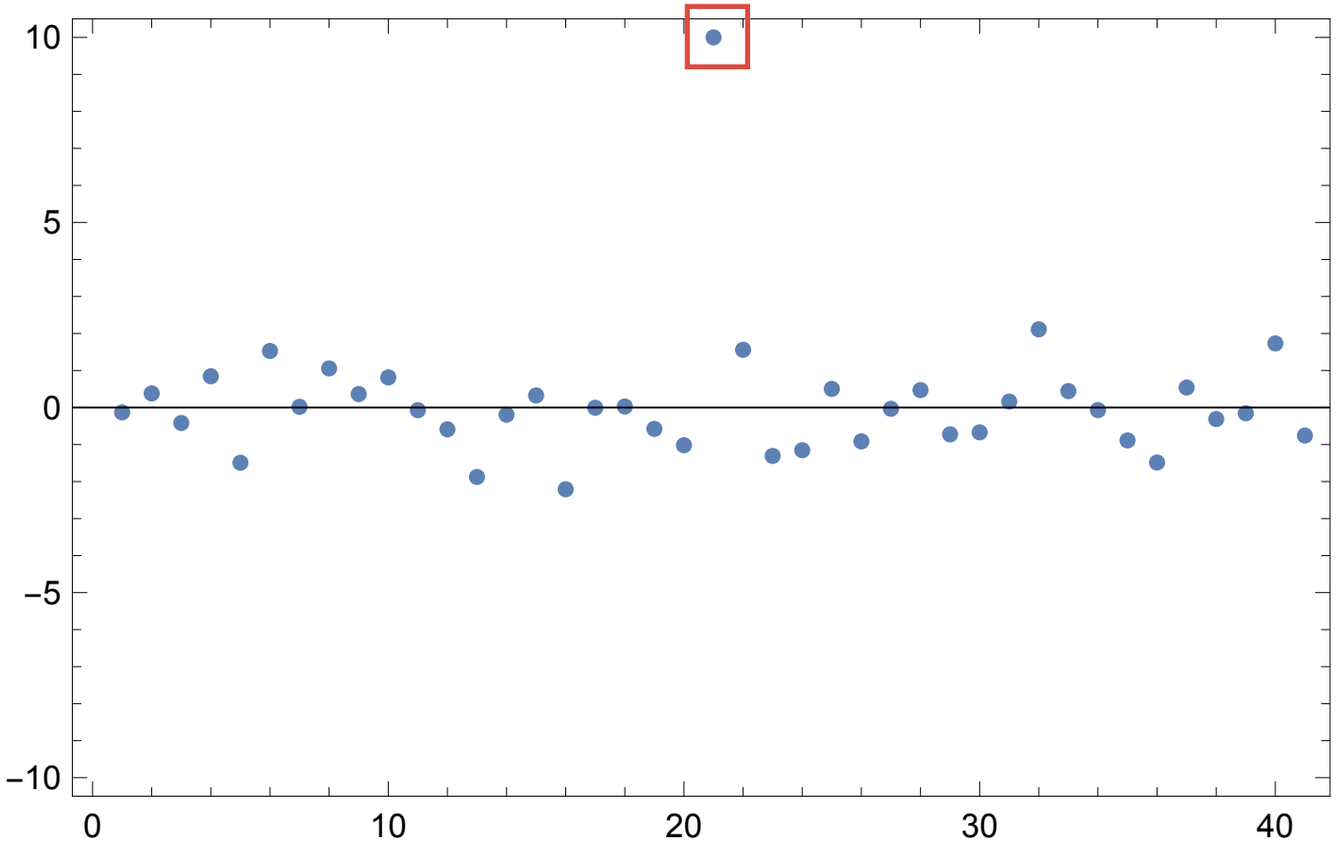
But this point is clearly different => if left-in, we’d have a **mis-specified likelihood function**

Q. How else, apart from outlier, might we get a mis-specified likelihood function?

Why? Well, we typically assume a single-form for the likelihood function, all of the data are distributed about the model as some function e.g.

$$\log \mathcal{L} = -\sum_{i=1}^{N} \frac{1}{2} \log(2\pi) - \sum_{i=1}^{N} \log \sigma_i - \frac{1}{2} \sum_{i=1}^{N} \left(\frac{r_i^2}{\sigma_i^2} \right)$$

“Outliers” in astronomical data are usually interpreted as data to be removed/discarded



not a particularly great definition (for us)!

outlier: an observation point that is distant from other observations

“distant” is not easily quantified

WIKIPEDIA
The Free Encyclopedia

```
samples = RandomVariate[NormalDistribution[0, 1], 10^6];
Histogram[samples, PlotRange → All, GridLines → {{Max[samples]}, {}}]
Max[samples]
```



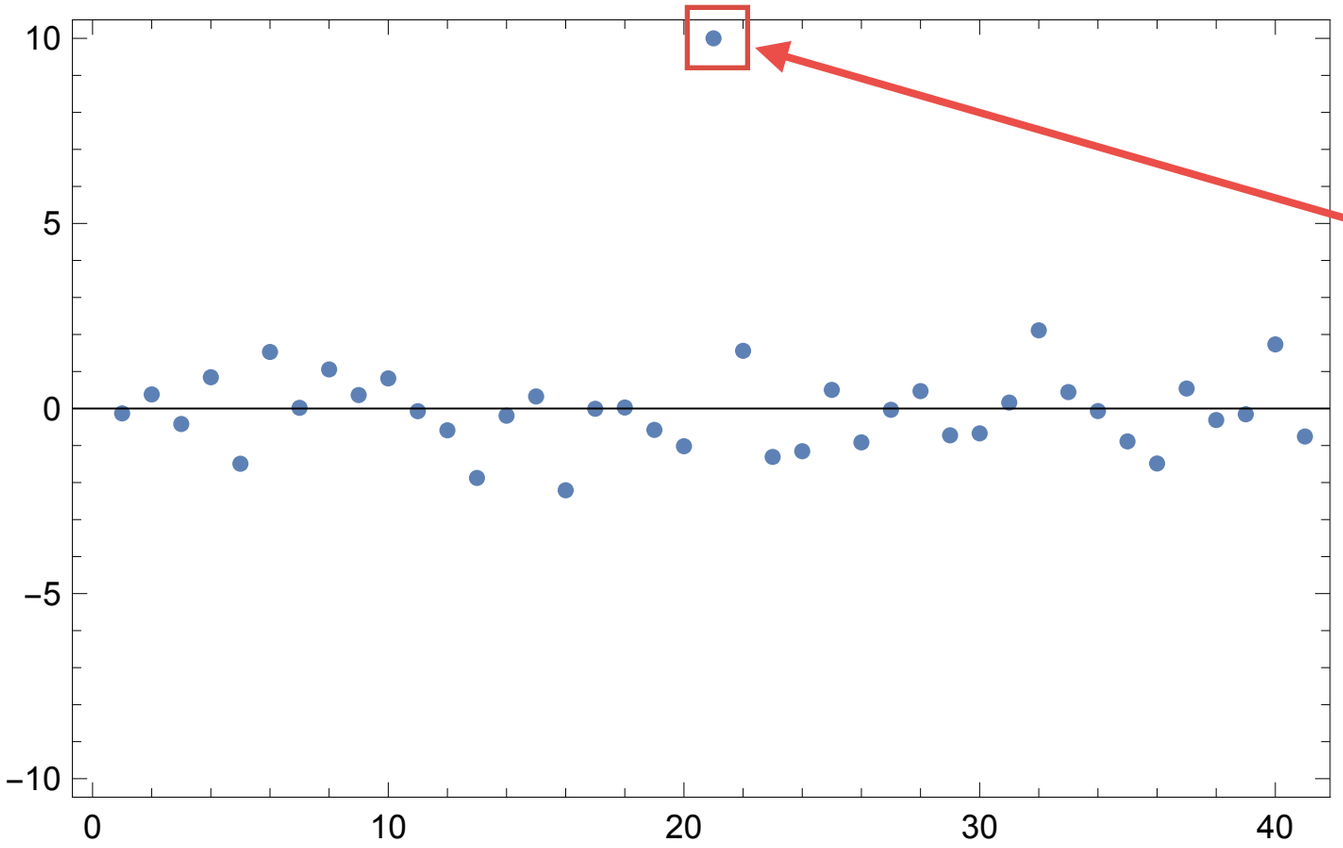

```
5.14886
```

The problem with this is that if you draw ≫1 samples from a Gaussian, sometimes it will lead to distant points

$$\log \mathcal{L} = -\sum_{i=1}^{N} \frac{1}{2} \log(2\pi) - \sum_{i=1}^{N} \log \sigma_i - \frac{1}{2} \sum_{i=1}^{N} \left(\frac{r_i^2}{\sigma_i^2} \right)$$

But this is still correct! The likelihood function has not been mis-specified, no need to exclude - in fact, excluding would make the likelihood mis-specified!

"Outliers" in astronomical data are usually interpreted as data to be removed/discarded



outlier: an observation point that is drawn from a distinct distribution to that described by our likelihood/noise model

"**mixture model**"

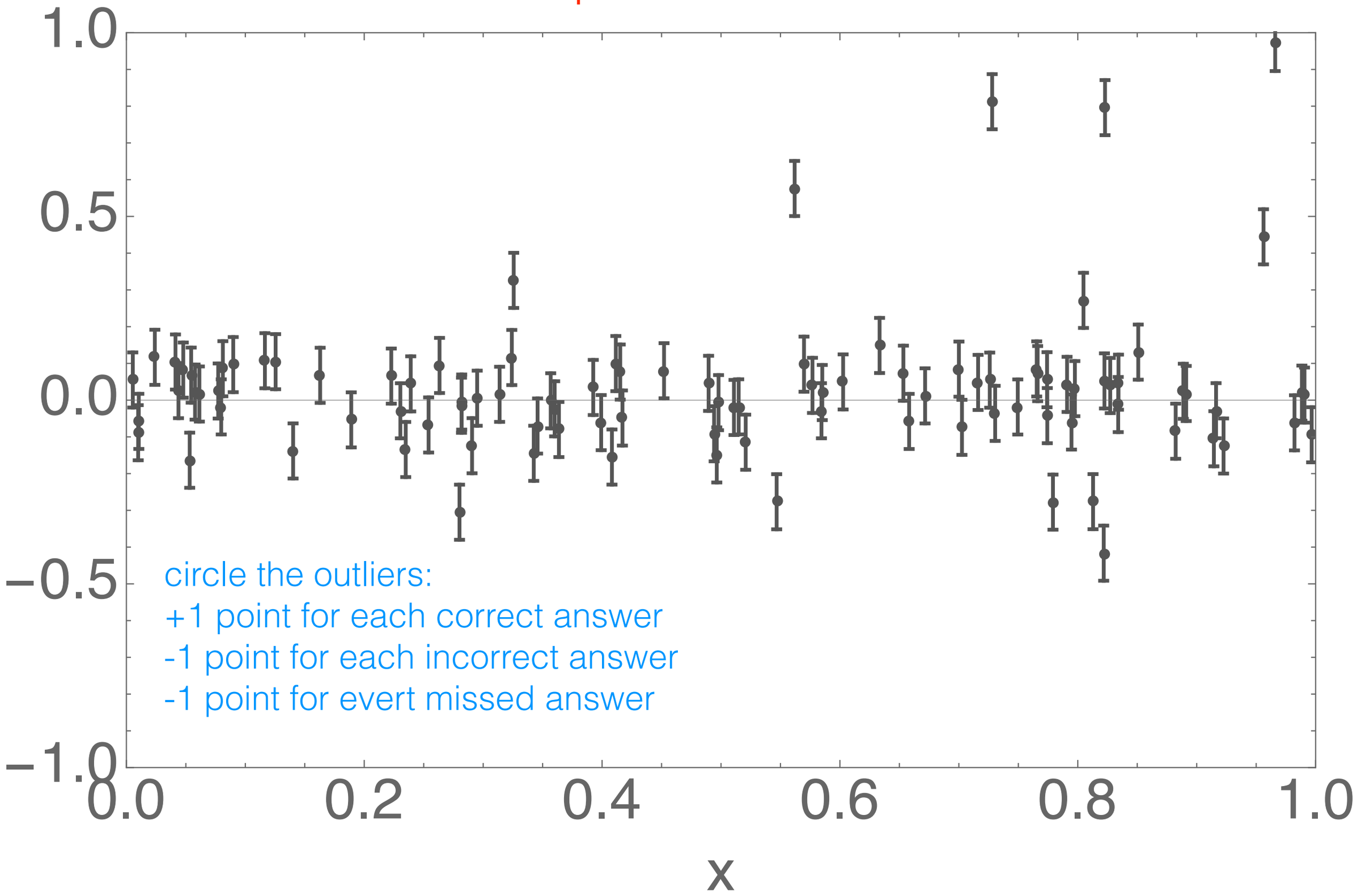
not a particularly great definition (for us)!

outlier: an observation point that is distant from other observations

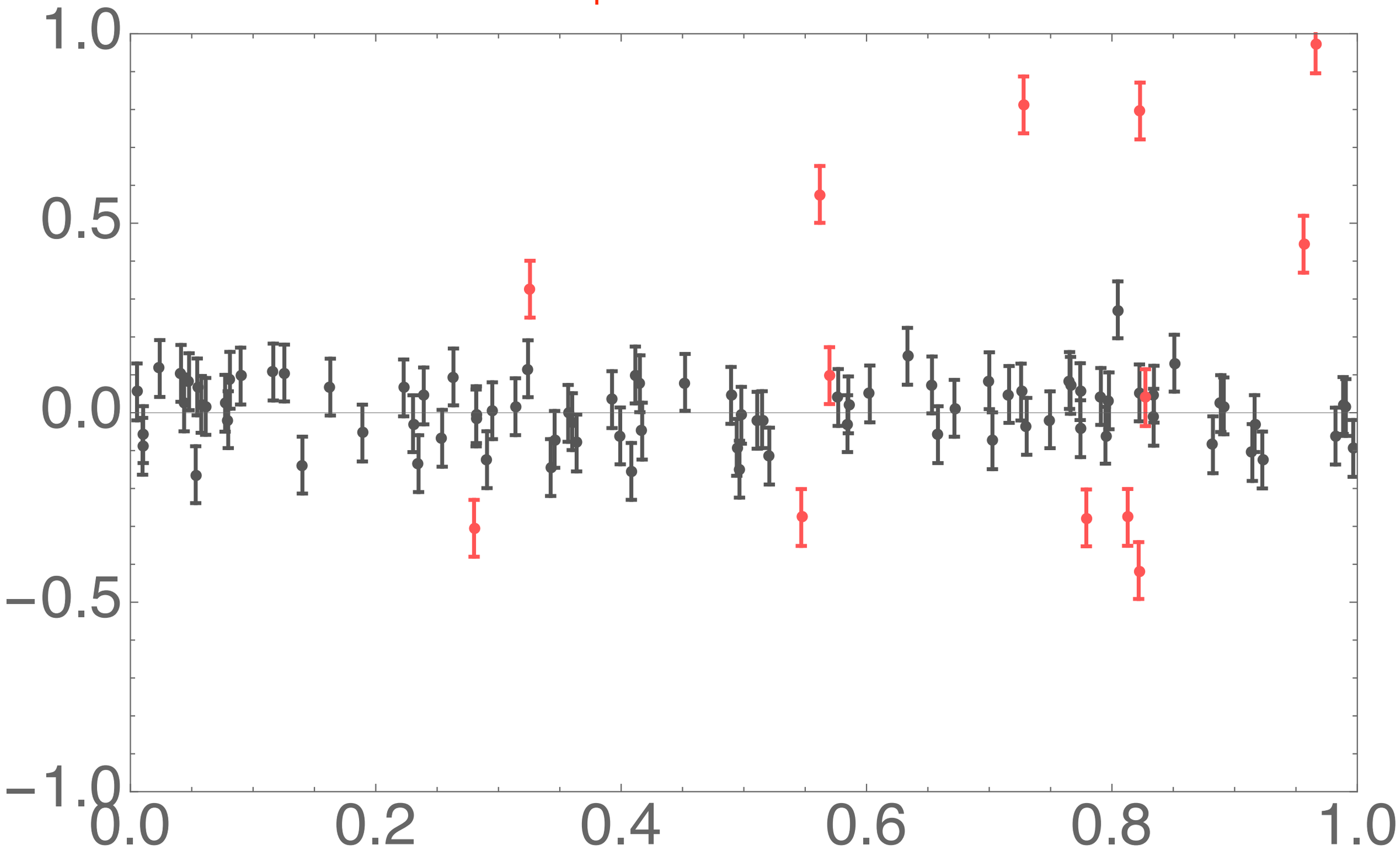
"distant" is not easily quantified

WIKIPEDIA
The Free Encyclopedia

spot the outliers!

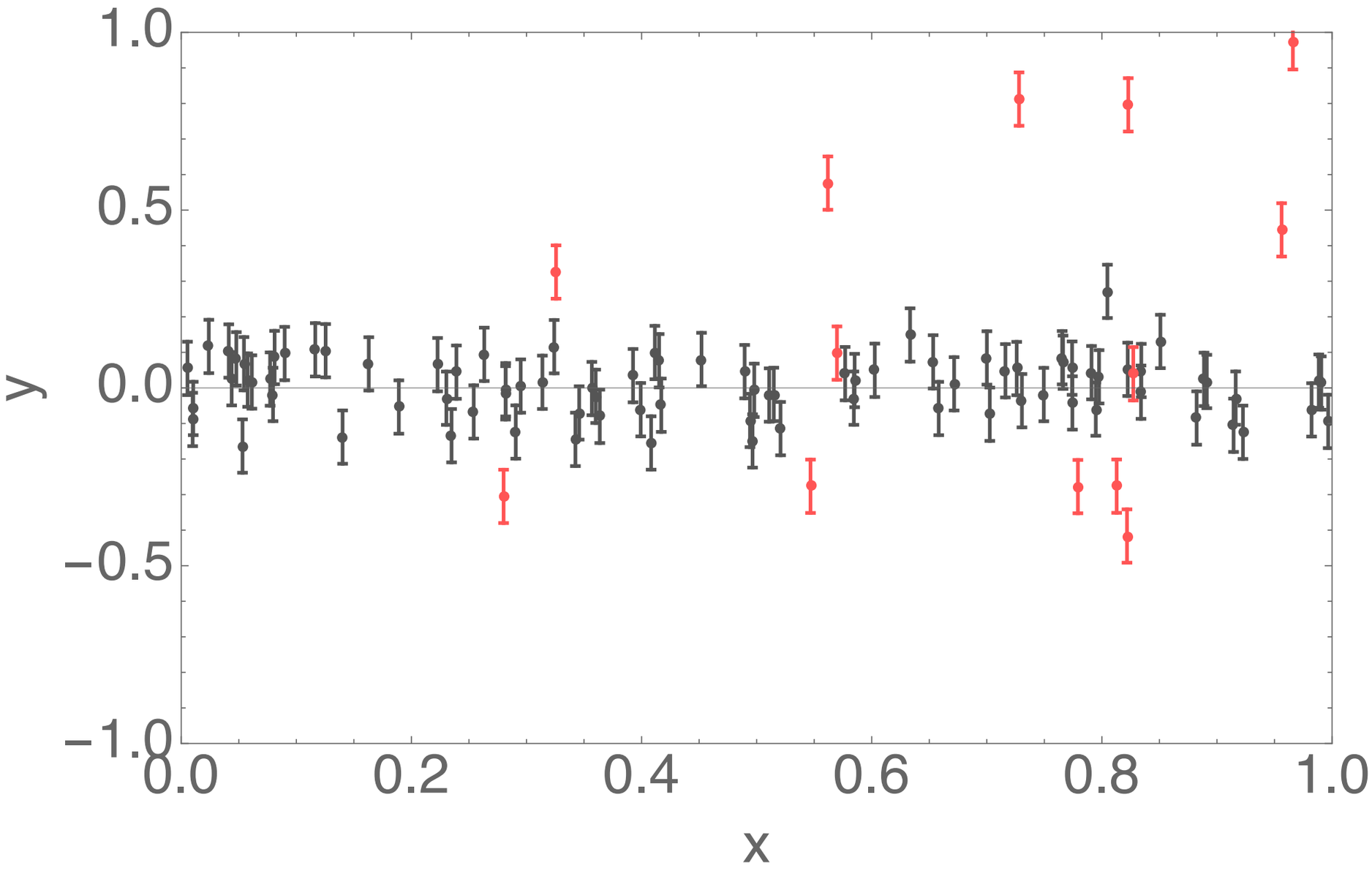


spot the outliers!



why are outliers important?

inferences made using data featuring outliers are flawed,
since presumably the likelihood function is mis-specified
(likelihood doesn't include both parts of the mixture usually)



we can't reliably fit the data whilst outliers are left untreated

↓

you could try modeling them, but typically they are simply discarded

mean with outliers = 0.029 +/- 0.008 ← one might assume this is significant!

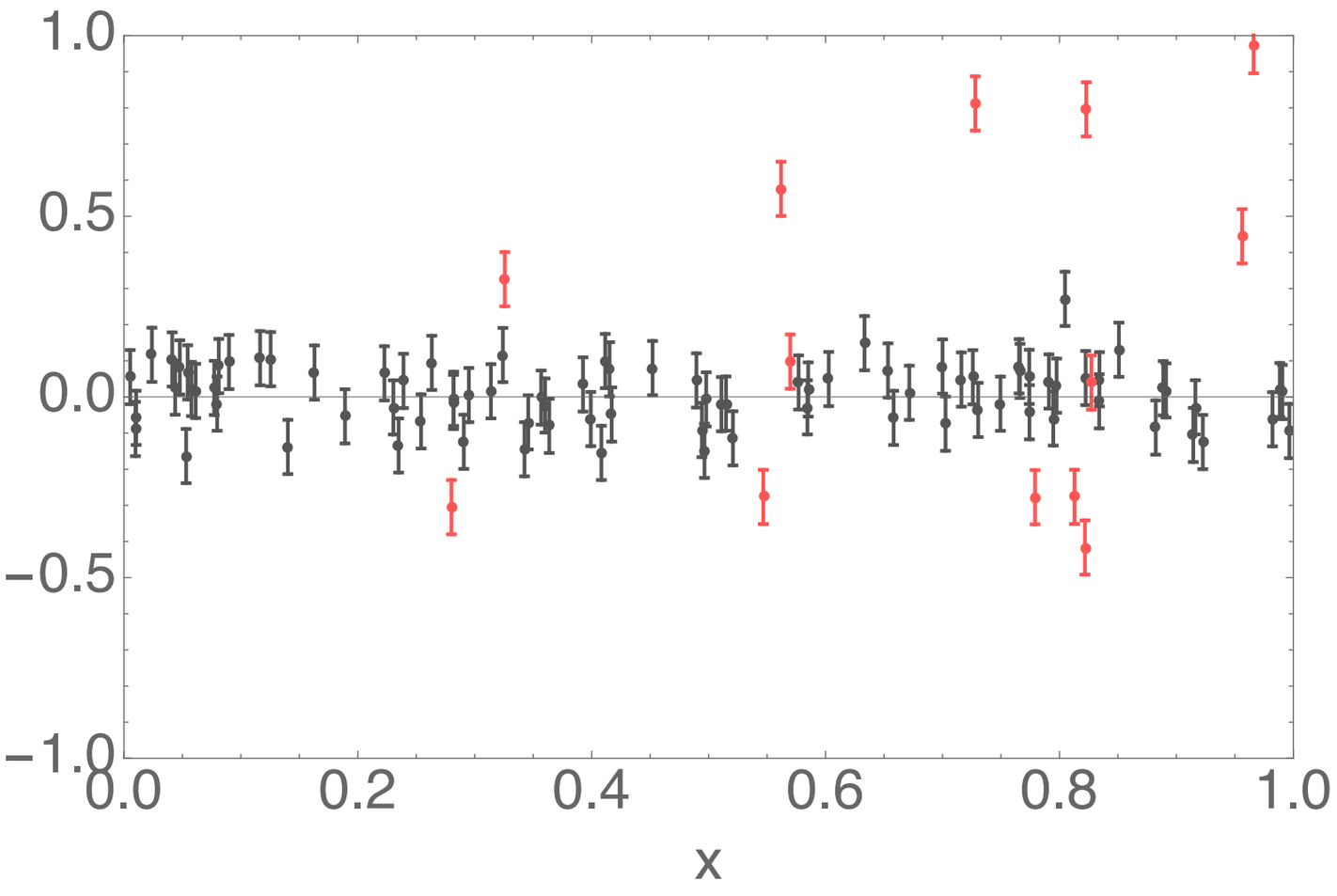
mean without outliers = 0.004 +/- 0.008

detecting outliers

human eye algorithm does something like this...

outlier if $| (y - f[x])/\sigma_y | \gg 1$

(where you probably used f[x]=0 for all x)



for this to work, need to know:

1. f[x]

2. σ_y (and distribution/$\mathscr{L}$)

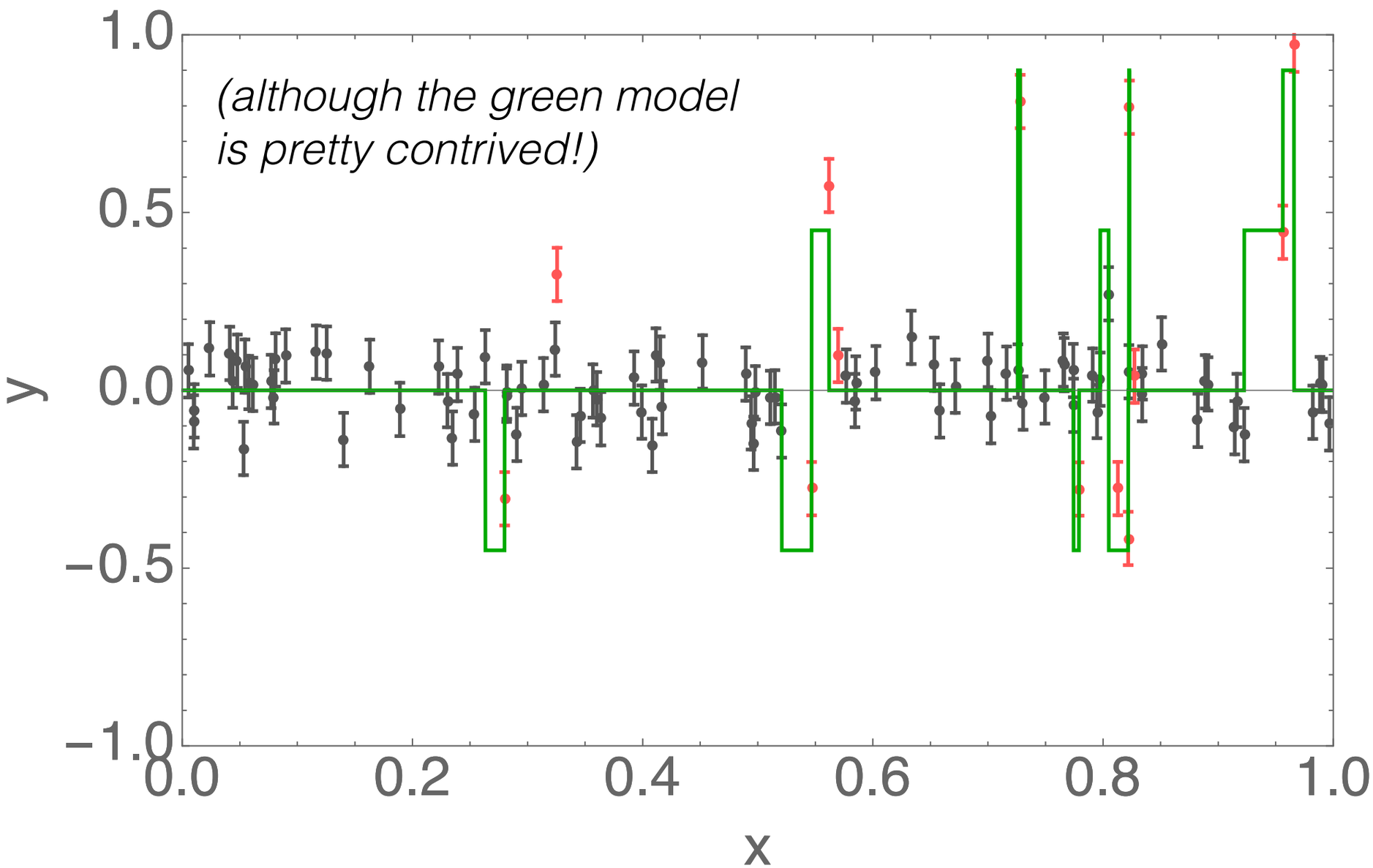
3. how big is ≫? σ cutoff

detecting outliers

problem 1: we usually don't know the true f[x]!

you all implicitly assumed the true model = 0,
but actually we don't know the true model!

if the model is really the green line, then many of those "outliers" are not really outliers!

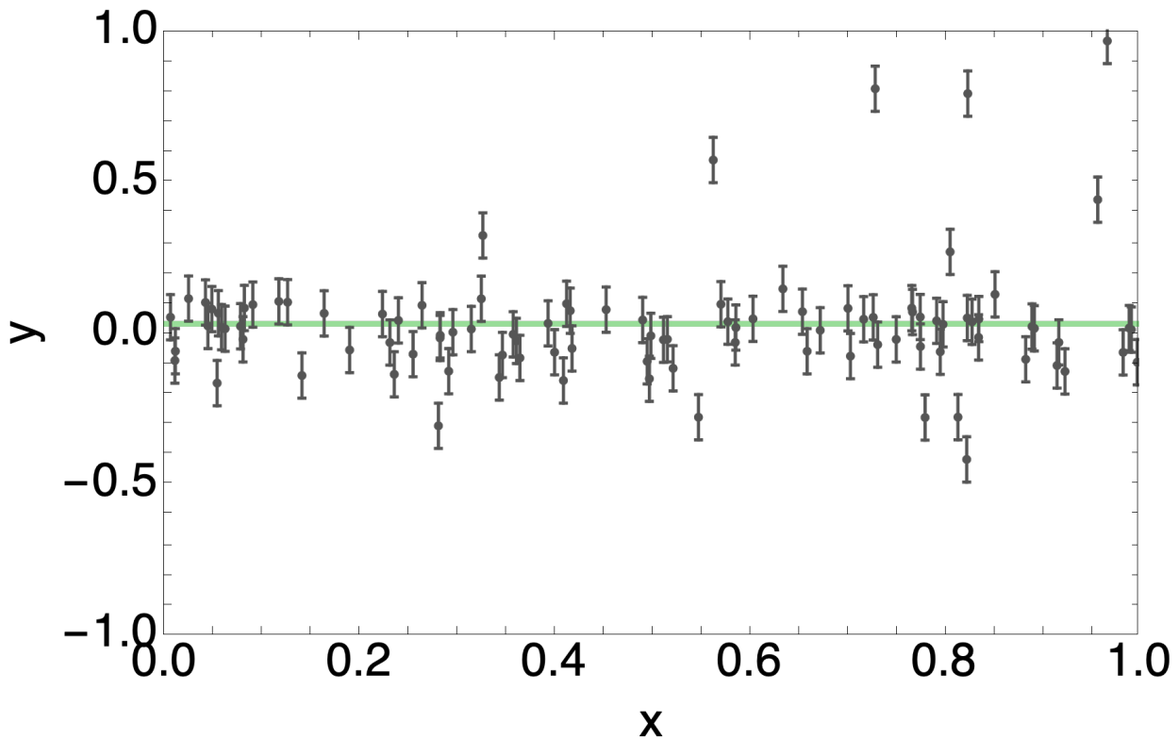


detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1A: if we believe we know the functional form of the model, but we don't know the model parameters (θ), we could try iterative cycle of max like fitting, rejecting >X sigma outliers, re-fitting, re-rejecting, etc until no X sigma outliers left

```
j = 0; cutoff = 4;
iteration_j = data;
μ_j = Mean[iteration_j[[All, 2]]];
figure_j = ErrorListPlot[iteration_j[[All, {1, 2, 3}]], PlotRange → {{0, 1}, {-1, 1}},
  Frame → True, FrameLabel → {"x", "y"}, ImageSize → {600, 400},
PlotStyle → Lighter[Black], BaseStyle → {FontFamily → "Helvetica", FontSize → 24},
Axes → None, GridLines → {{}, {{μ_j, Directive[Darker[Green], Thickness[0.005]]}}}]
```

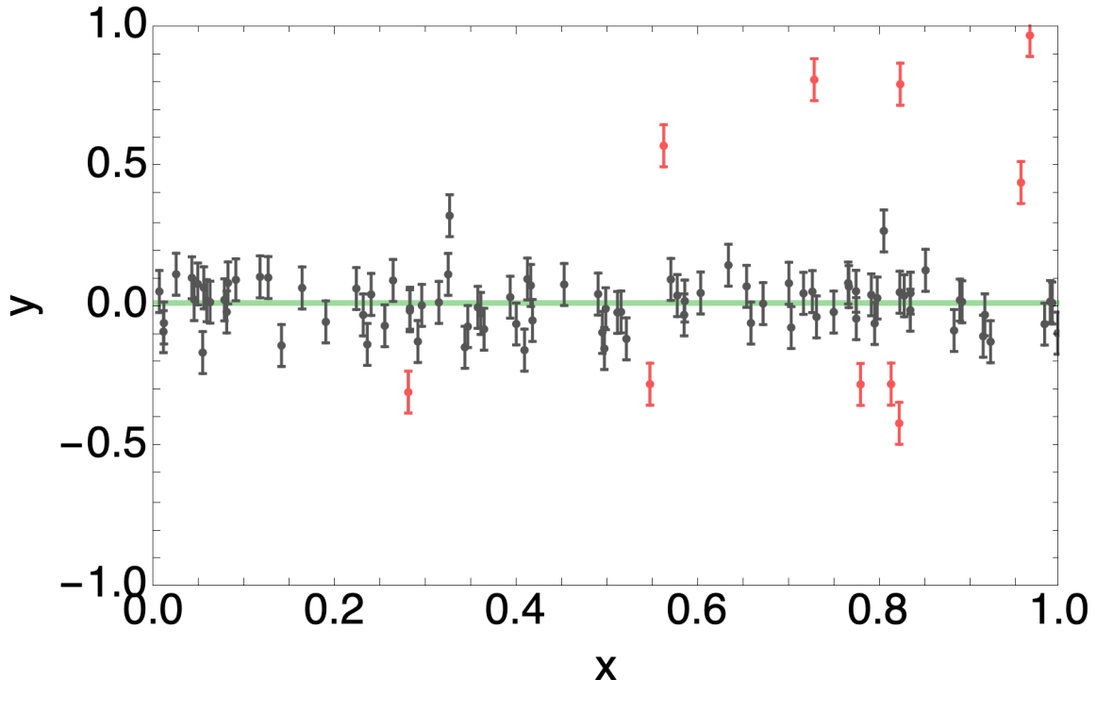


detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1A: if we believe we know the functional form of the model, but we don't know the model parameters (θ), we could try iterative cycle of max like fitting, rejecting >Z sigma outliers, re-fitting, re-rejecting, etc until no Z sigma outliers left

```
j = 1;
iteration_j = Select[iteration_(j-1), Abs[(#[[2]] - μ_(j-1))/#[[3]]] < cutoff &];
outliers_j = Select[iteration_(j-1), ! MemberQ[iteration_j, #] &];
μ_j = Mean[iteration_j[[All, 2]]];
figure_j = ErrorListPlot[{iteration_j[[All, {1, 2, 3}]], outliers_j[[All, {1, 2, 3}]]},
  PlotRange → {{0, 1}, {-1, 1}}, Frame → True, FrameLabel → {"x", "y"},
  ImageSize → {600, 400}, PlotStyle → {Lighter[Black], Lighter[Red]},
  BaseStyle → {FontFamily → "Helvetica", FontSize → 24}, Axes → None,
  GridLines → {{}, {{μ_j, Directive[Darker[Green], Thickness[0.005]]}}}]
```



$$\left| \frac{y_i - f(x_i)}{\sigma_i} \right| > Z$$

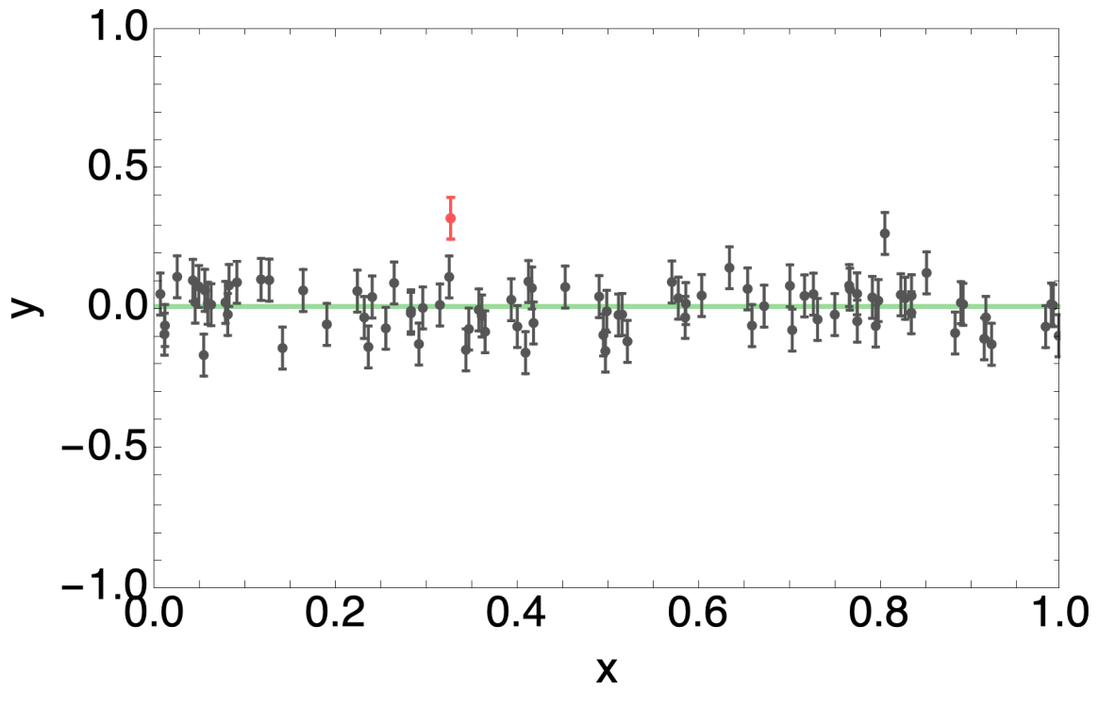
flags outliers

detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1A: if we believe we know the functional form of the model, but we don't know the model parameters (θ), we could try iterative cycle of max like fitting, rejecting >Z sigma outliers, re-fitting, re-rejecting, etc until no Z sigma outliers left

```
j = 2;
iteration_j = Select[iteration_(j-1), Abs[(#[[2]] - μ_(j-1))/#[[3]]] < cutoff &];
outliers_j = Select[iteration_(j-1), ! MemberQ[iteration_j, #] &];
μ_j = Mean[iteration_j[[All, 2]]];
figure_j = ErrorListPlot[{iteration_j[[All, {1, 2, 3}]], outliers_j[[All, {1, 2, 3}]]},
  PlotRange → {{0, 1}, {-1, 1}}, Frame → True, FrameLabel → {"x", "y"},
  ImageSize → {600, 400}, PlotStyle → {Lighter[Black], Lighter[Red]},
  BaseStyle → {FontFamily → "Helvetica", FontSize → 24}, Axes → None,
  GridLines → {{}, {{μ_j, Directive[Darker[Green], Thickness[0.005]]}}}]
```



$$\left| \frac{y_i - f(x_i)}{\sigma_i} \right| > Z$$

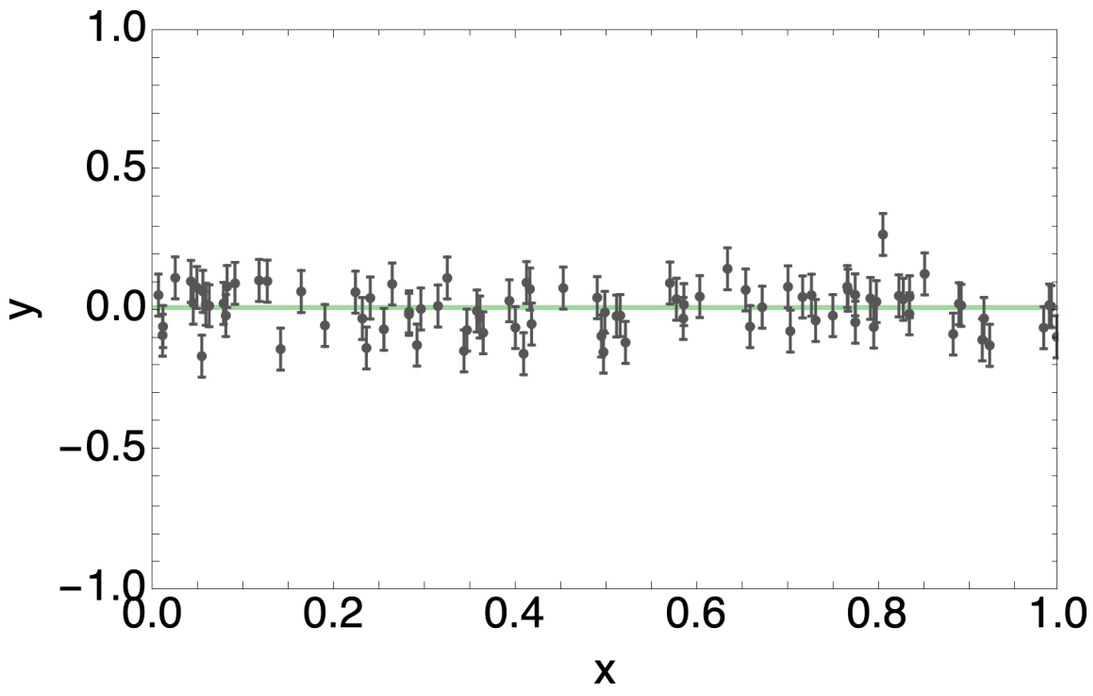
flags outliers

detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1A: if we believe we know the functional form of the model, but we don't know the model parameters (θ), we could try iterative cycle of max like fitting, rejecting >Z sigma outliers, re-fitting, re-rejecting, etc until no Z sigma outliers left

```
j = 3;
iteration_j = Select[iteration_(j-1), Abs[(#[[2]] - μ_(j-1))/#[[3]]] < cutoff &];
outliers_j = Select[iteration_(j-1), ! MemberQ[iteration_j, #] &];
μ_j = Mean[iteration_j[[All, 2]]];
figure_j = ErrorListPlot[iteration_j[[All, {1, 2, 3}]], PlotRange → {{0, 1}, {-1, 1}},
  Frame → True, FrameLabel → {"x", "y"}, ImageSize → {600, 400},
  PlotStyle → {Lighter[Black]}, BaseStyle → {FontFamily → "Helvetica", FontSize → 24},
  Axes → None, GridLines → {{}, {{μ_j, Directive[Darker[Green], Thickness[0.005]]}}}]
```



$$\left| \frac{y_i - f(x_i)}{\sigma_i} \right| > Z$$

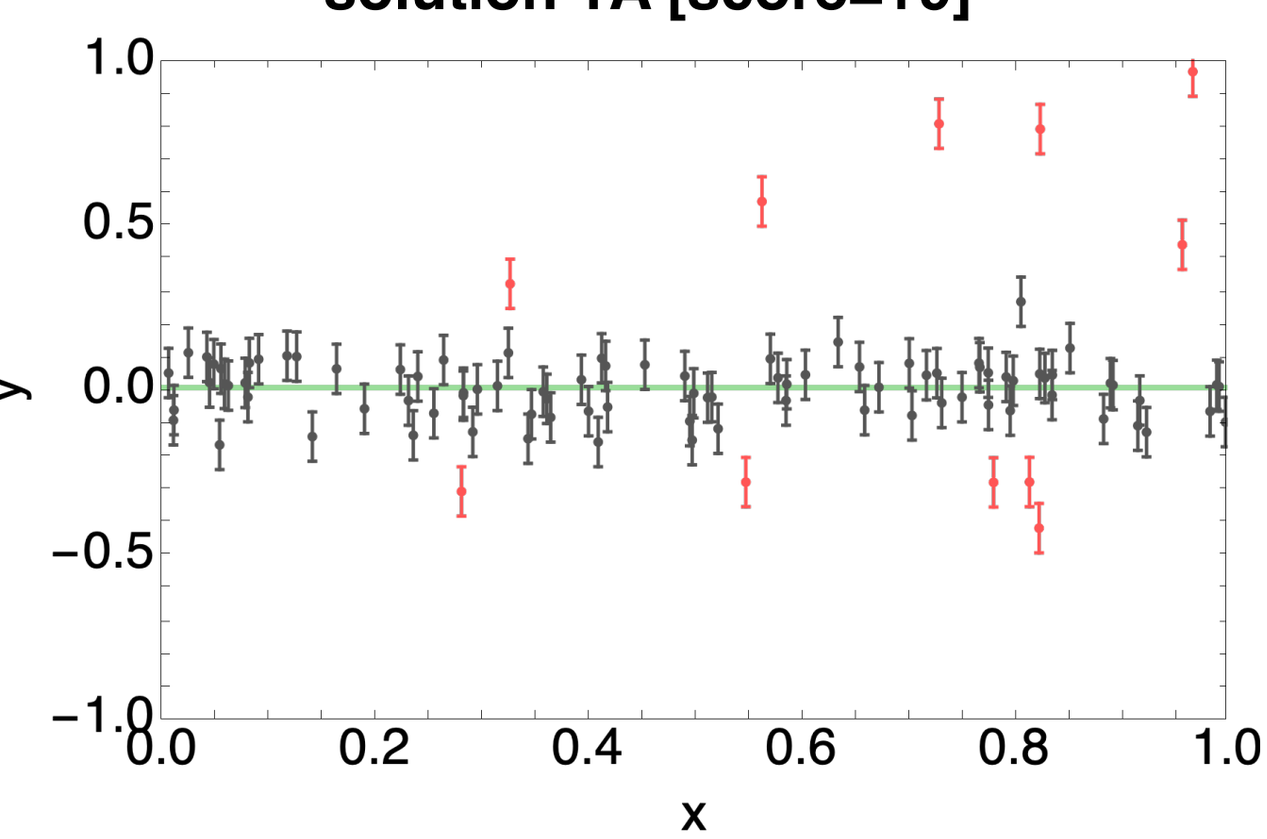
flags outliers

detecting outliers

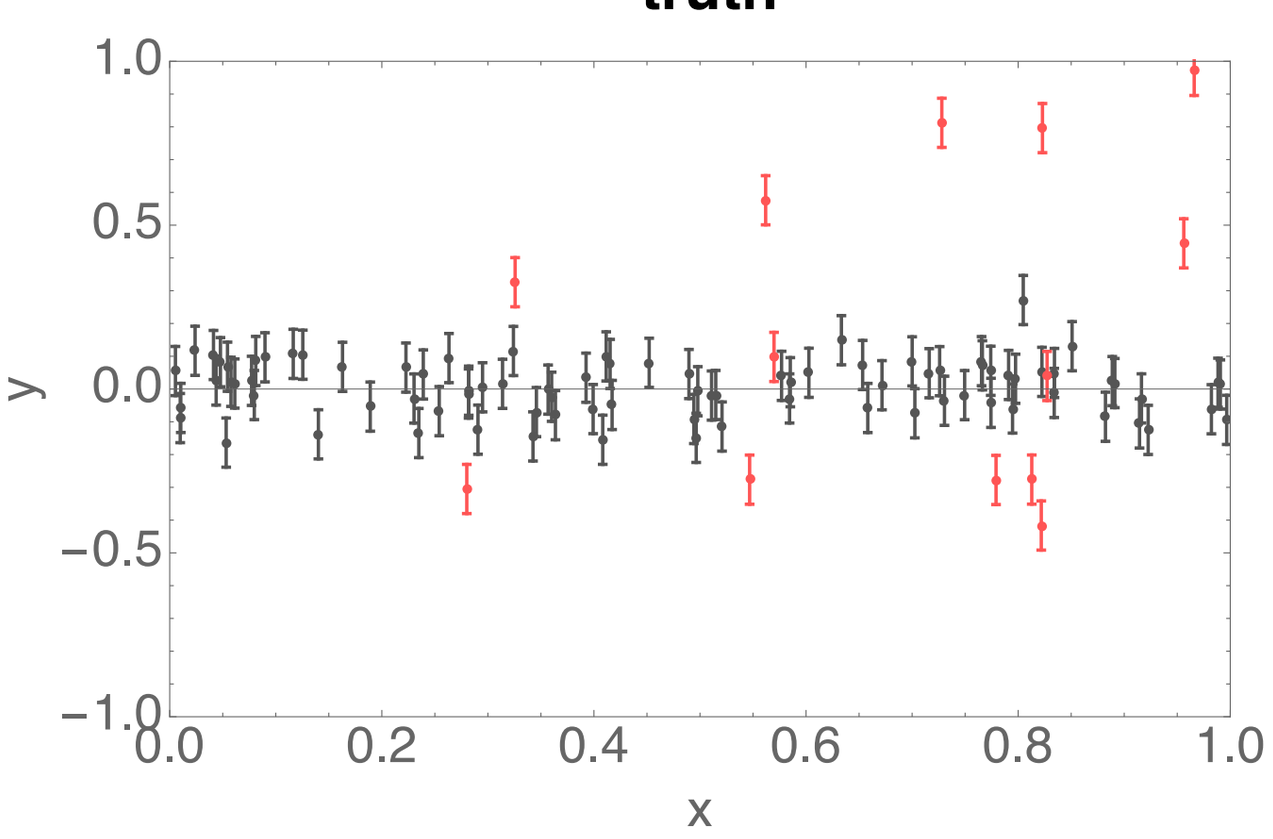
problem 1: we usually don't know the true f[x]!

solution 1A: if we believe we know the functional form of the model, but we don't know the model parameters (θ), we could try iterative cycle of max like fitting, rejecting >Z sigma outliers, re-fitting, re-rejecting, etc until no Z sigma outliers left

solution 1A [score=+9]



truth

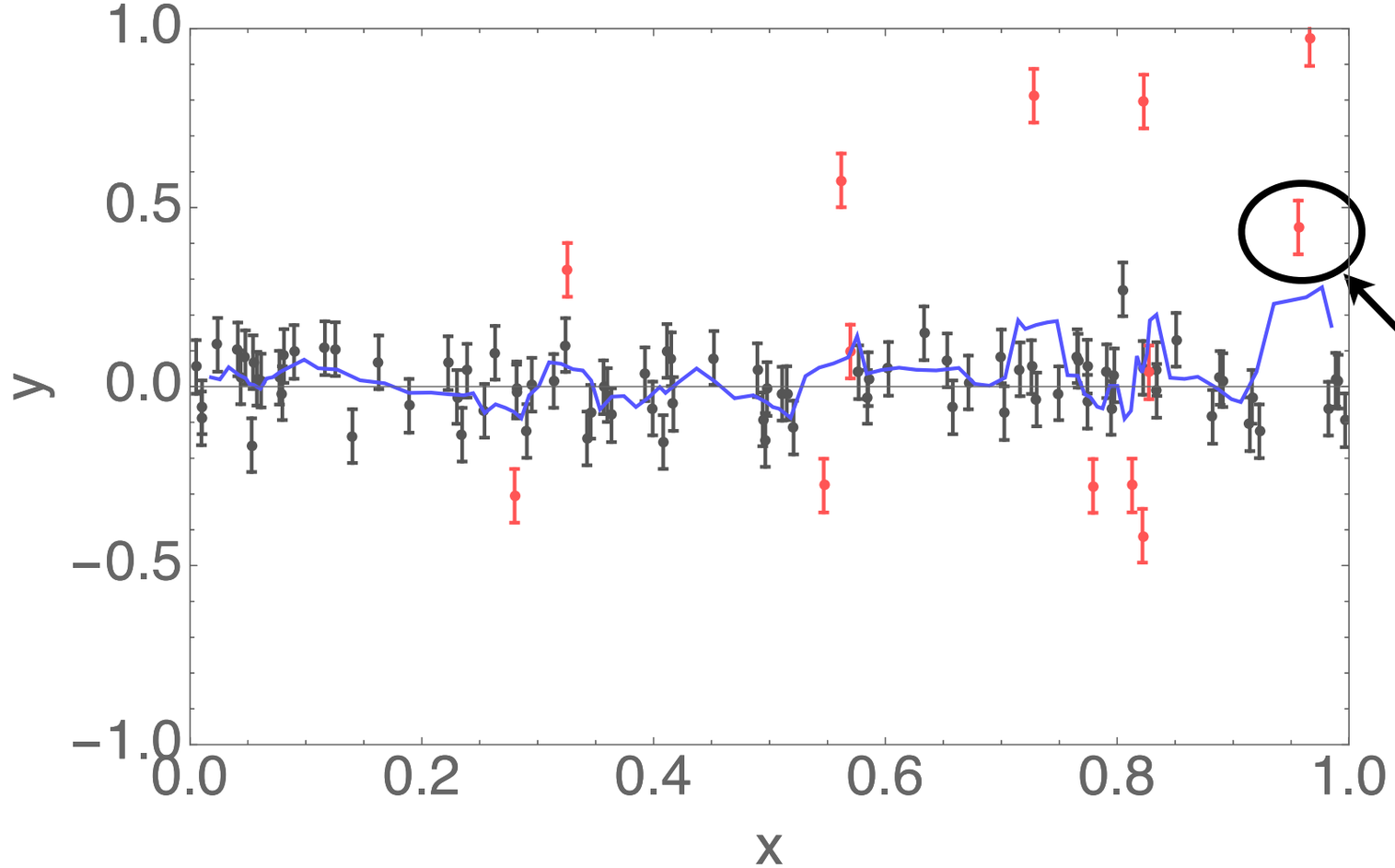


detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1B: can we define some kind of ***heuristic model*** using the data, for the sake of rejecting outliers?

e.g. take a moving average sliding along every 5 points



inferences made using data featuring outliers are flawed

thus the moving average is flawed! some outliers will be missed using this method

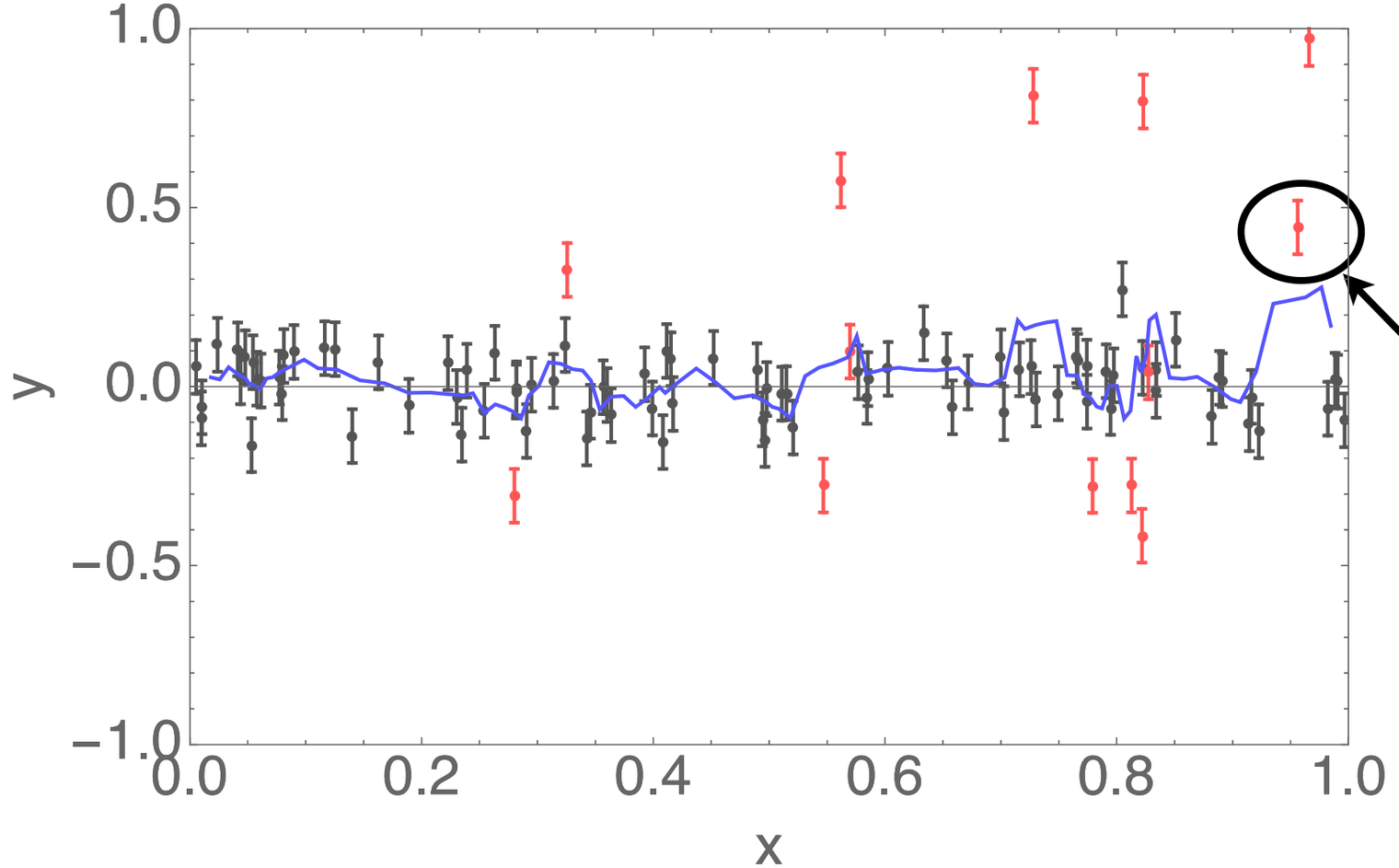
detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1B: can we define some kind of ***heuristic model*** using the data, for the sake of rejecting outliers?

"smoothing" **"kernel"** ***"bandwidth"***

e.g. take a moving average sliding along every 5 points



inferences made using data featuring outliers are flawed

thus the moving average is flawed! some outliers will be missed using this method

could iterate… but there's a better solution…

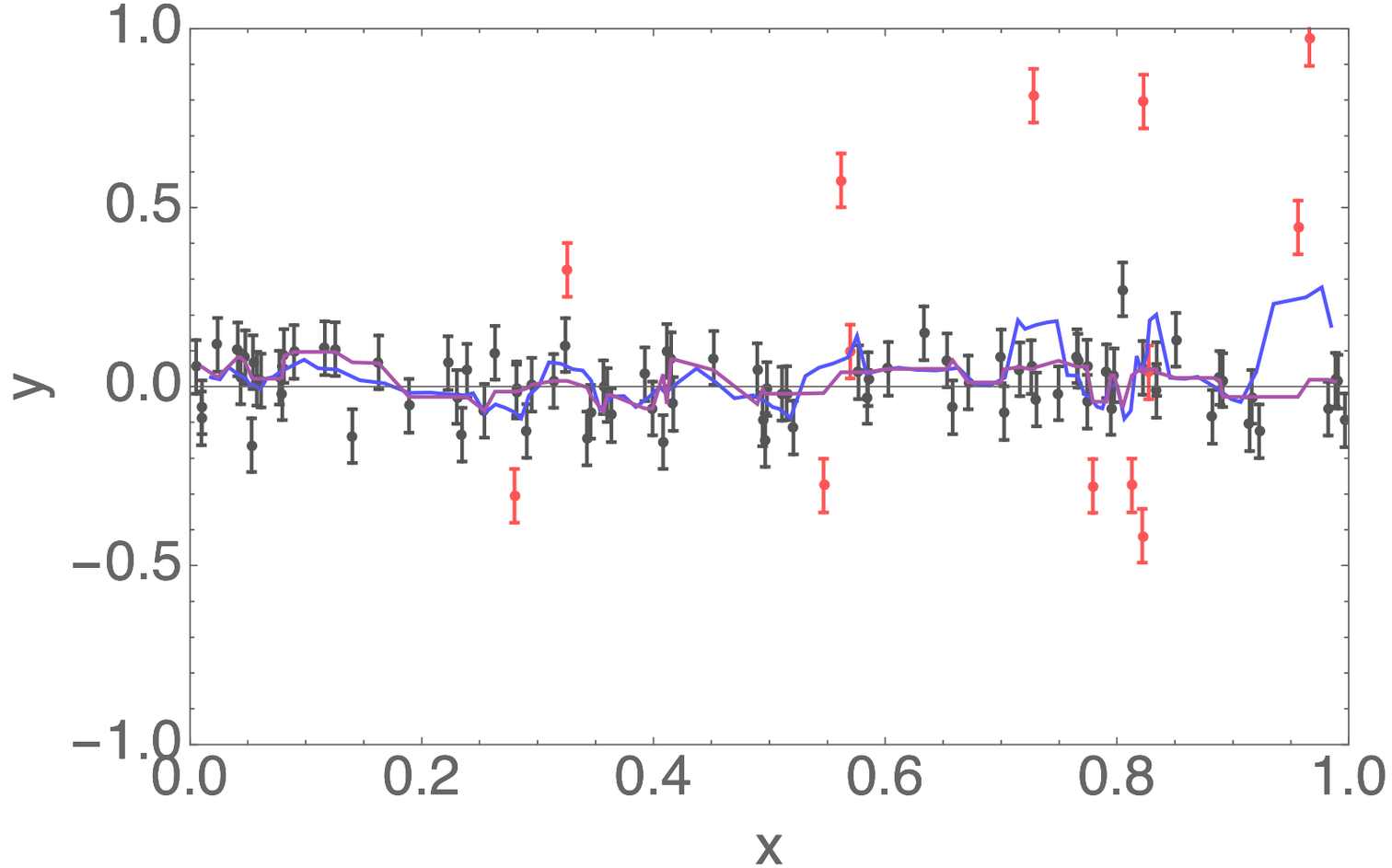
detecting outliers

problem 1: we usually don't know the true f[x]!

solution 1B: can we define some kind of ***heuristic model*** using the data, for the sake of rejecting outliers?

"kernel"

median statistics are less sensitive to outliers



whilst the moving median is still affected by outliers, it is less influenced

if we don't know f[x], then moving median is the most common strategy to define it for the purpose of outlier detection

detecting outliers

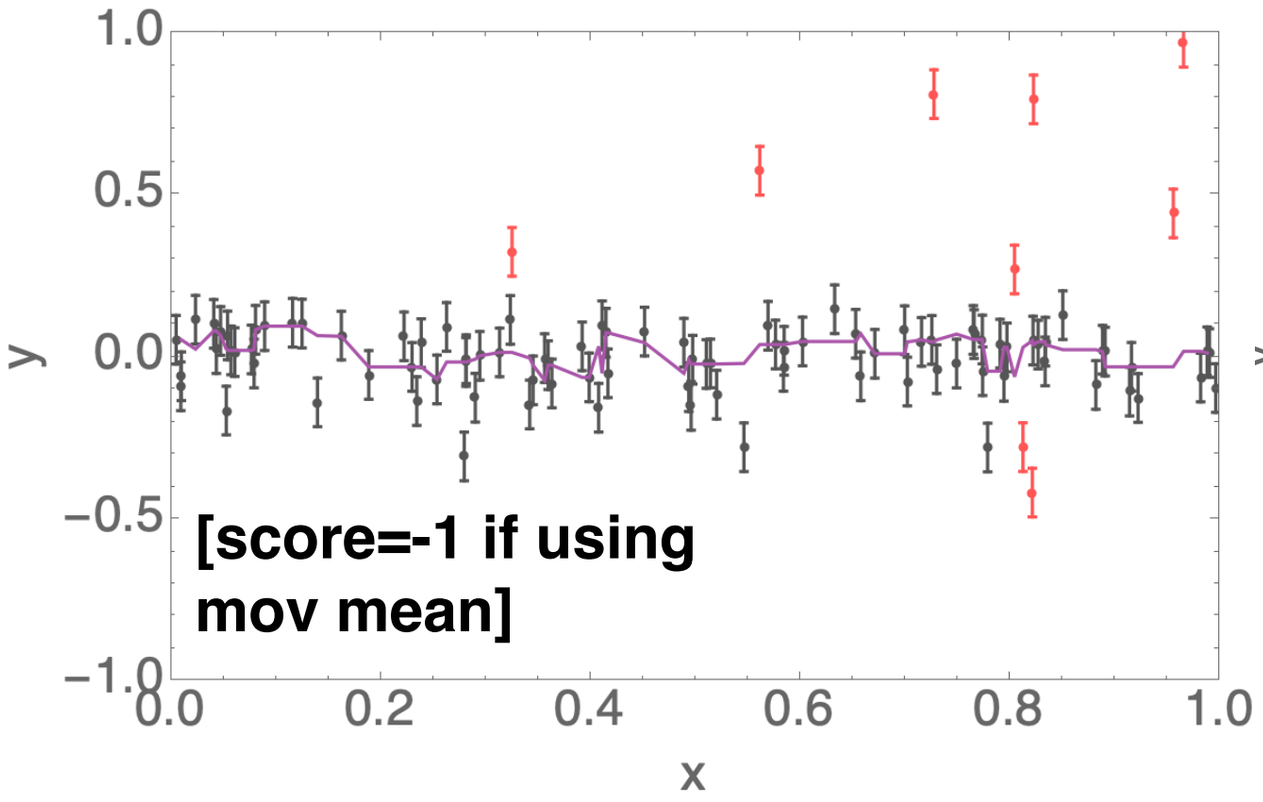
problem 1: we usually don't know the true f[x]!

solution 1B: can we define some kind of ***heuristic model*** using the data, for the sake of rejecting outliers?

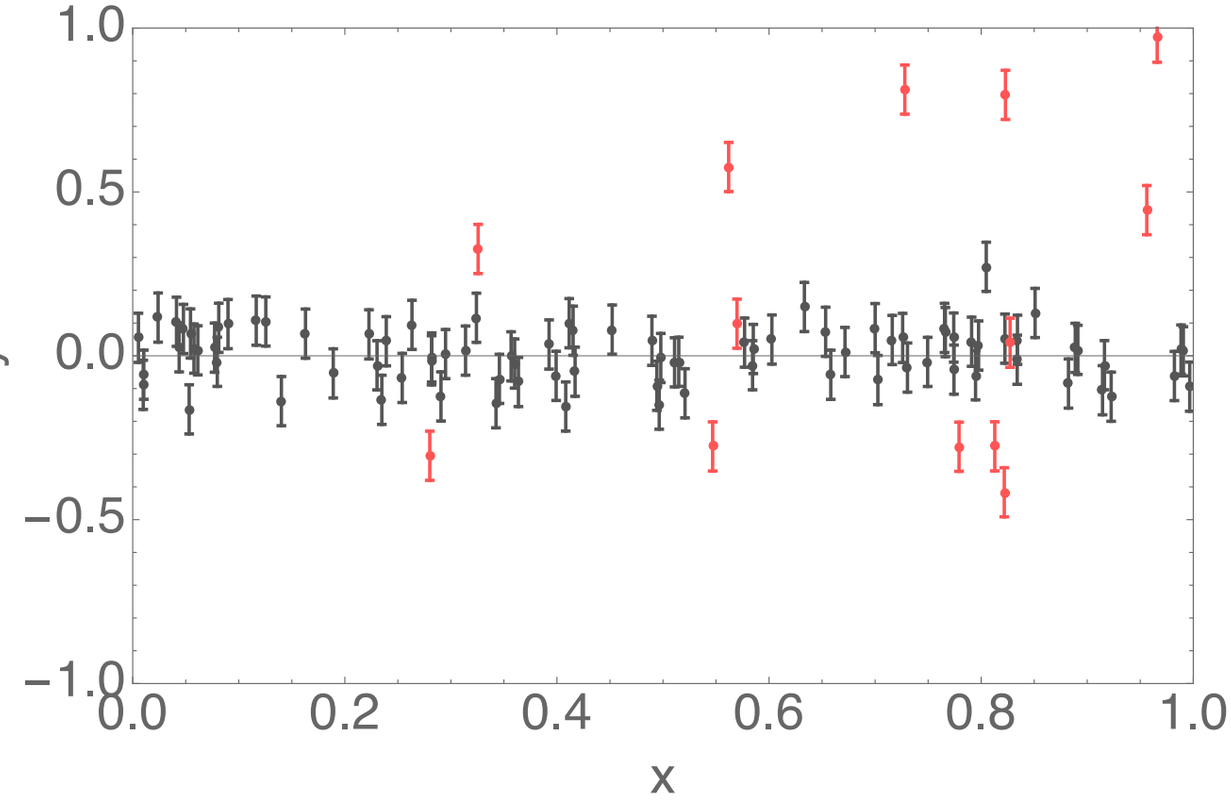
note: missed points are buried deep in the Gaussian core and thus hardly bias us

median statistics are less sensitive to outliers

solution 1B [score=+2]



truth



median statistics are less sensitive to outliers

y = {5,6,3,1,6,5,9,8,625}

Mean of **y** = 74.2

Mean of **y** without outlier = 5.4

Median of **y** = 6

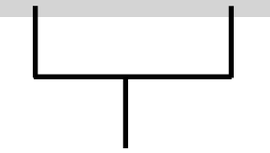
Median of **y** without outlier = 5.5

if fraction of data points belonging to outliers is not large (less than 10-20%) then median statistics perform well

e.g. a moving median with bandwidth = 3

t = {t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}}

y = {y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}}



median of these 3

t' = { t_2

y' = { y_1||y_3

take a window of *n* points, compute the median of those *n* *x*'s and *y*'s and that defines an element in your movingmedian list

shift window along by just 1 point and repeat

e.g. a moving median with bandwidth = 3

t = {t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}}

y = {y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}}

median of these 3

t' = { t_2 t_3

y' = { y_1‖y_3 y_3‖y_4

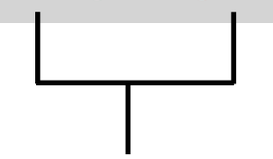
take a window of *n* points, compute the median of those *n* *x*'s and *y*'s and that defines an element in your movingmedian list

shift window along by just 1 point and repeat

e.g. a moving median with bandwidth = 3

t = {t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}}

y = {y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}}



median of these 3

t' = { t_2 t_3 t_4

y' = { y_1||y_3 y_3||y_4 y_3||y_4

take a window of *n* points, compute the median of those *n* *x*'s and *y*'s and that defines an element in your movingmedian list

shift window along by just 1 point and repeat

e.g. a moving median with bandwidth = 3

t = {t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}}

y = {y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}}

median of these 3

len(**t**)=len(**y**)=20

len(**t'**)=len(**y'**)=18

t' = { t_2 t_3 t_4 t_5 t_6 t_7 t_8 … t_{19} }

y' = { y_1||y_3 y_3||y_4 y_3||y_4 y_4||y_6 y_6||y_7 y_6||y_7||y_8 y_7||y_8||y_9 y_{18}||y_{19} }

take a window of *n* points, compute the median of those *n* *x*'s and *y*'s and that defines an element in your movingmedian list

shift window along by just 1 point and repeat

moving windows will cut off floor(n/2) points either side of the original data

e.g. a moving median with bandwidth = 3

$\mathbf{t} = \{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}\}$

$\mathbf{y} = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}\}$



median of these 3

$\mathbf{t'} = \{ \; t_2$

$\mathbf{y'} = \{ y_1 \| y_3$

e.g. a moving median with bandwidth = 3

$\mathbf{t} = \{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}\}$

$\mathbf{y} = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}\}$

median of these 3

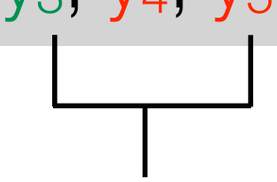
$\mathbf{t'} = \{ \quad t_2 \qquad t_3$

$\mathbf{y'} = \{ y_1 \| y_3 \quad y_2 \| y_4$

e.g. a moving median with bandwidth = 3

$\mathbf{t} = \{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}\}$

$\mathbf{y} = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}\}$



median of these 3

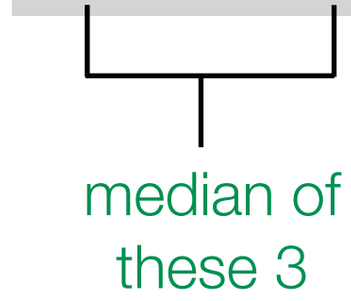
$\mathbf{t'} = \{ \quad t_2 \qquad t_3 \qquad t_4$

$\mathbf{y'} = \{ y_1 \| y_3 \quad y_2 \| y_4 \quad y_4 \| y_5$

e.g. a moving median with bandwidth = 3

$\mathbf{t} = \{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}, t_{14}, t_{15}, t_{16}, t_{17}, t_{18}, t_{19}, t_{20}\}$

$\mathbf{y} = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_{10}, y_{11}, y_{12}, y_{13}, y_{14}, y_{15}, y_{16}, y_{17}, y_{18}, y_{19}, y_{20}\}$



$\mathbf{t'} = \{ \; t_2 \quad t_3 \quad t_4 \quad t_5 \quad t_6 \quad t_7 \quad t_8 \quad \ldots \quad t_{19} \; \}$

$\mathbf{y'} = \{ \; y_1 \| y_3 \;\; y_2 \| y_4 \;\; y_4 \| y_5 \;\; y_4 \| y_5 \;\; y_6 \| y_7 \;\; y_6 \| y_7 \| y_8 \;\; y_7 \| y_8 \| y_9 \quad\quad y_{18} \| y_{19} \; \}$

moving windows will cut off floor(n/2) points either side of the original data
=> we want to use a short window to avoid truncating too much data

localized clusters of outliers can defeat movmed
=> we want to use a wide window to avoid risk of a cluster overwhelming

$$\mathbf{t'} = \{\ t_2 \quad t_3 \quad t_4 \quad t_5 \quad t_6 \quad t_7 \quad t_8 \quad \ldots \quad t_{19}\ \}$$

$$\mathbf{y'} = \{\ y_1 \| y_3 \quad y_2 \| y_4 \quad y_4 \| y_5 \quad y_4 \| y_5 \quad y_6 \| y_7 \quad y_6 \| y_7 \| y_8 \quad y_7 \| y_8 \| y_9 \quad y_{18} \| y_{19}\ \}$$

don't ever try to treat your smoothed data as training set to a model!



smoothing = passing a moving window function through the data

bandwidth = width of the window used

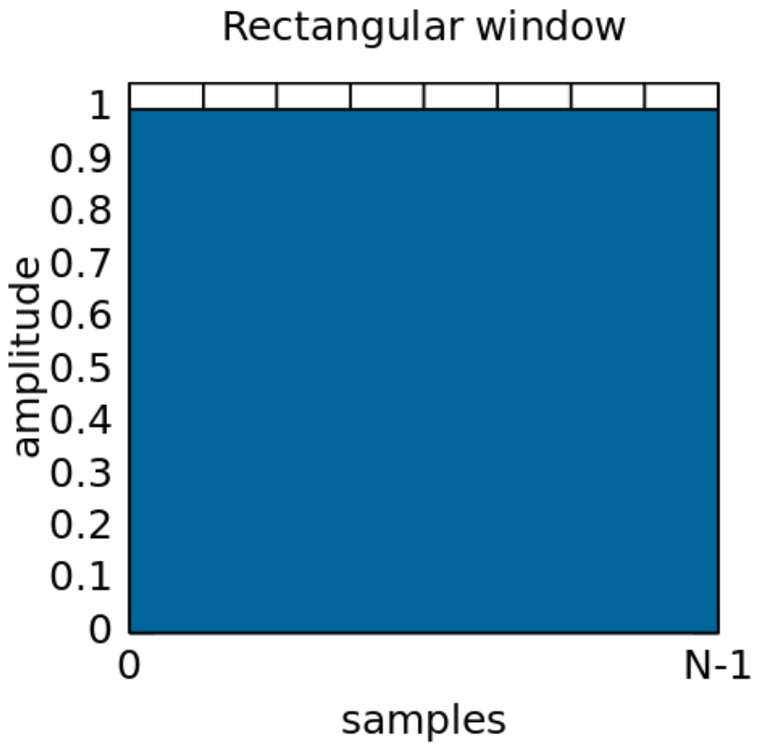
window = type of the weighting scheme to use for the kernel

kernel = defined by the type of the operation being executed (e.g. mean vs median) and the window used

window of a moving average/median is a *rectangular window*

bandwidth here is *N* (better to use N seconds than N points)

operator used by the moving mean kernel is a mean, whereas moving median uses a median



e.g. moving mean with rectangular window is explicitly doing this:

$$y'_j = \frac{\sum_{i=1}^{M} w_{j,i} y_i}{\sum_{i=1}^{M} w_{j,i}}$$

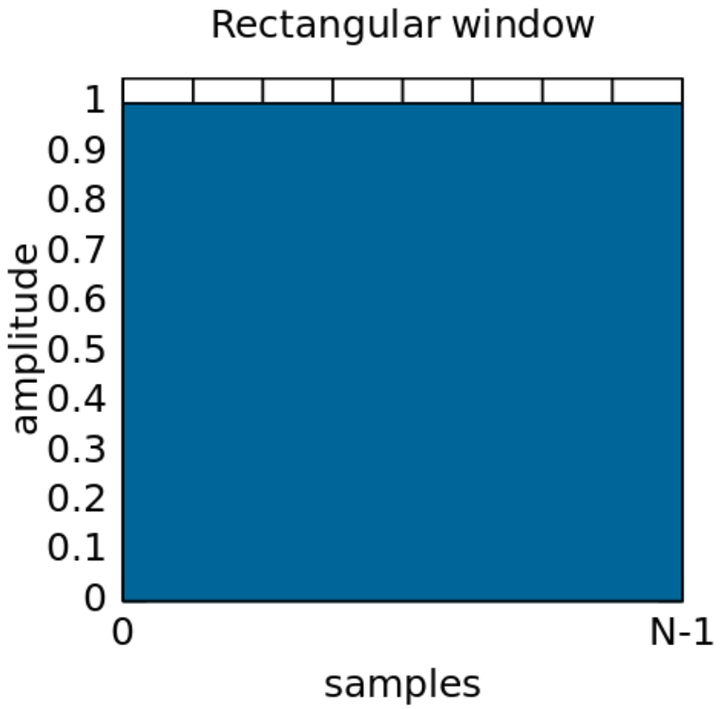
$$w_{i,j} = \begin{cases} 1 & \text{if } x_j - b/2 < x_i < x_j + b/2, \\ 0 & \text{otherwise}, \end{cases} \quad \Pi[x_j - b/2, x_j + b/2]$$

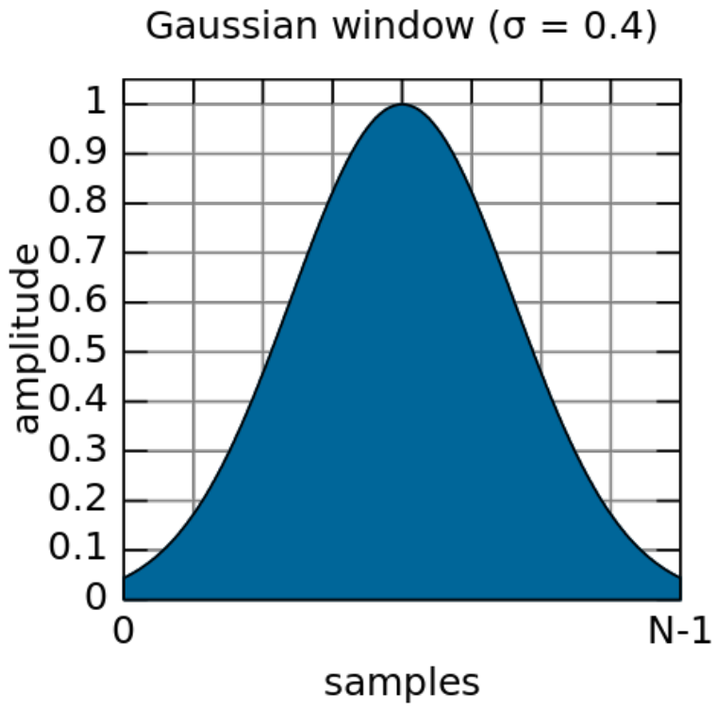
smoothing = passing a moving window function through the data

bandwidth = width of the window used

window = type of the weighting scheme to use for the kernel

kernel = defined by the type of the operation being executed (e.g. mean vs median) and the window used





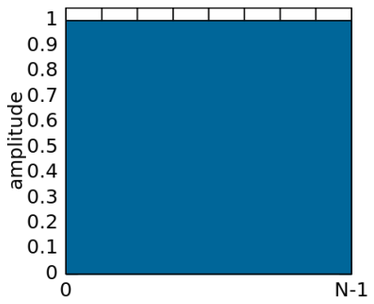
e.g. moving mean with rectangular window is explicitly doing this:

$$y'_j = \frac{\sum_{i=1}^{M} w_{j,i} y_i}{\sum_{i=1}^{M} w_{j,i}} \qquad w_{i,j} = \Pi[x_j - b/2, x_j + b/2]$$

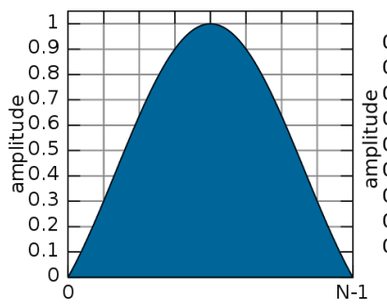
e.g. moving mean with Gaussian window is explicitly doing this:

$$y'_j = \frac{\sum_{i=1}^{M} w_{j,i} y_i}{\sum_{i=1}^{M} w_{j,i}} \qquad w_{i,j} = \frac{1}{\sqrt{2\pi} b} \exp\Big(- \frac{(x_j - x_i)^2}{2b^2} \Big)$$

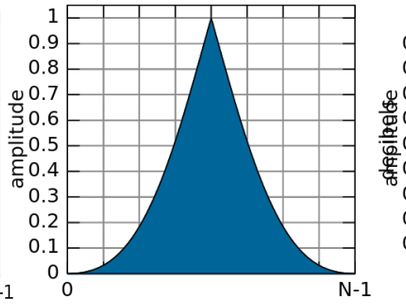
Rectangular window



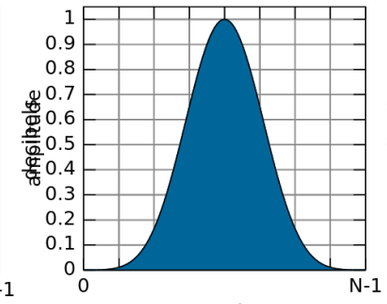
Lanczos window



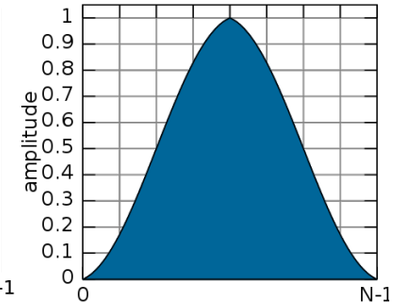
Hann–Poisson window (α = 2)



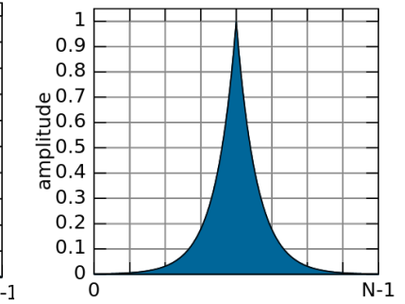
Planck–Bessel window (ε = 0.1, α = 4



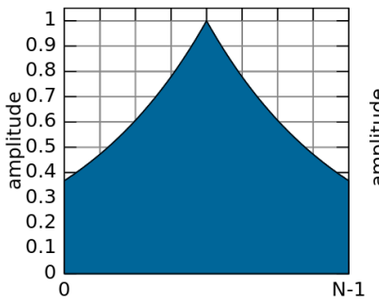
Bartlett–Hann window



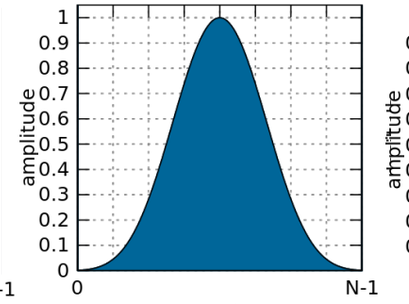
Exponential window (τ = (N/2)/(60/8.



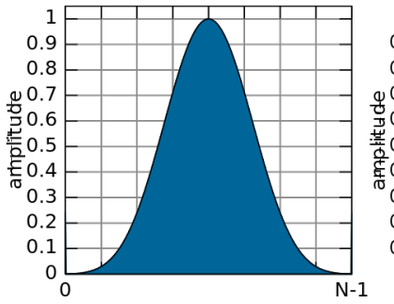
Exponential window (τ = N/2)



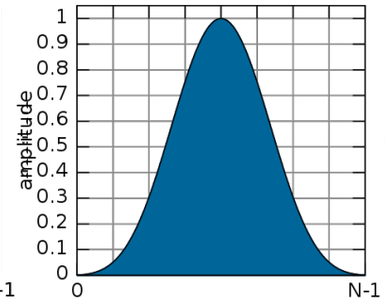
Ultraspherical window (μ = –0.5)



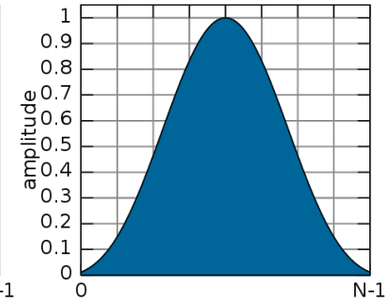
Dolph–Chebyshev window (α = 5)



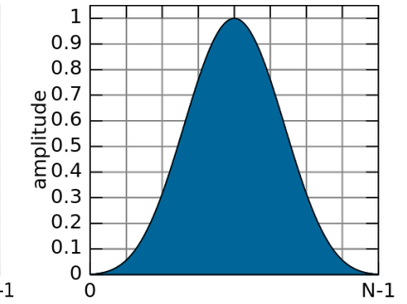
Kaiser window (α = 3)



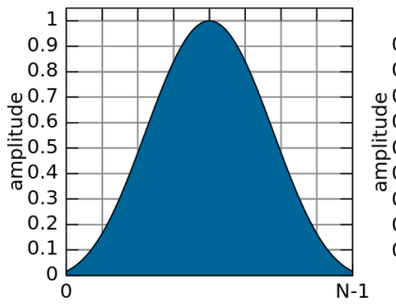
Kaiser window (α = 2)



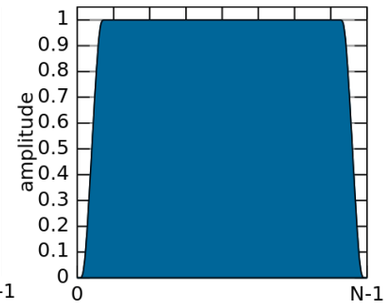
DPSS window (α = 3)



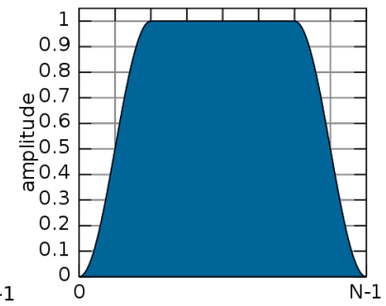
DPSS window (α = 2)



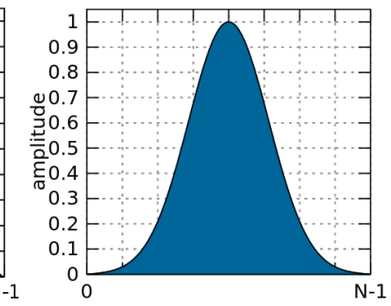
Planck-taper window (ε = 0.1)



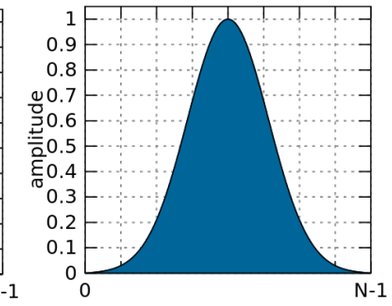
Tukey window (α = 0.5)



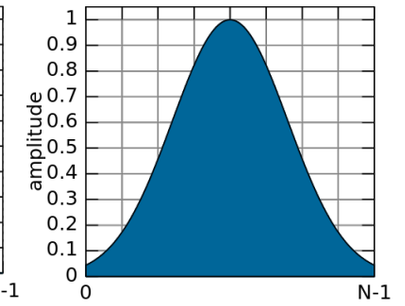
App. conf. Gaussian window (σₜ = 0.:



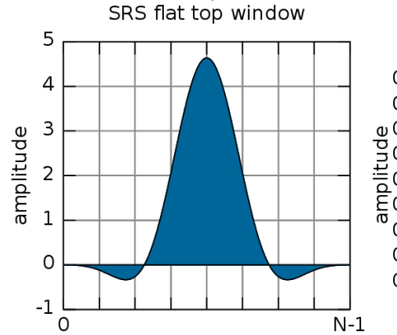
Confined Gaussian window (σₜ = 0.1



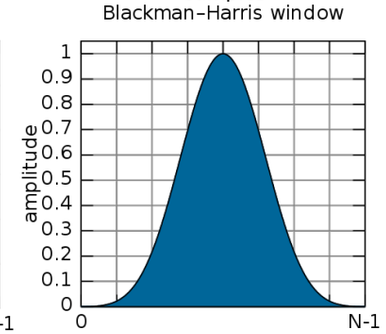
Gaussian window (σ = 0.4)



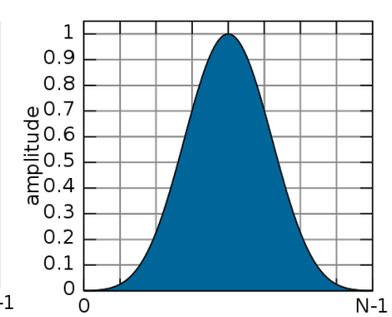
SRS flat top window



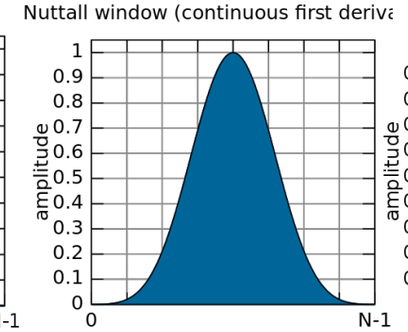
Blackman–Harris window



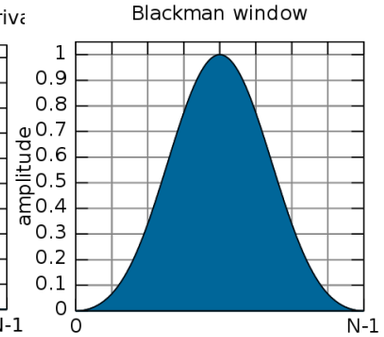
Blackman–Nuttall window



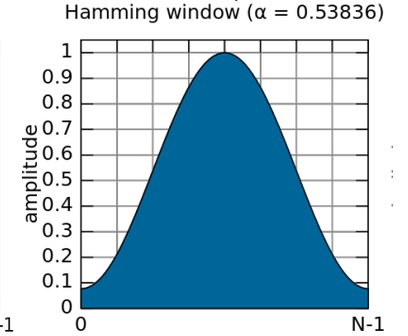
Nuttall window (continuous first deriva



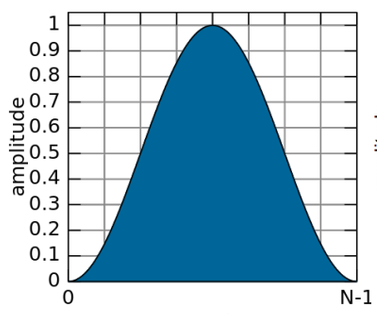
Blackman window



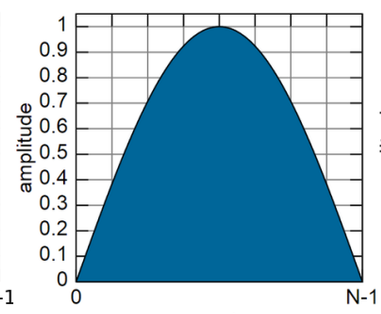
Hamming window (α = 0.53836)



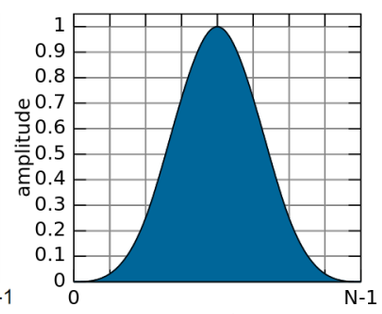
Hann window



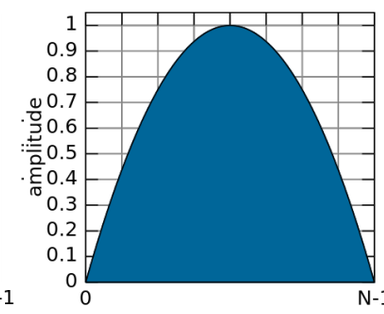
Sine window



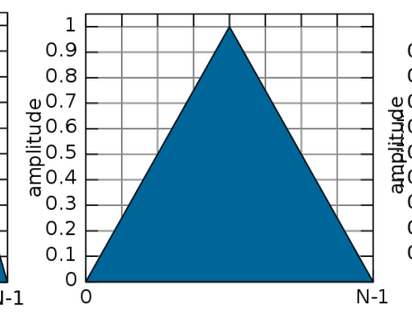
Parzen window



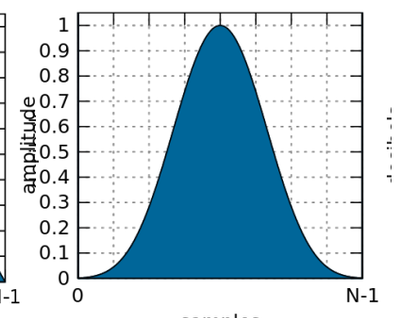
Welch window



Triangular window



Ultraspherical window (μ = –0.5)



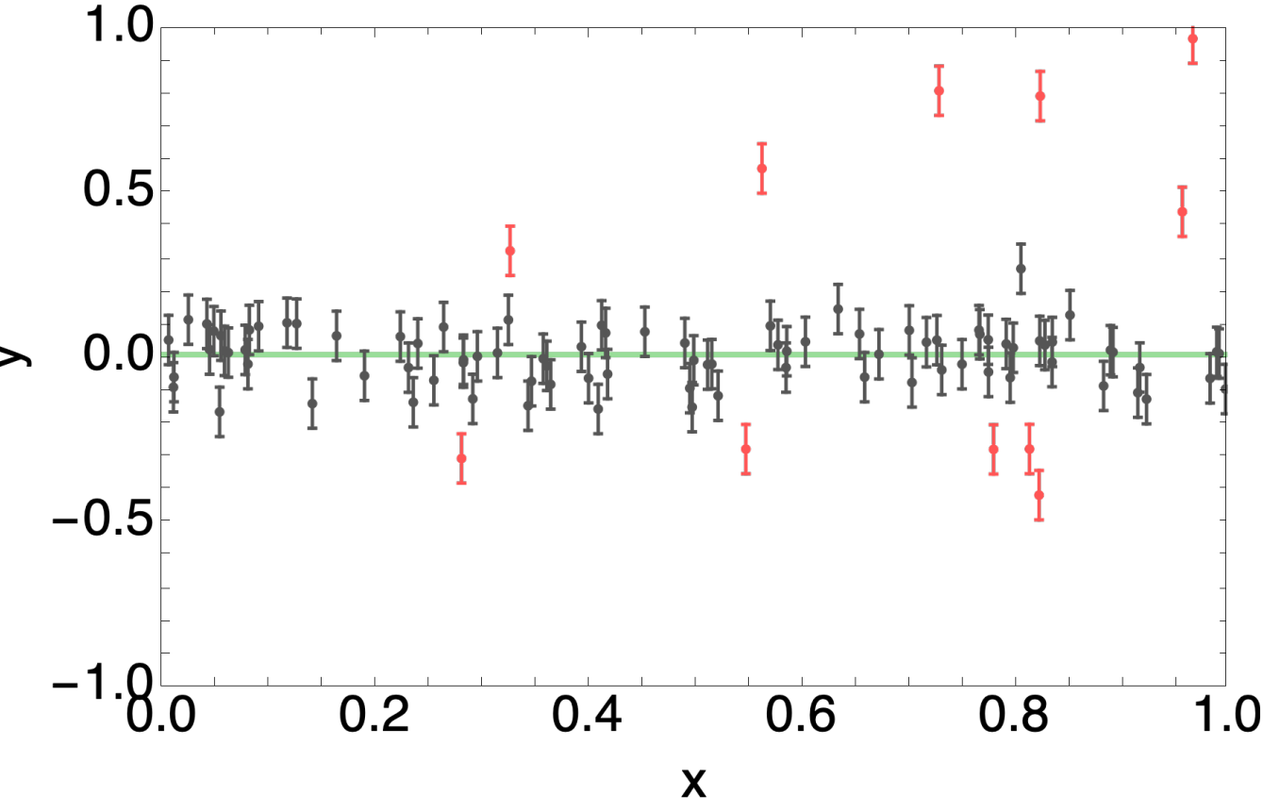
missed outliers

Even when we know f[x] form, our best score is imperfect!

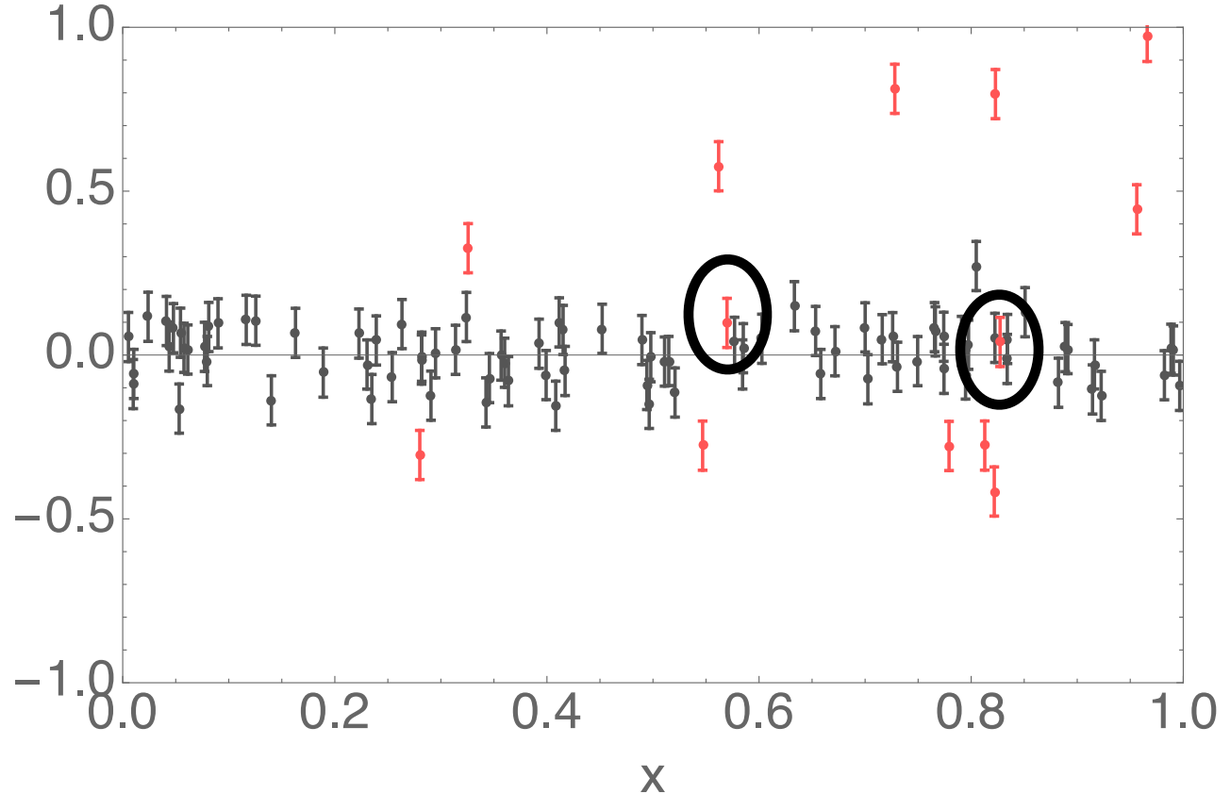
These two points are practically impossible to identify as outliers with current data, they do still matter and skew your inference :-(

But, there might be meta-data that flags them!
e.g. temperature/seeing/vibration/stray light

solution 1A [score=+9]

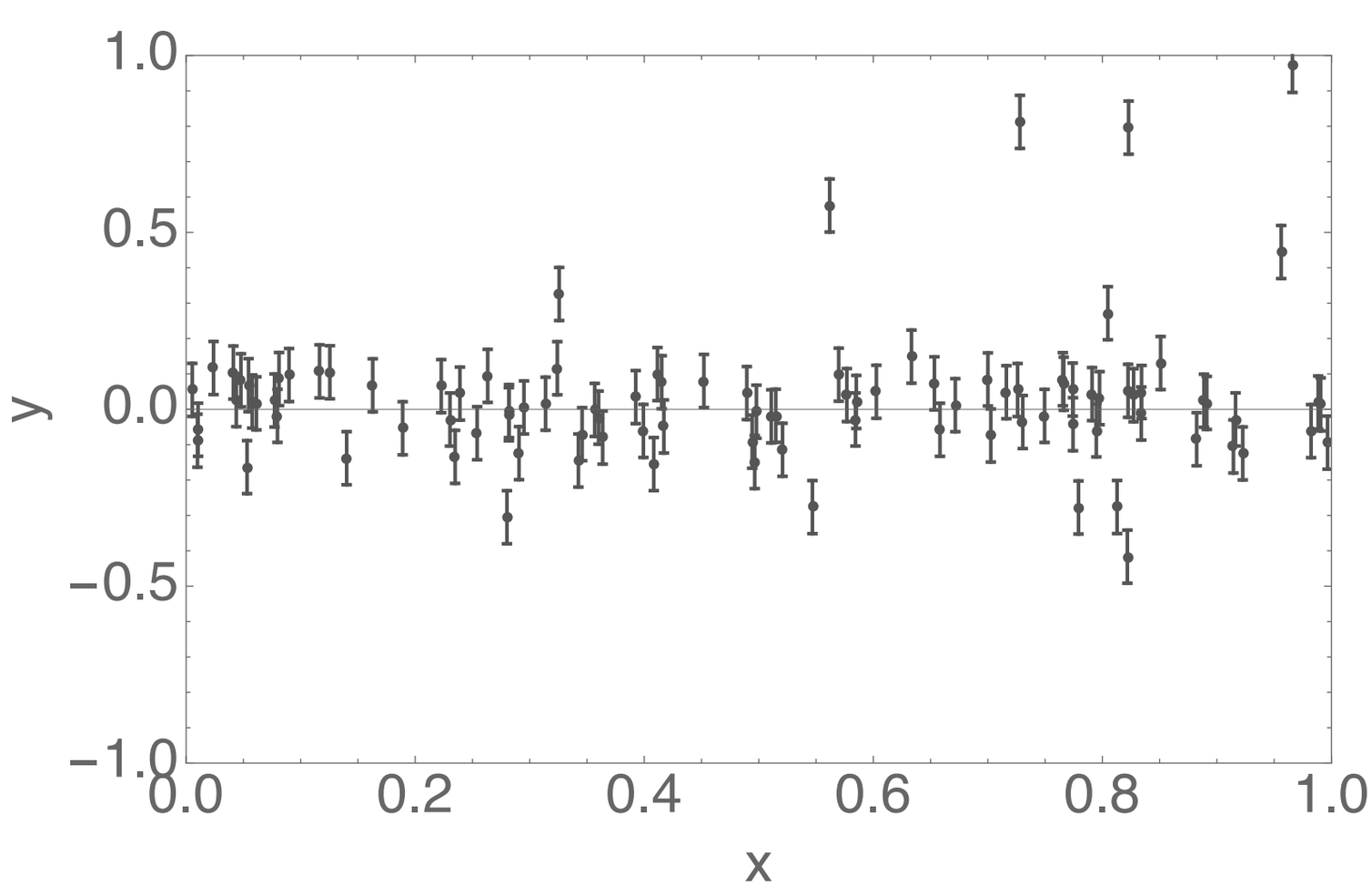


truth



detecting outliers

problem 2: we sometimes don't know the true errors, σ_y



detecting outliers

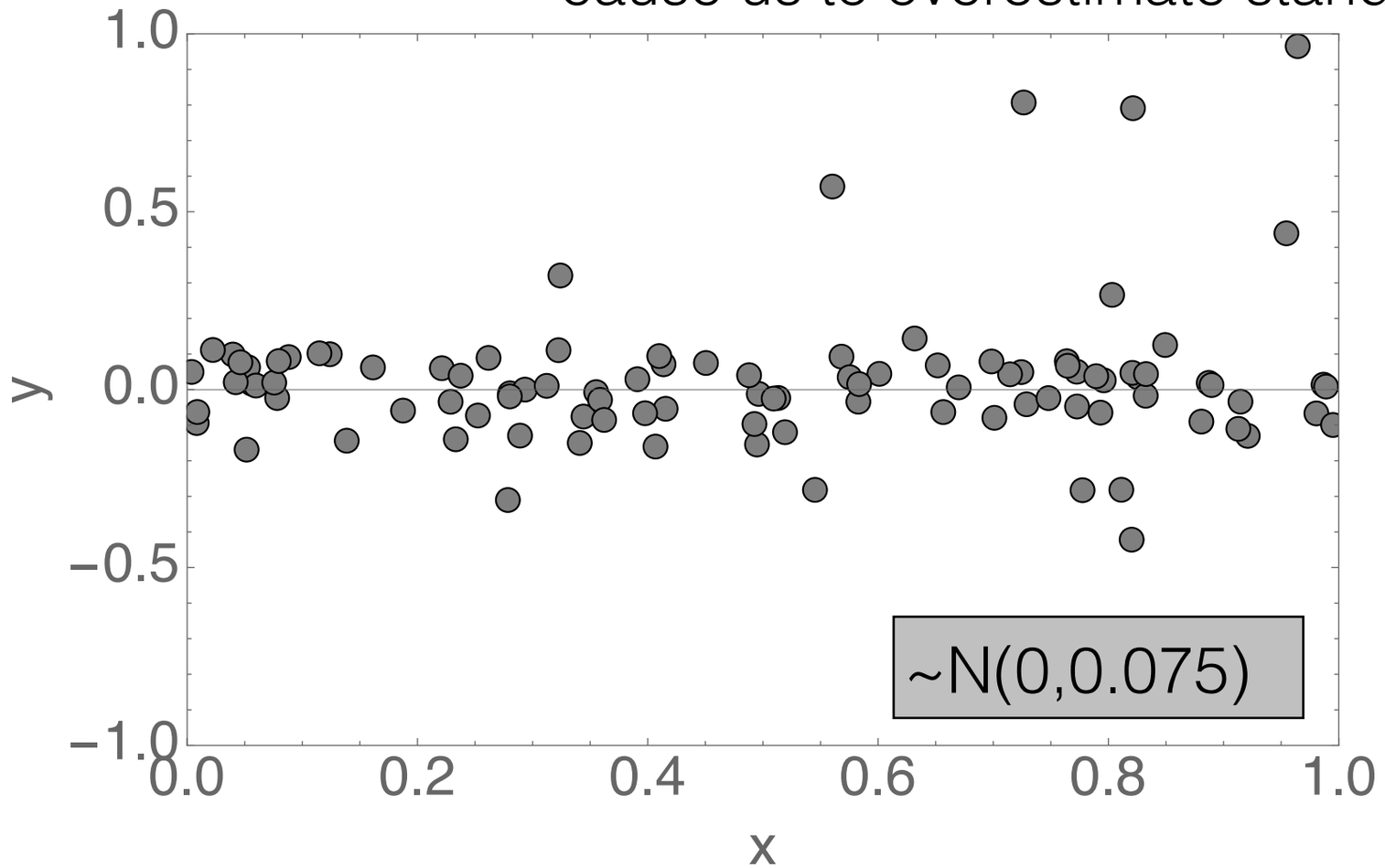
problem 2: we sometimes don't know the true errors, σ_y

if we don't know the errors, we have no choice but to estimate them from the data itself, but...

inferences made using data featuring outliers are flawed

standard deviation = 0.198

(here, the outliers increase the scatter and cause us to overestimate standard deviation)



detecting outliers

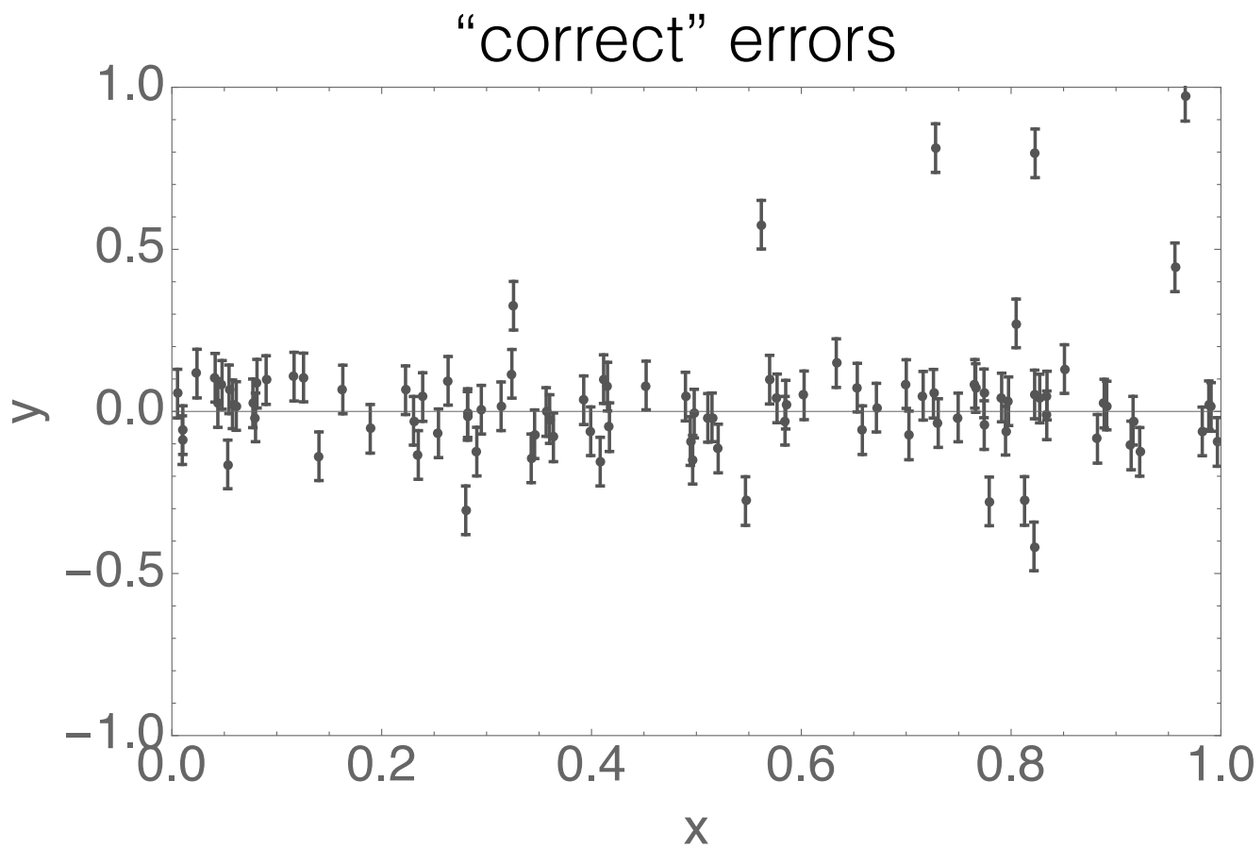
problem 2: we sometimes don't know the true errors, σ_y

if we don't know the errors, we have no choice but to estimate them from the data itself, but...

inferences made using data featuring outliers are flawed

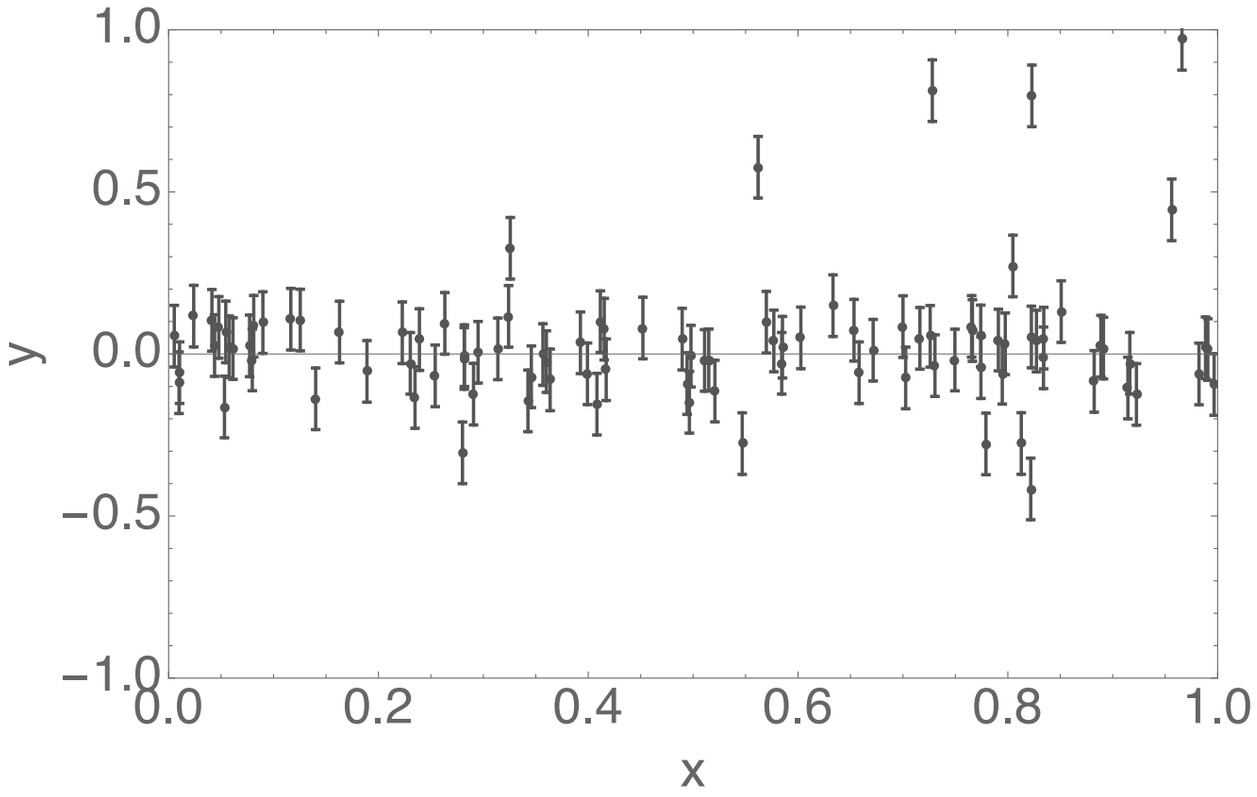
standard deviation = 0.198

(here, the outliers increase the scatter and cause us to overestimate standard deviation)





median absolute deviation (MAD) errors



once again, median statistics can help

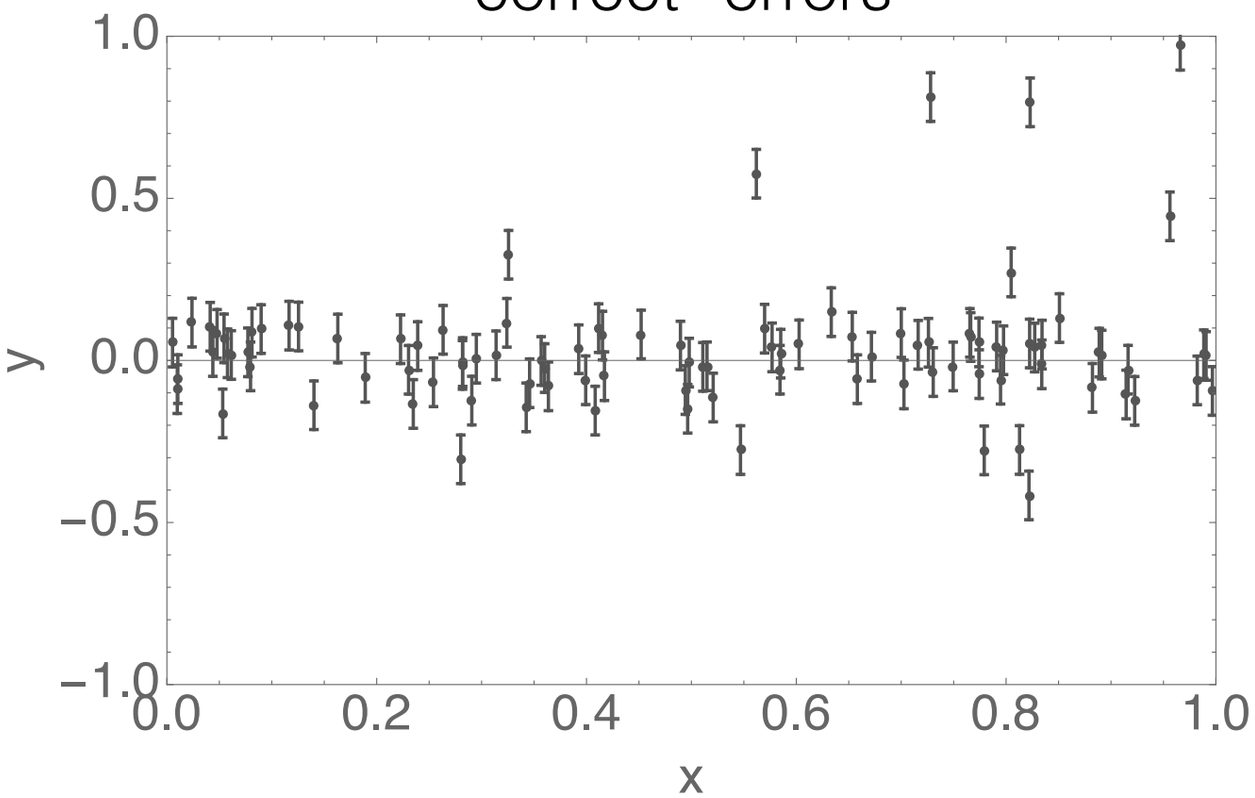
standard deviation = 0.198

1.4826*MAD = 0.095

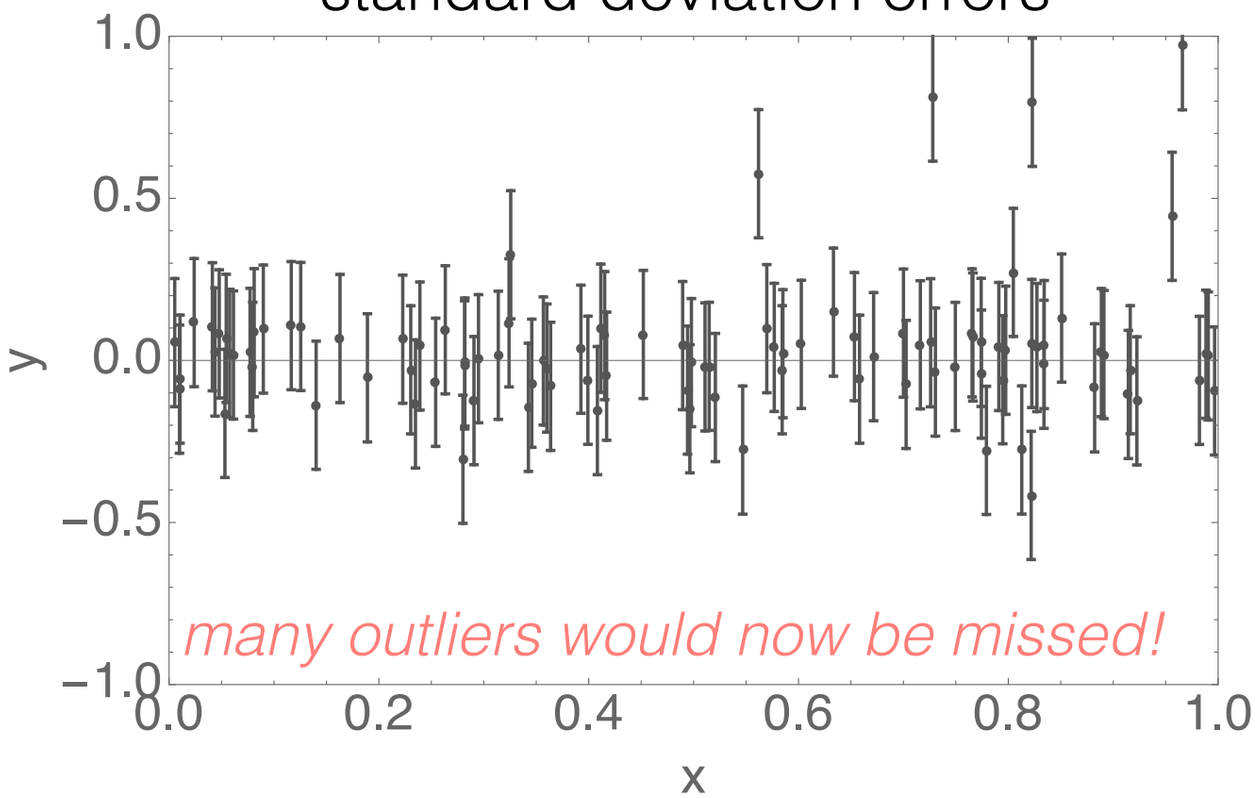
“correct” = 0.075

but we implicitly assume homoscedastic

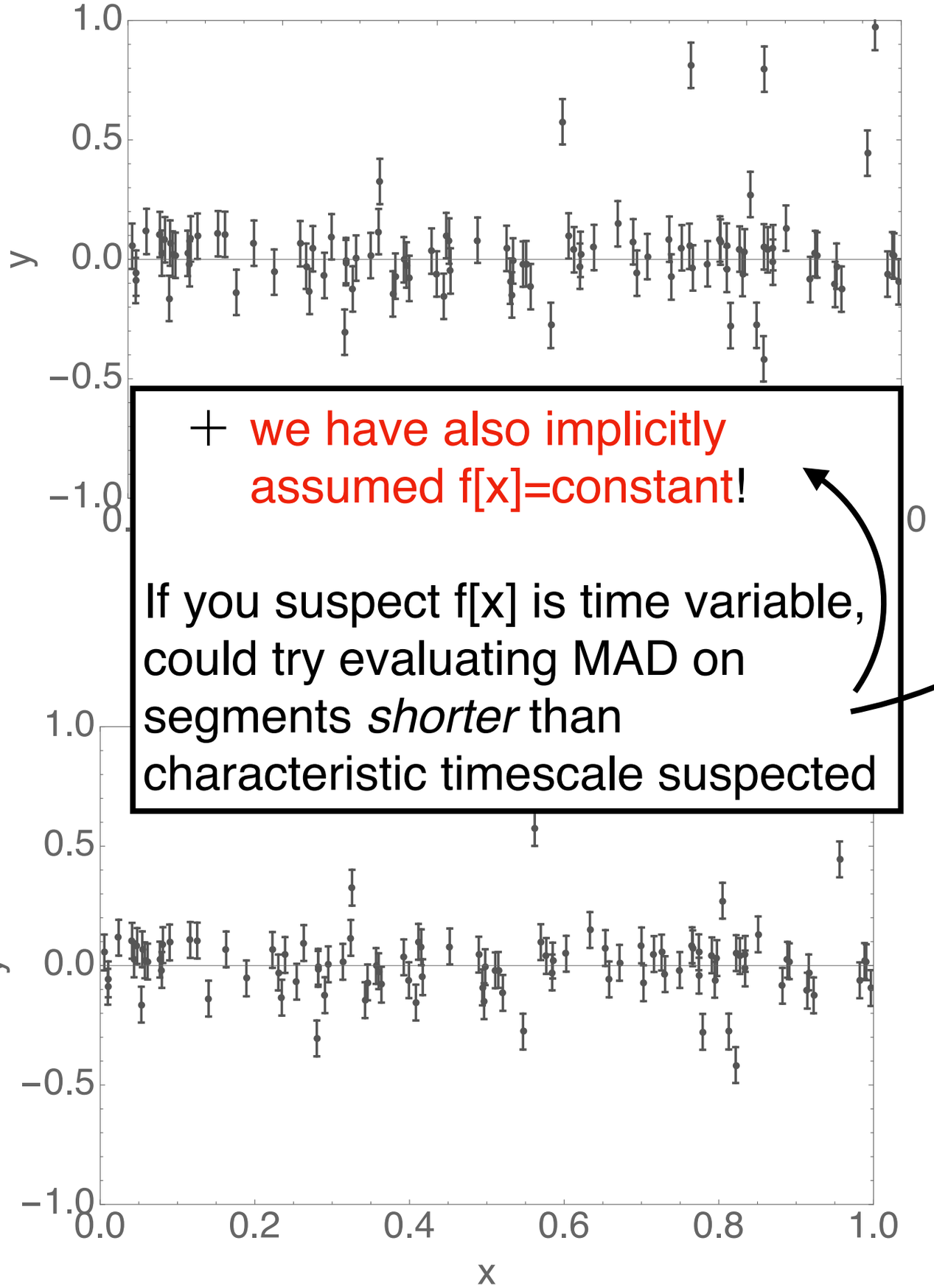
“correct” errors



standard deviation errors



median absolute deviation (MAD) errors



+ we have also implicitly assumed f[x]=constant!

If you suspect f[x] is time variable, could try evaluating MAD on segments *shorter* than characteristic timescale suspected

once again, median statistics can help

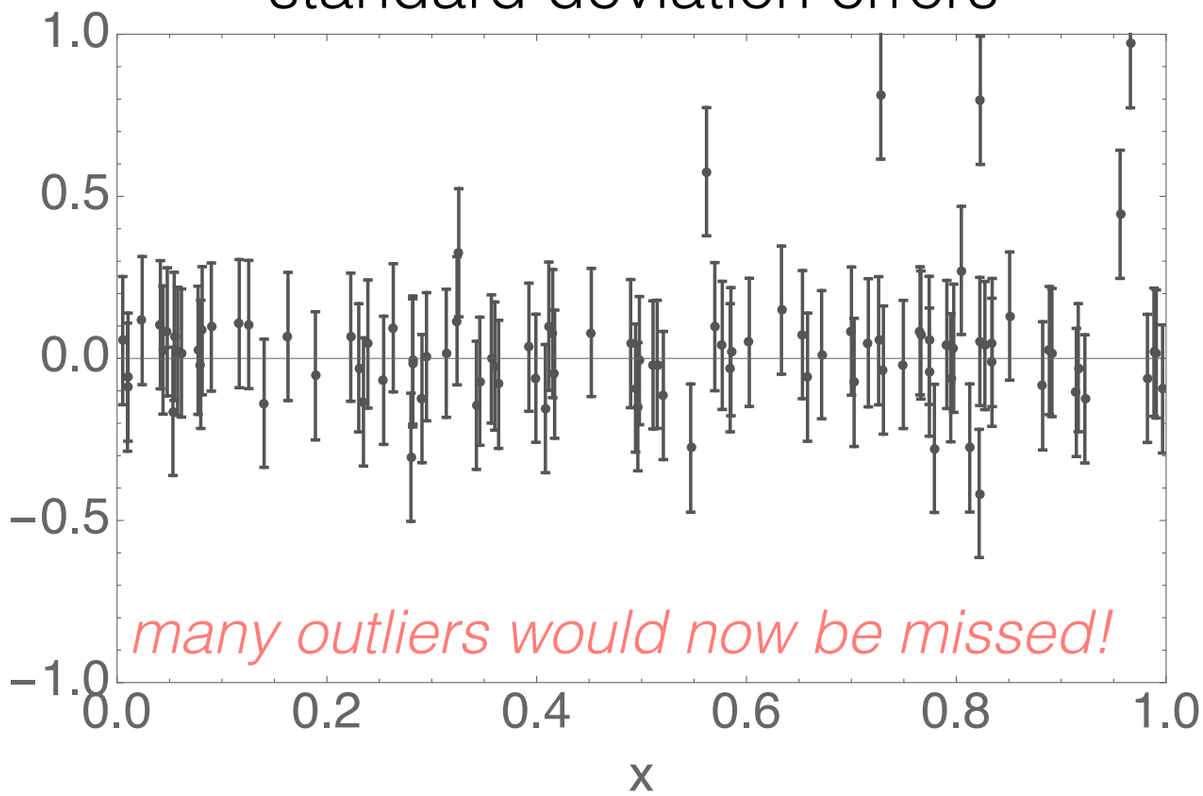
standard deviation = 0.198

1.4826*MAD = 0.095

"correct" = 0.075

but we implicitly assume homoscedastic

standard deviation errors



many outliers would now be missed!

median absolute deviation (MAD)

MAD is the median of the absolute deviations from the data's median

$$\mathrm{MAD} = \mathrm{median}(\ |X_i - \mathrm{median}(X_i)|\ )$$

jupyter mad Last Checkpoint: 2 minutes ago (unsaved changes)

File Edit View Insert Cell Kernel Help Python 2

Code CellToolbar

In [13]:
```
import numpy as np

def mad(x):
    median = np.median(x)
    devs=[0 for i in range(len(x))]
    for i in range(len(x)):
        devs[i] = np.absolute(x[i] - median)
    return np.median(devs)
```

In [17]:
```
# test it
samples=np.random.normal(0,1,1000000)
print np.std(samples)/mad(samples)
```

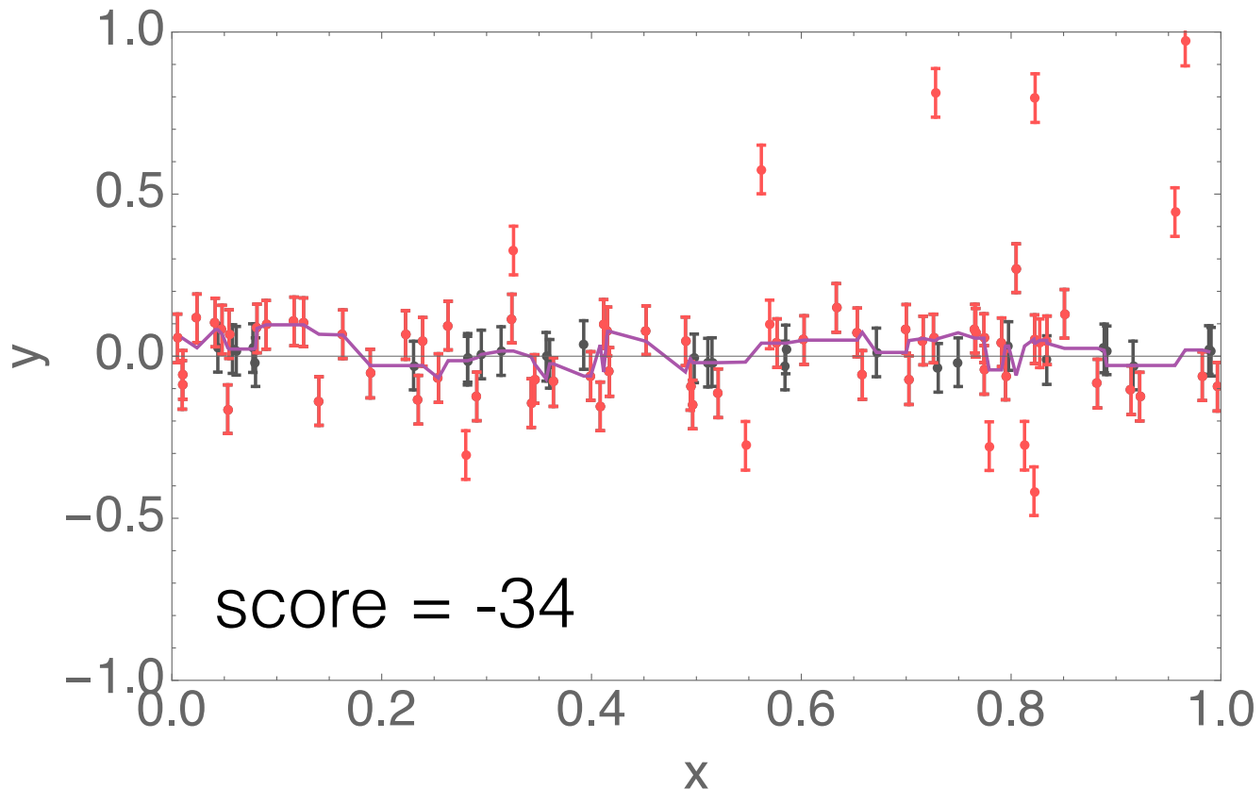
```
1.48235672887
```

for data drawn from a normal distribution,
MAD = 1.4826*stdev

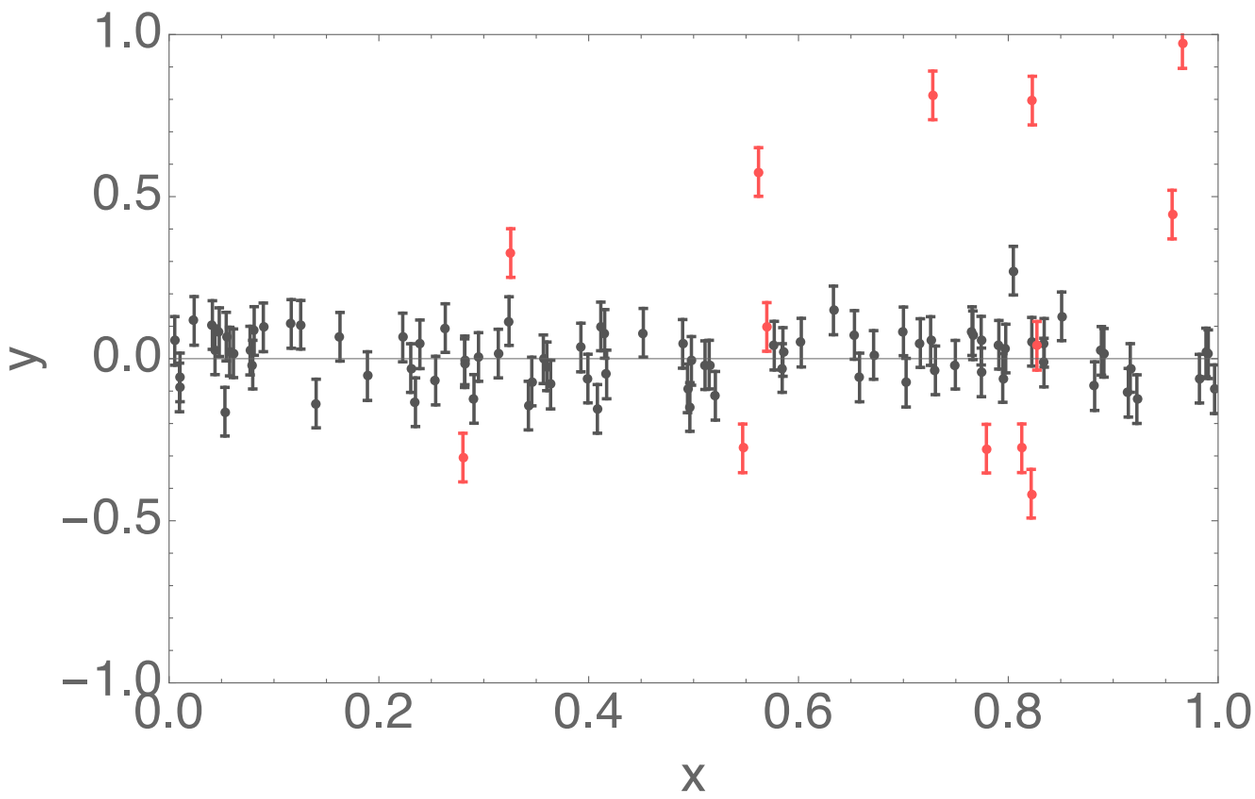
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{0.5}$



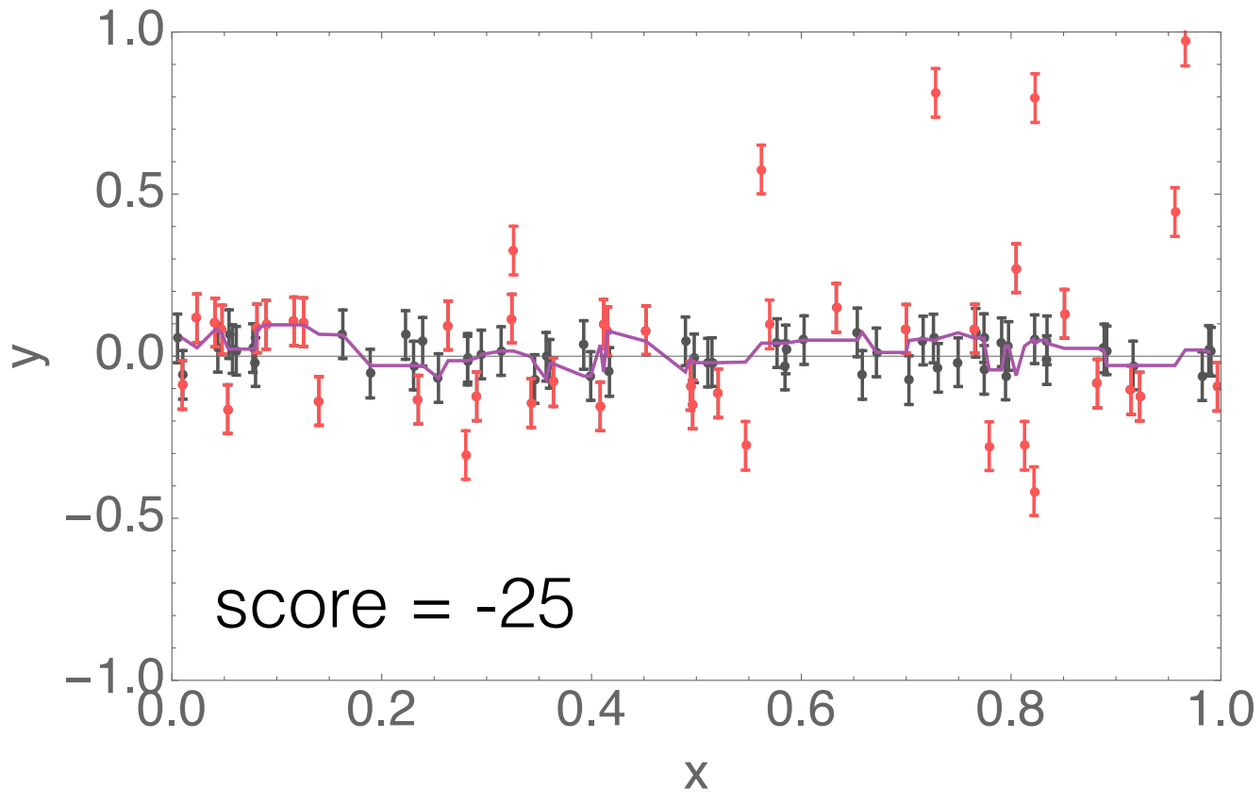
true outliers



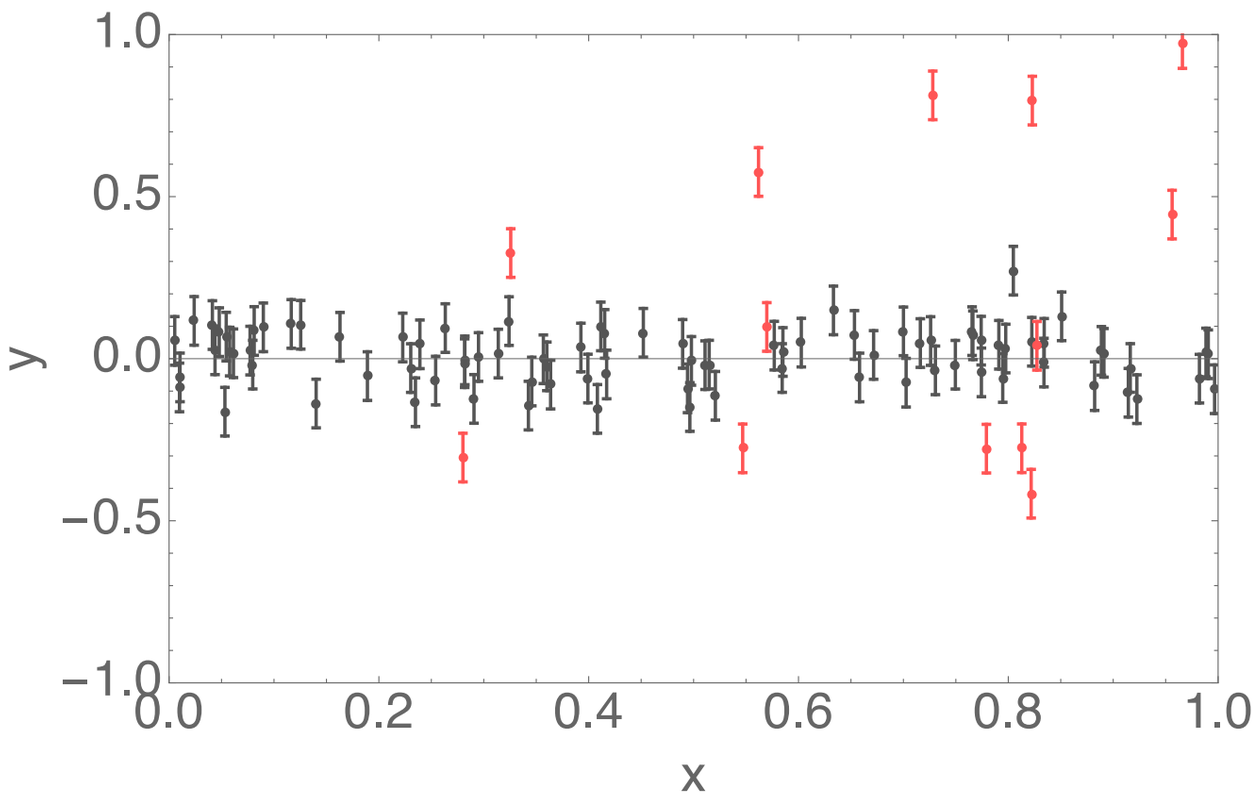
detecting outliers

problem 3: choosing a sigma cut off

outlier if $| (y - f[x])/\sigma_y | > \mathbf{1.0}$



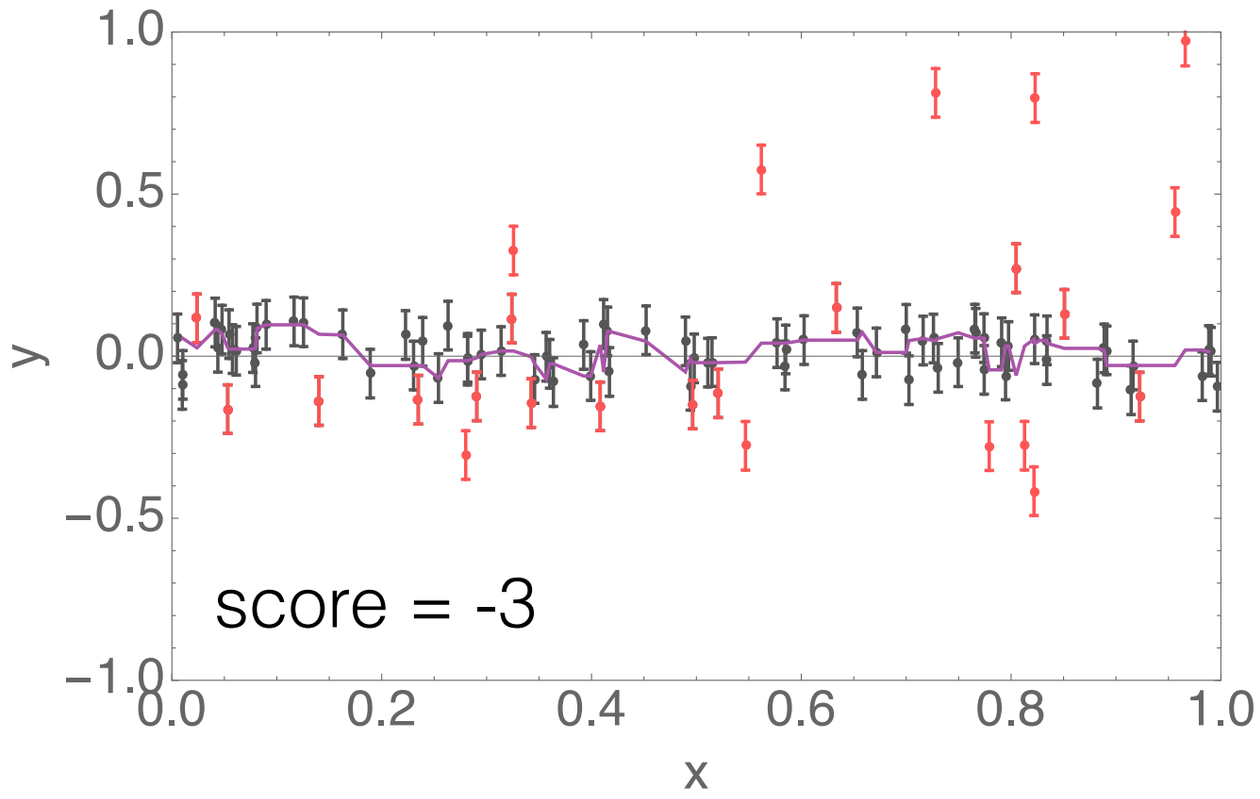
true outliers



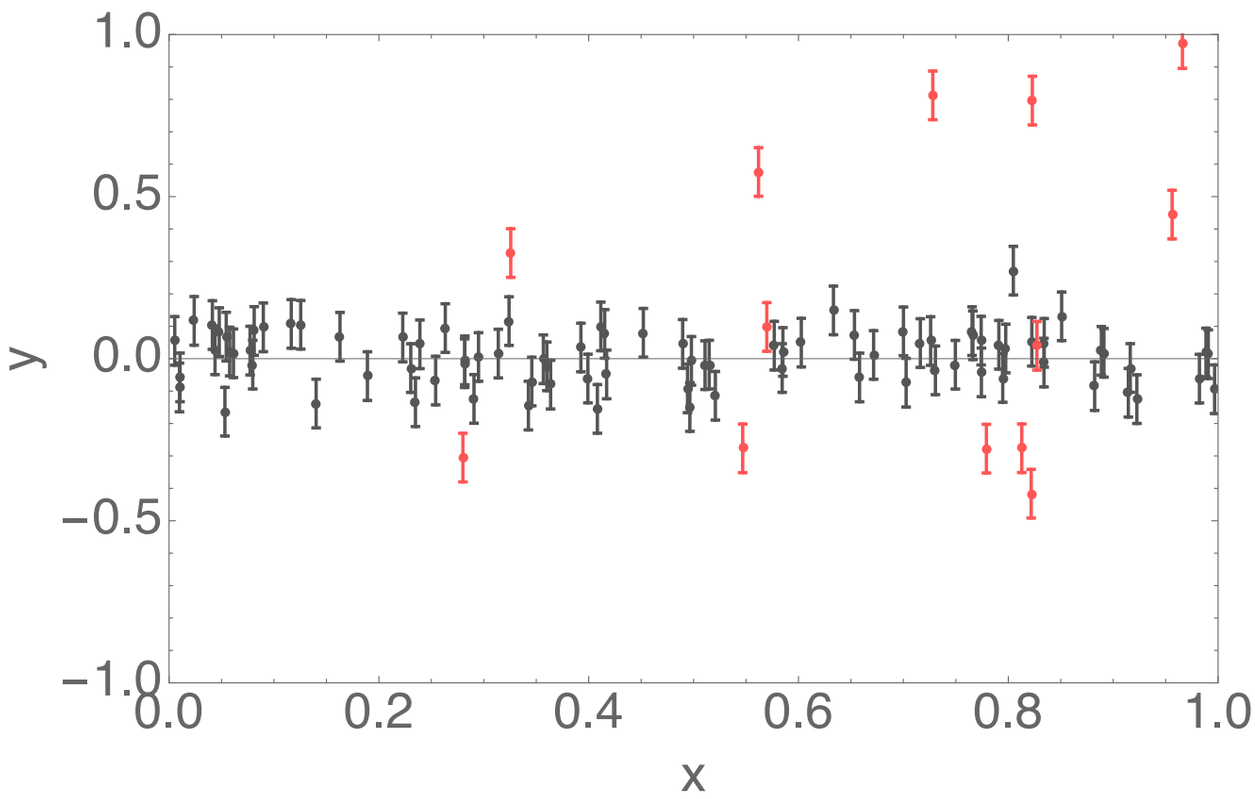
detecting outliers

problem 3: choosing a sigma cut off

outlier if $| (y - f[x])/\sigma_y | >$ **1.5**



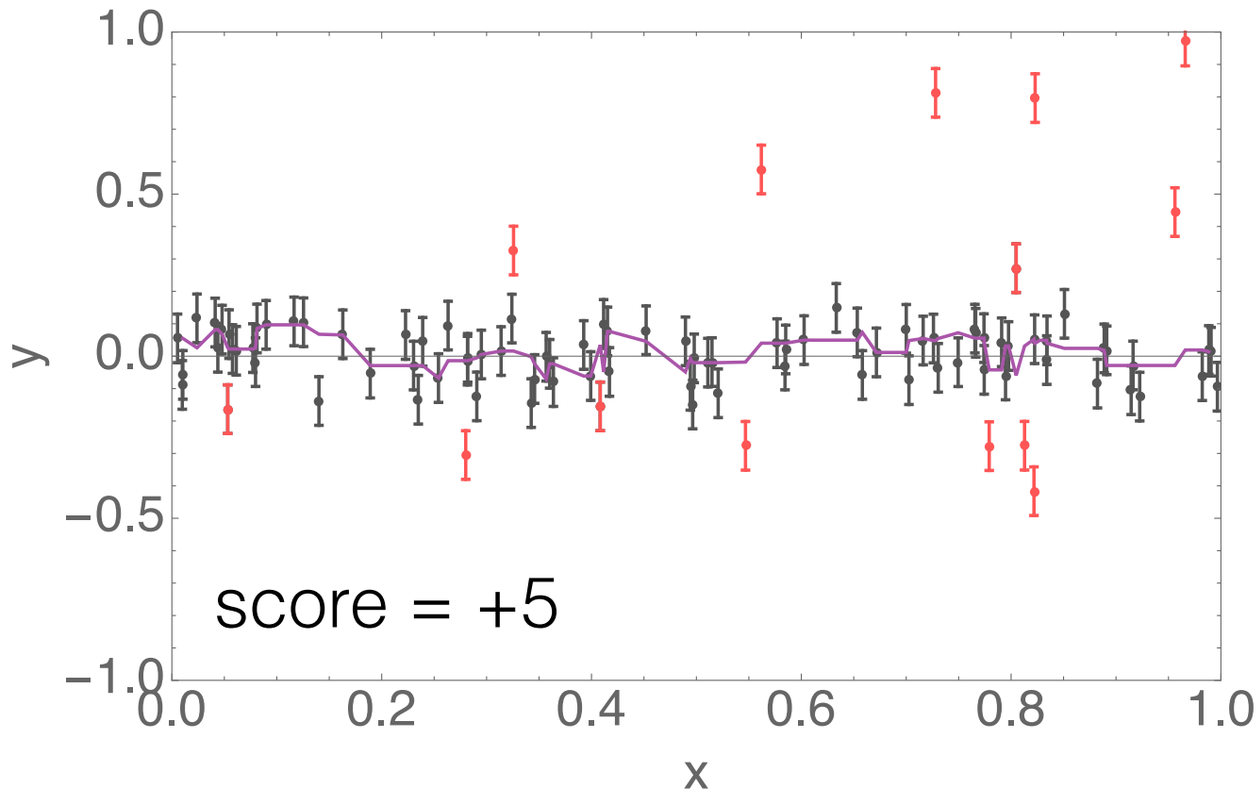
true outliers



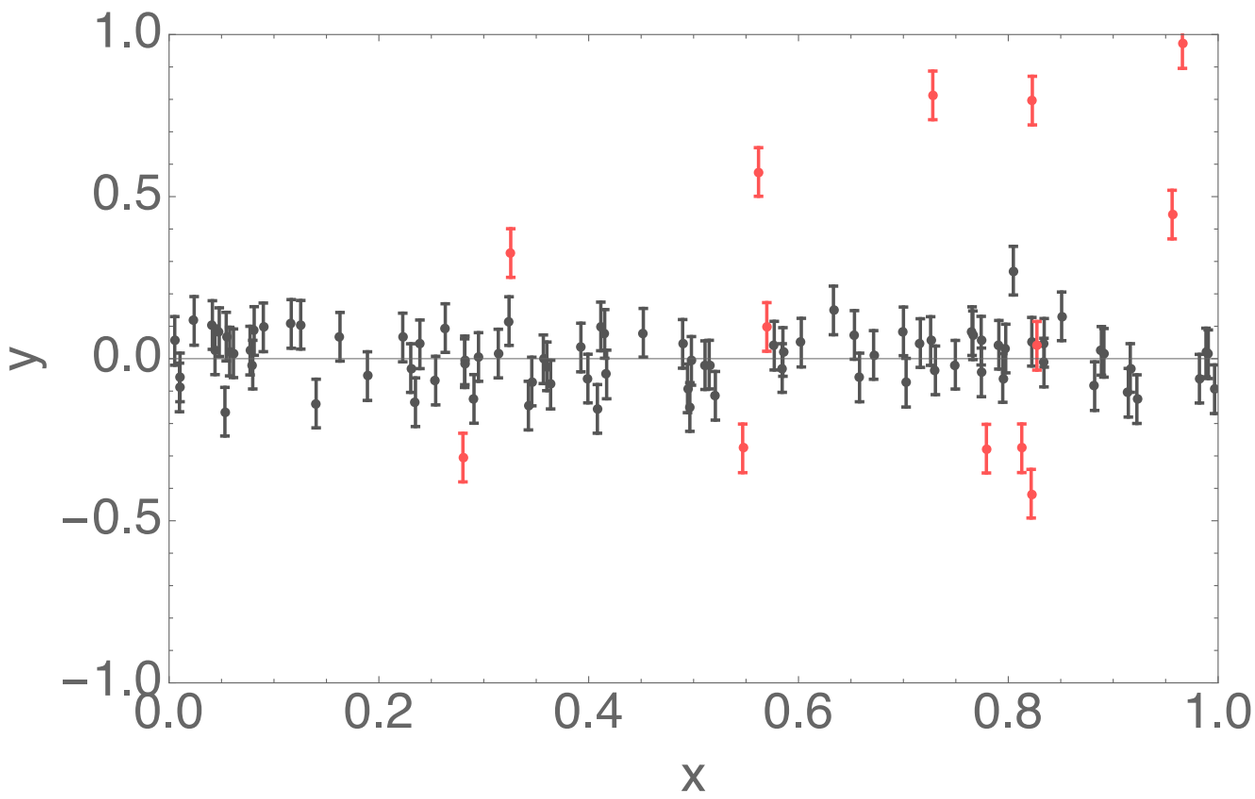
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{2.0}$



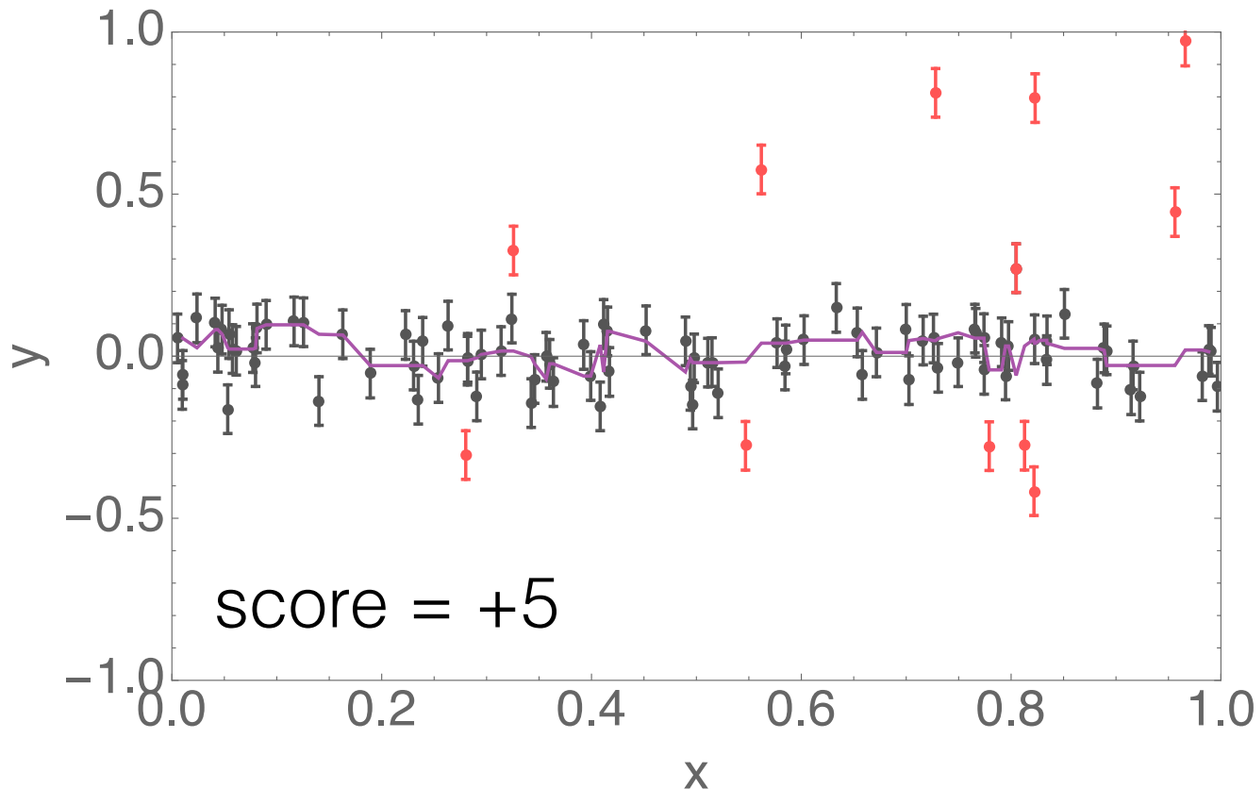
true outliers



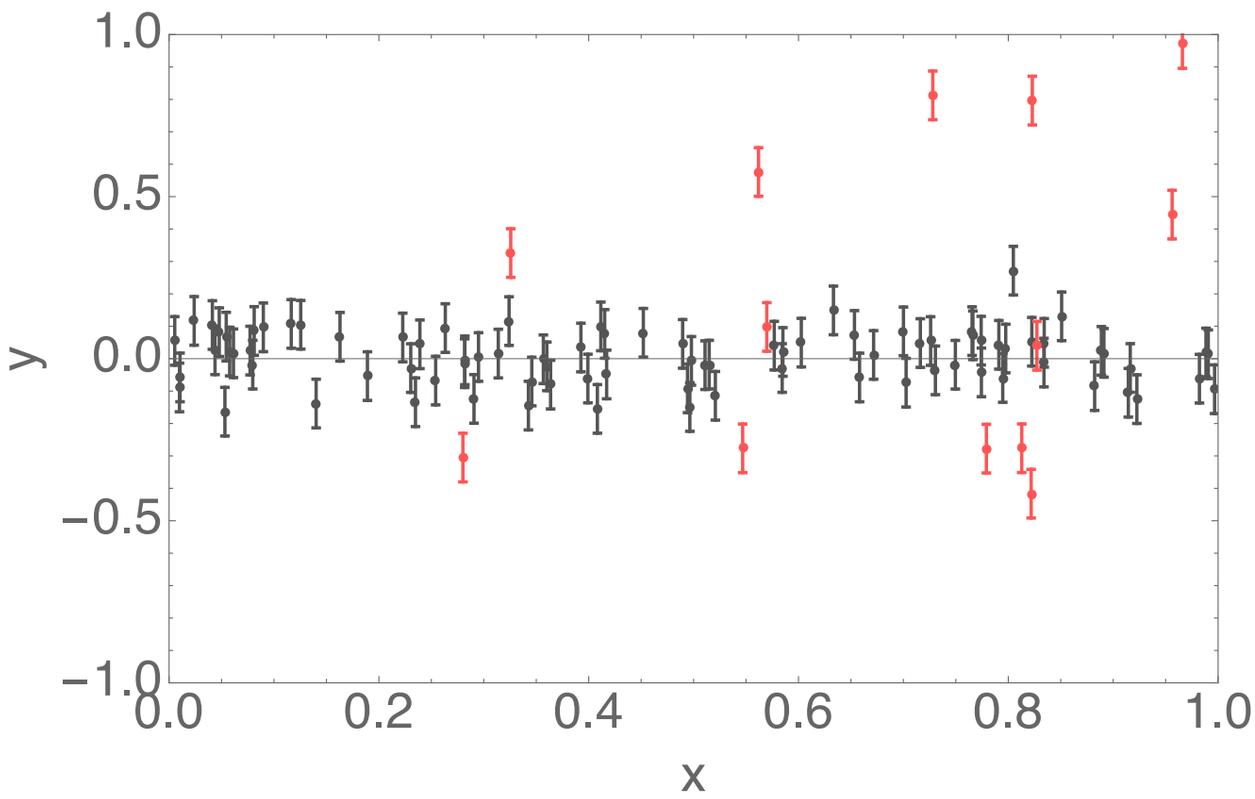
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{2.5}$



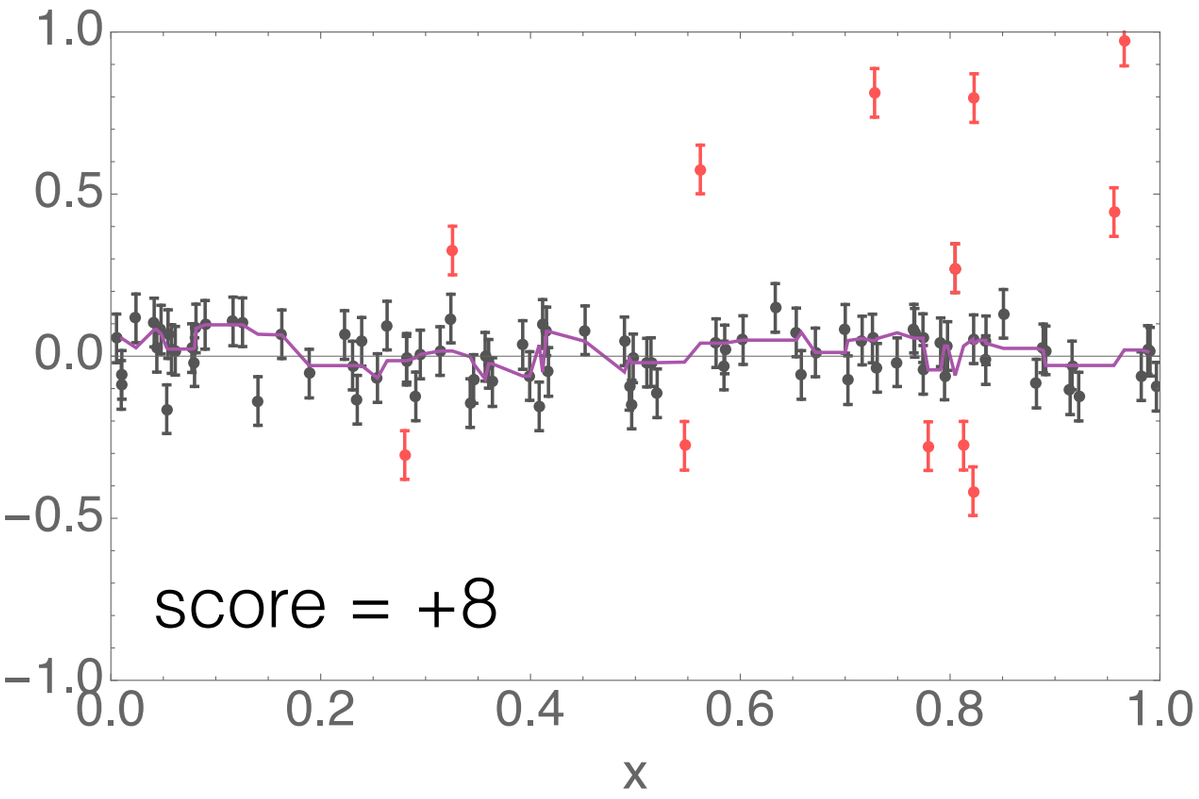
true outliers



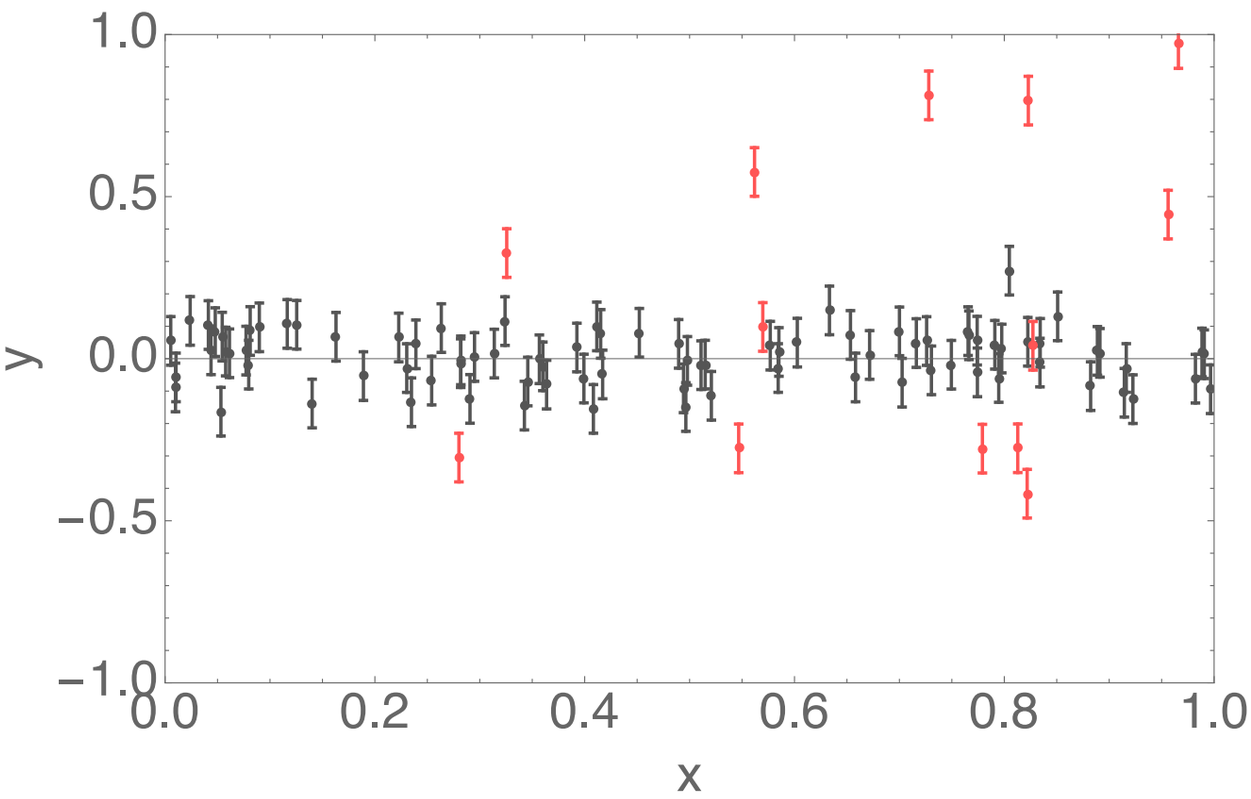
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{3.0}$



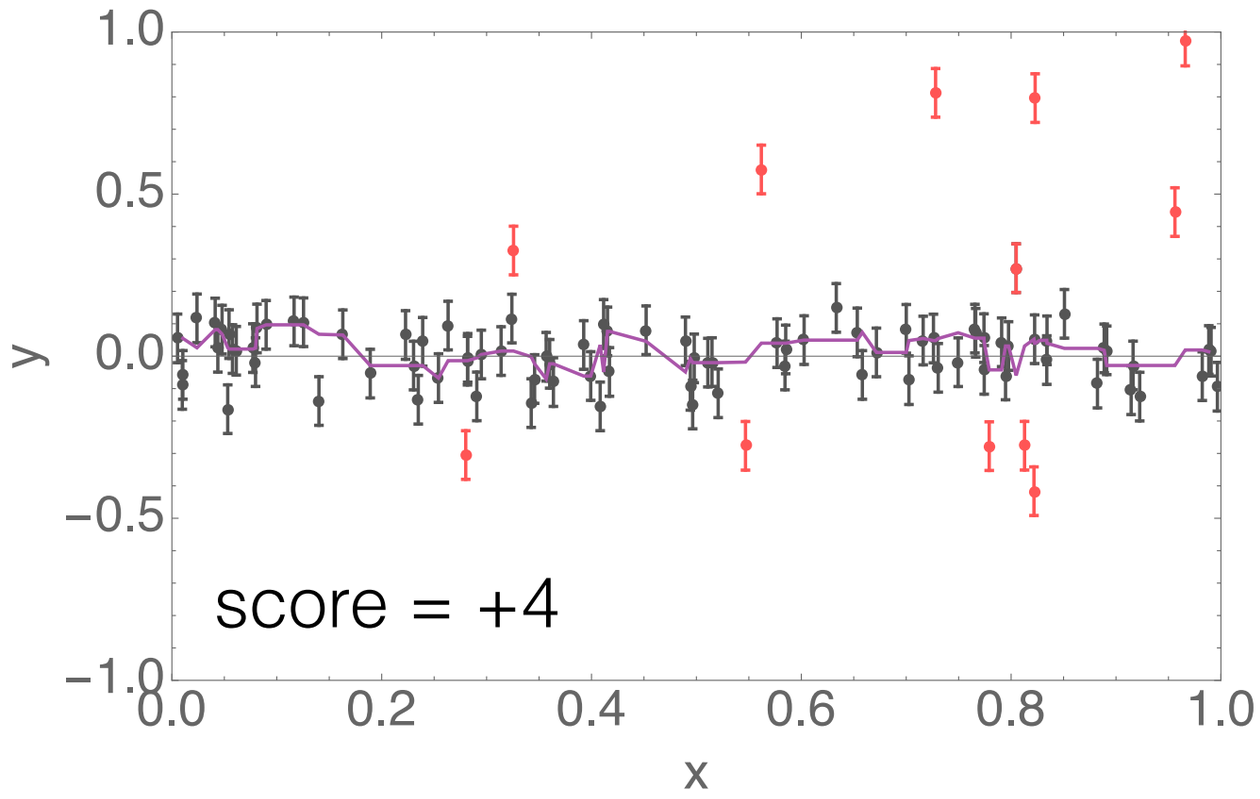
true outliers



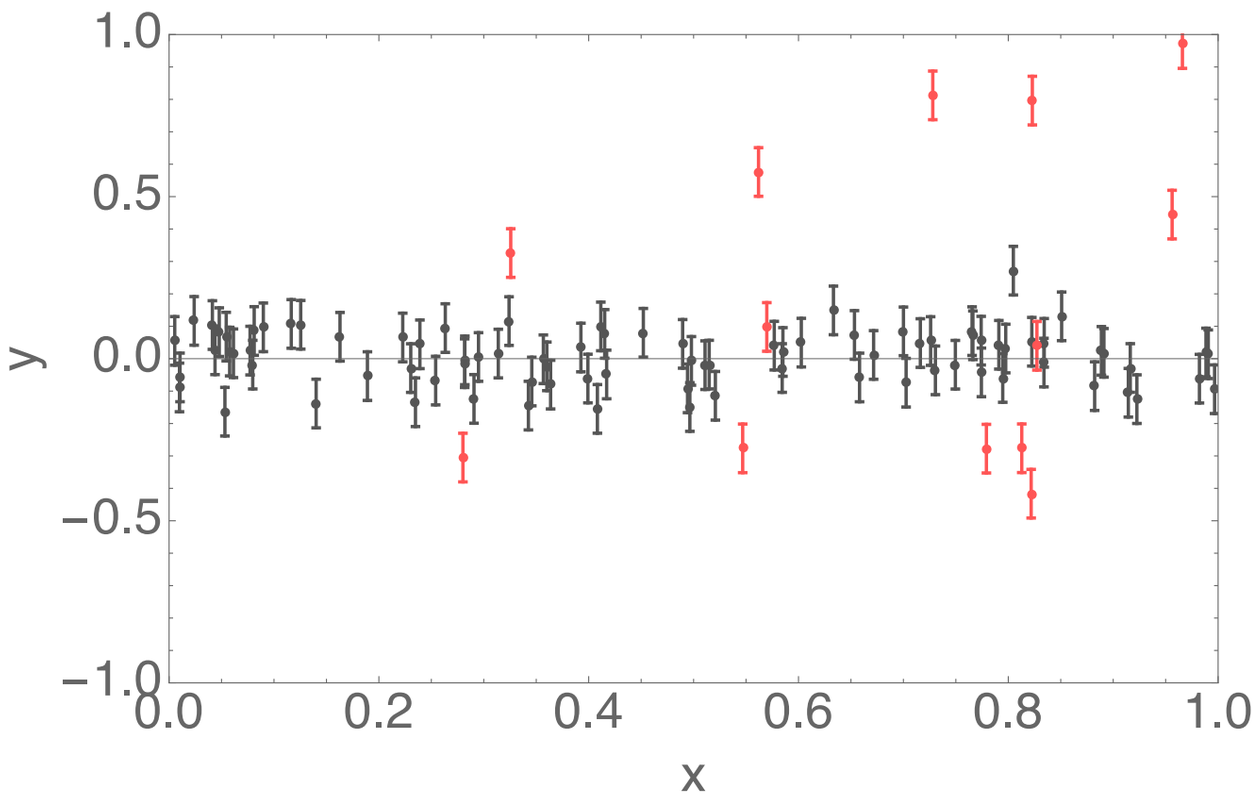
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{3.5}$



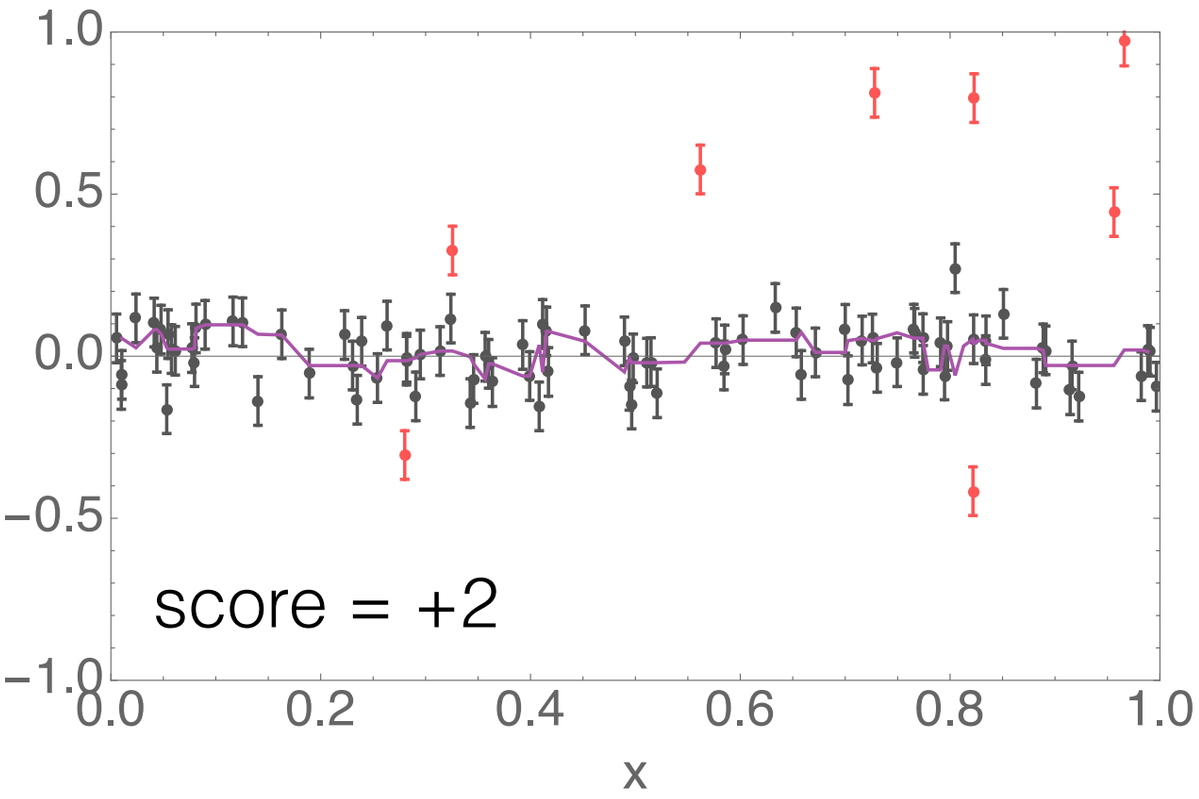
true outliers



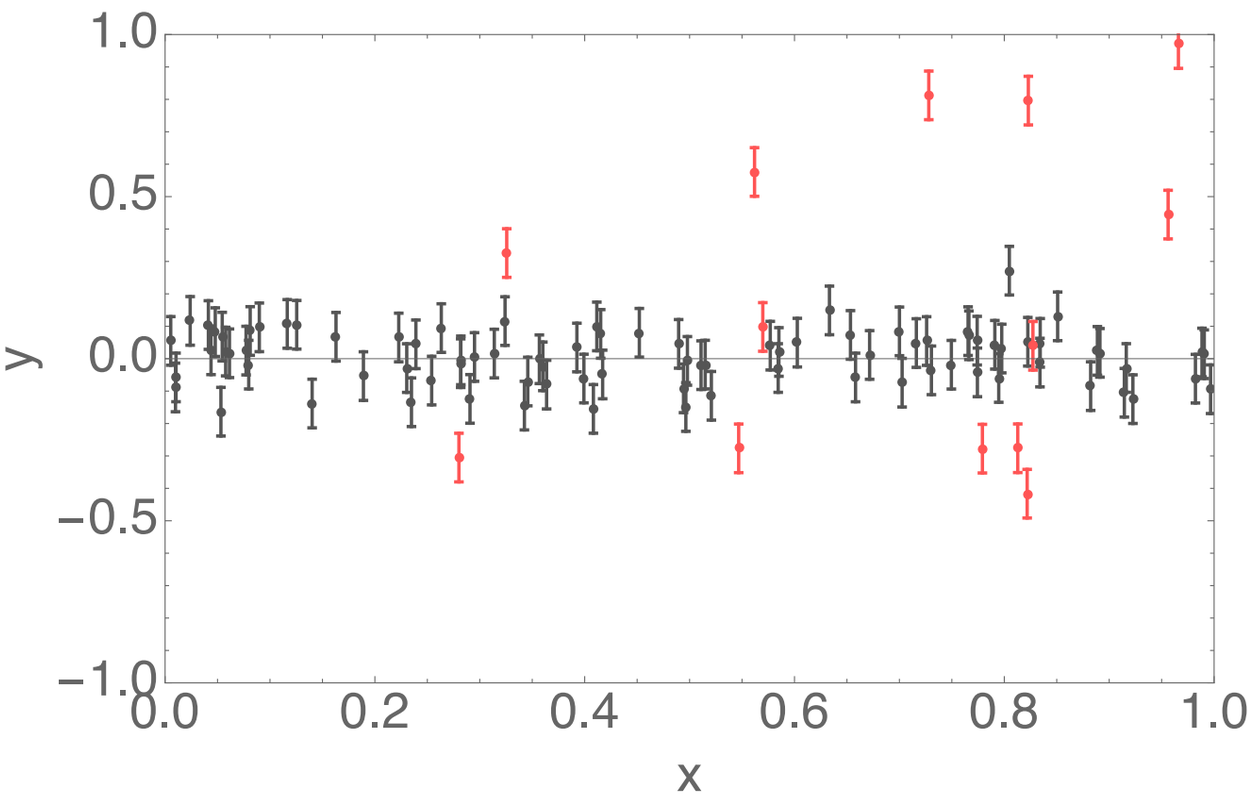
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{4.0}$



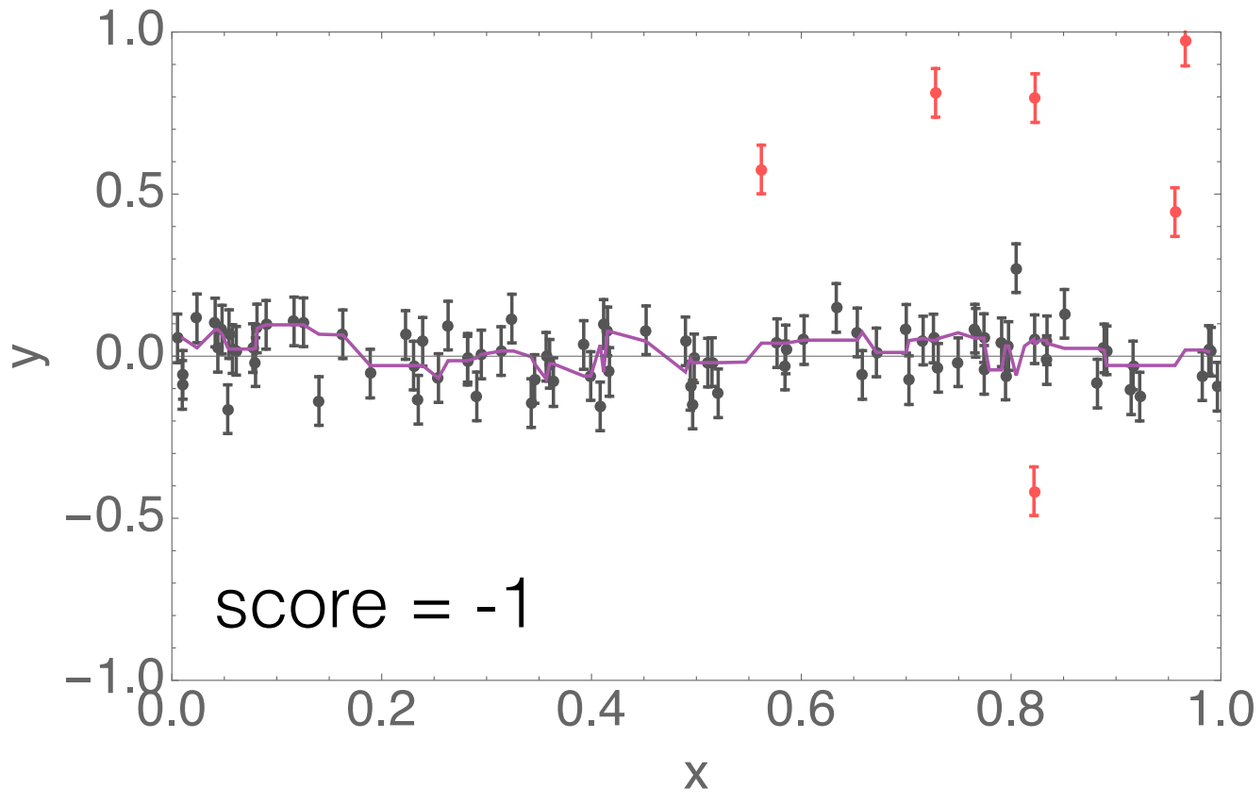
true outliers



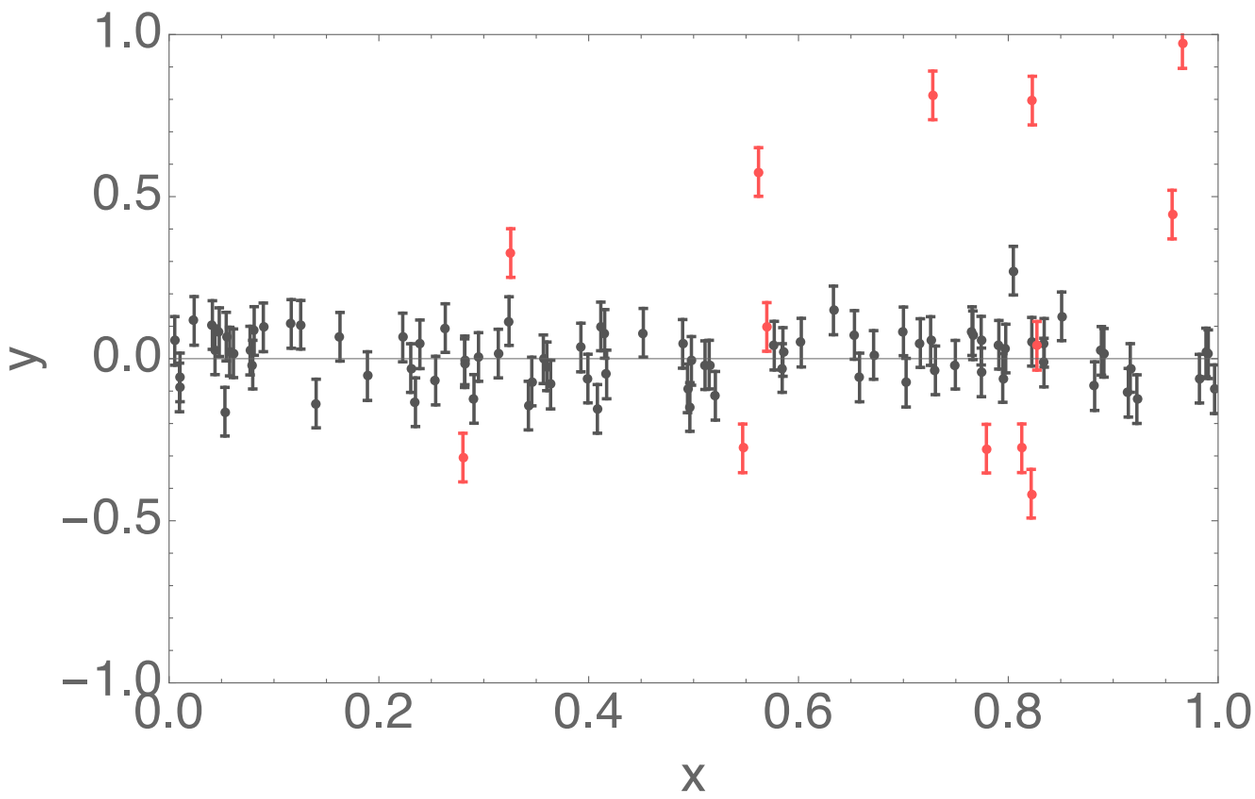
detecting outliers

problem 3: choosing a sigma cut off

outlier if $|(y - f[x])/\sigma_y| > \mathbf{4.5}$



true outliers



detecting outliers

problem 3: choosing a sigma cut off

so a cutoff around 3 sigma looks optimal **for this data**, but I only know this because I have a list of the true outliers!

we can estimate this more objectively

an outlier = a point where $y>\mu+f\sigma$ or $y<\mu+f\sigma$, where f is the sigma cutoff

if we make f too small, we will remove too much “good” data

if all data were normally distributed...
prob of being classed as an outlier = $1 - P(\mu-f\sigma<y<\mu+f\sigma)$

detecting outliers

problem 3: choosing a sigma cut off

if all data were normally distributed...
prob of being classed as an outlier = 1 - P(μ-fσ<y<μ+fσ)

prob of being classed as an outlier = $1 - \int_{y=\mu-f\sigma}^{\mu+f\sigma} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big[-\frac{(y-\mu)^2}{2\sigma^2}\Big]$

prob of being classed as an outlier = $1 - \mathrm{erf}(f/\sqrt{2})$

prob of being classed as an outlier = $\mathrm{erfc}(f/\sqrt{2})$

~1/len(data) → 0.01 = $\mathrm{erfc}(f/\sqrt{2})$

$$f = \sqrt{2}\mathrm{erfc}^{-1}(0.01)$$

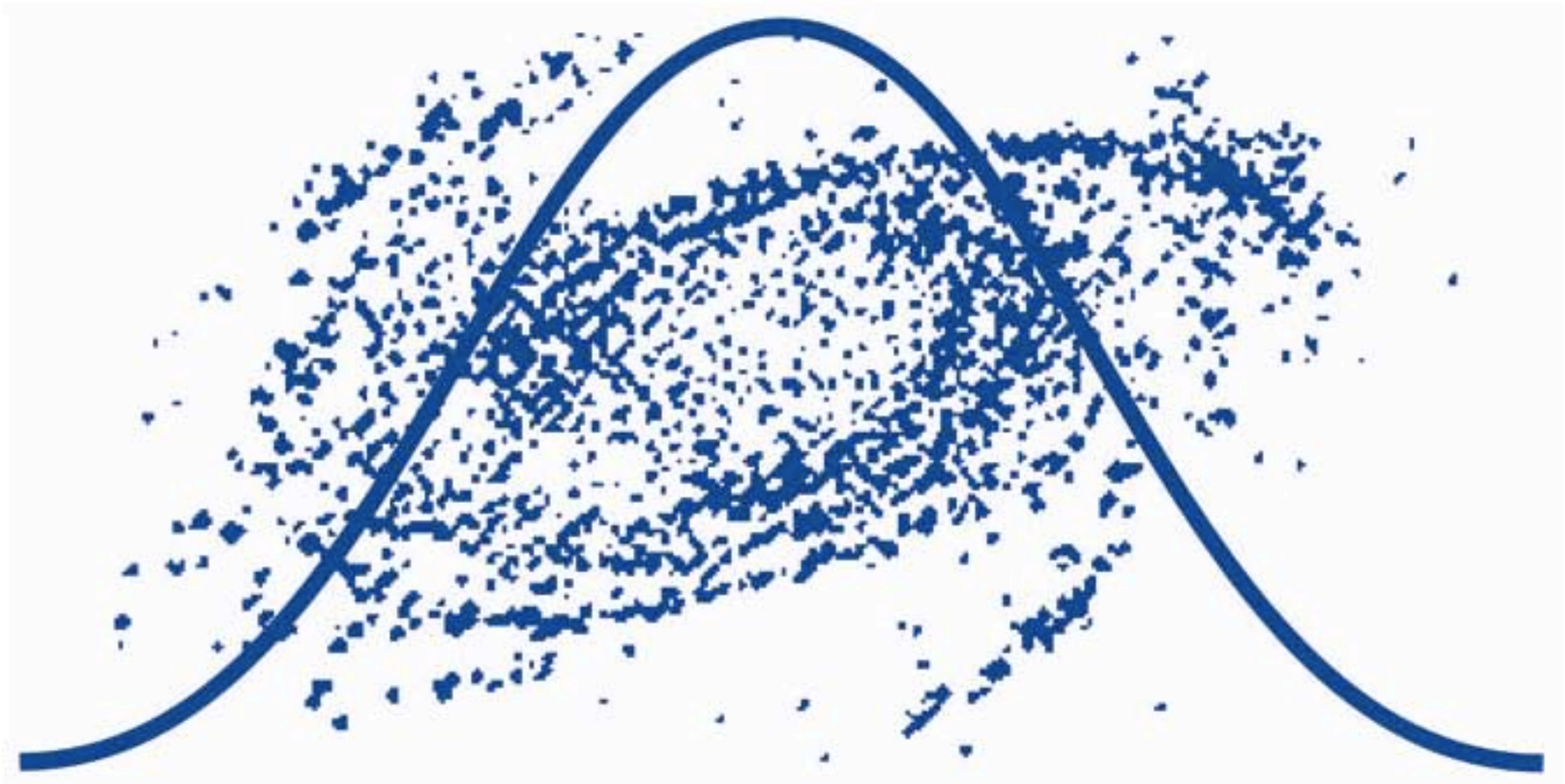
f = 2.57... sigmas

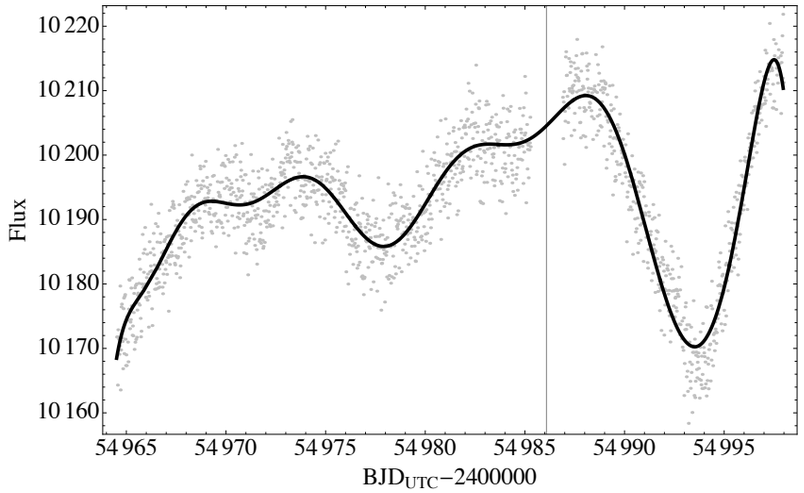
score = +7

INDEP PROJECT

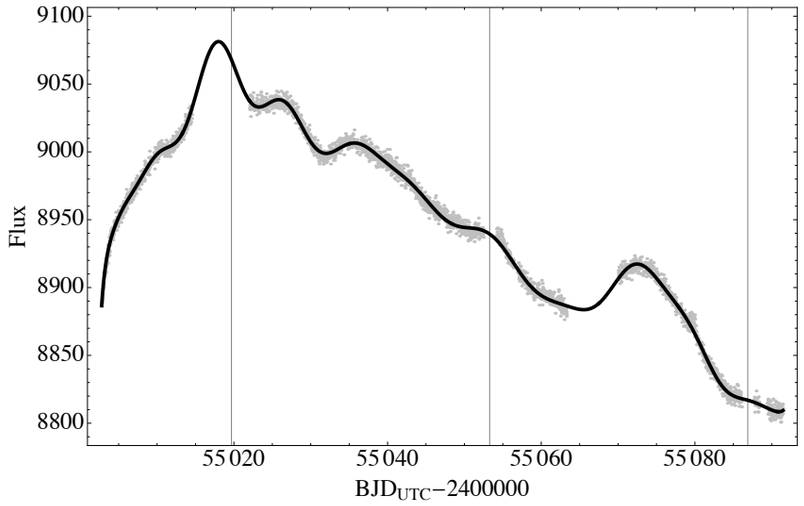
By Nov 13, identify
1) a scientific question (one sentence) data set (link)
2) a data set (link or identification)
3) a proposed analysis to address question (3-4 sentences)

L7.2 Detrending

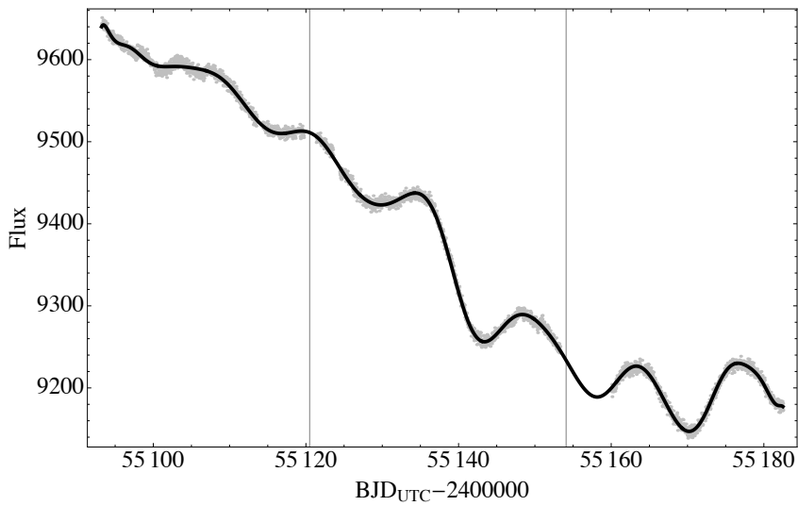




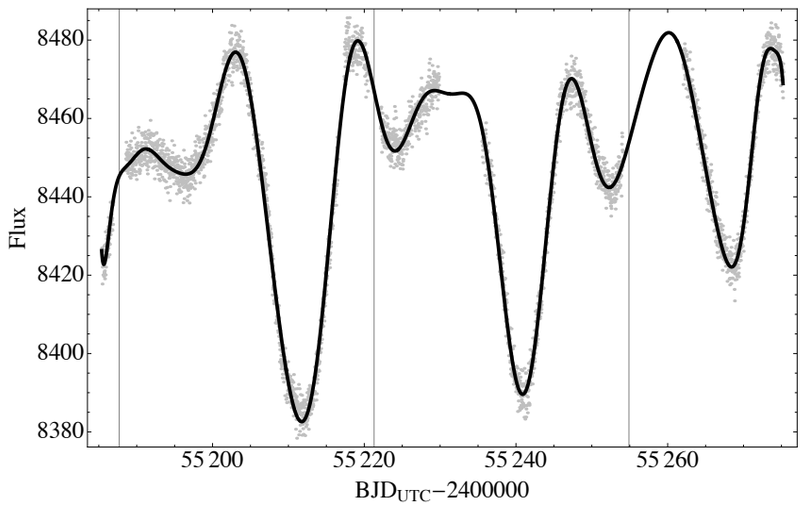
(a) Quarter 1



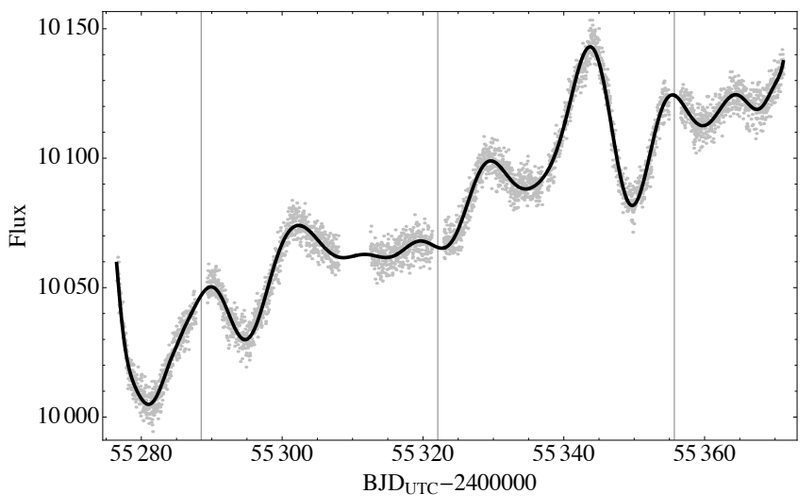
(b) Quarter 2



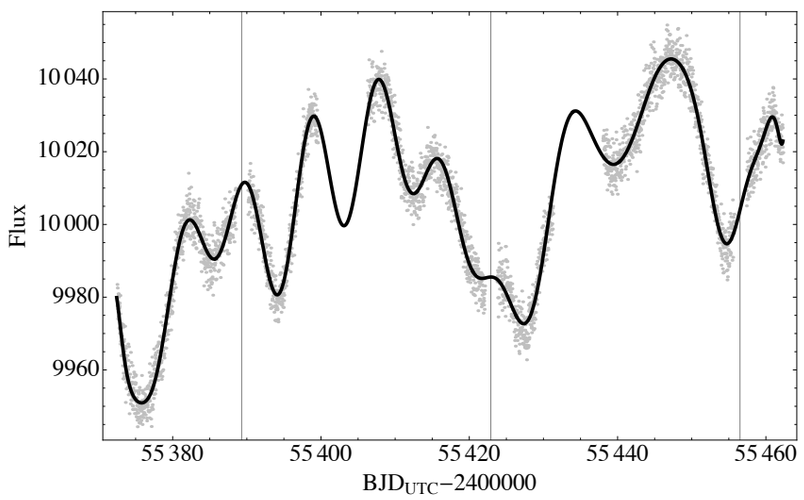
(c) Quarter 3



(d) Quarter 4



(e) Quarter 5



(f) Quarter 6

astronomical data often displays **trends** which are either *instrumental* or *astrophysical*

if these trends are not of interest (i.e. they are a nuisance) then they require **detrending**

as with our discussion of outliers, we are only going to consider 2D arrays, like a time series

the typical practice is to remove the trends in an initial stage of analysis, and **then** take the data product and analyze it independently in a separate stage

technically this is wrong!!

this is what we want...

$$P(\theta_{interest}|D) = \int P(\theta_{interest},\theta_{nuisance}|D)\, d\theta_{nuisance}$$

where $P(\theta_{interest},\theta_{nuisance}|D) = P(D|\theta_{interest},\theta_{nuisance})\, P(\theta_{interest})\, P(\theta_{nuisance}) / P(D)$

so $P(\theta_{interest}|D) = [P(\theta_{interest})/P(D)] \boxed{\int P(D|\theta_{interest},\theta_{nuisance})\, P(\theta_{nuisance})\, d\theta_{nuisance}}$

but this is what you actually end up getting...

$$P(\theta_{interest}|D,\hat{\theta}_{nuisance}) = \boxed{P(D|\theta_{interest},\hat{\theta}_{nuisance})}[P(\theta_{interest}) / P(D)]$$

$$\int P(D|\theta_{interest},\theta_{nuisance})\, P(\theta_{nuisance})\, d\theta_{nuisance} \neq P(D|\theta_{interest},\hat{\theta}_{nuisance})$$

unless $P(\theta_{nuisance}) = DiracDelta[\hat{\theta}_{nuisance}]$

the typical practice is to remove the trends in an initial stage of analysis, and then take the data product and analyze it independently in a separate stage

technically this is wrong!!

why does this matter?

there is some uncertainty in our inference of $\theta_{nuisance}$ (i.e. its a distribution) and by assuming it equals just a fixed value, that uncertainty is ignored

$\theta_{nuisance}$ and $\theta_{interest}$ must be covariant (or you wouldn't be trying to detrend!) so your posterior on $\theta_{interest}$ will be
a) systematically skewed b) underestimate the width/uncertainty

unless $P(\theta_{nuisance}) = \text{DiracDelta}[\hat{\theta}_{nuisance}]$

the typical practice is to remove the trends in an initial stage of analysis, and then take the data product and analyze it independently in a separate stage

technically this is wrong!!

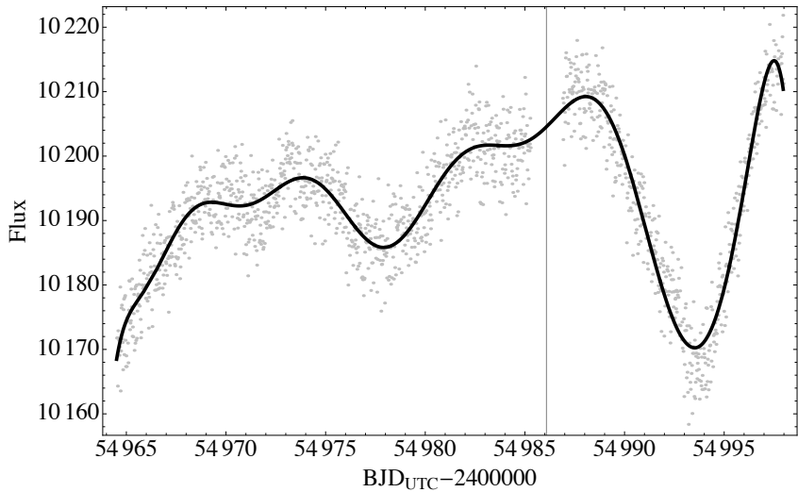
is this ever OK?

in practice, if the data constrain $\theta_{nuisance}$ much better than $\theta_{interest}$, then $\theta_{nuisance}$'s uncertainty has negligible impact on $\theta_{interest}$

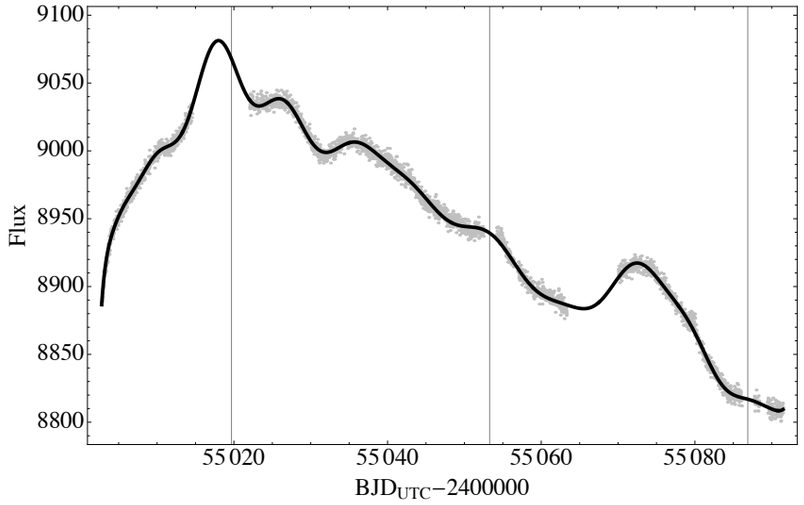
i.e. $P(\theta_{nuisance}) \sim DiracDelta[\theta_{nuisance}]$ is OK relatively speaking

this generally means that the data used to infer $\theta_{nuisance}$ must be *much more* constraining than that used to infer $\theta_{interest}$

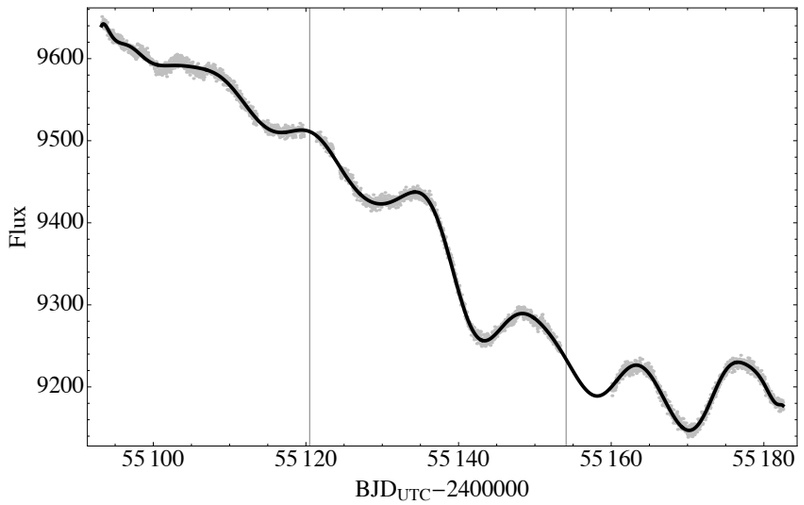
the simplest way to do this is to have plenty of training data



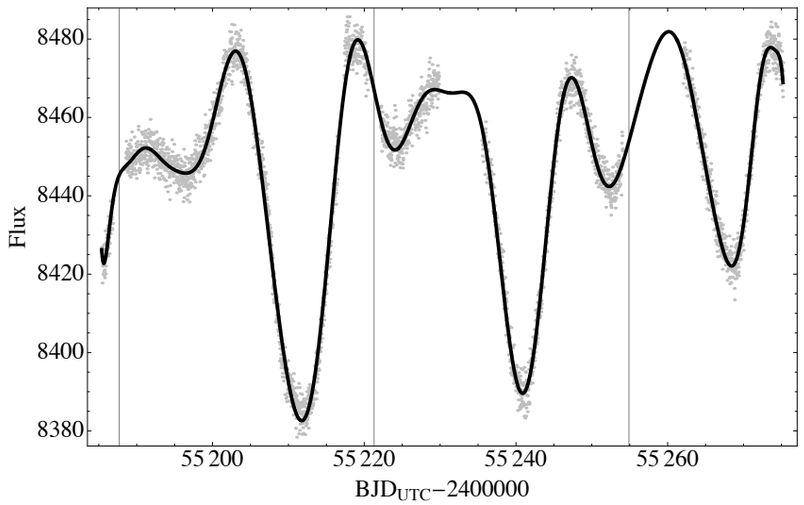
(a) Quarter 1



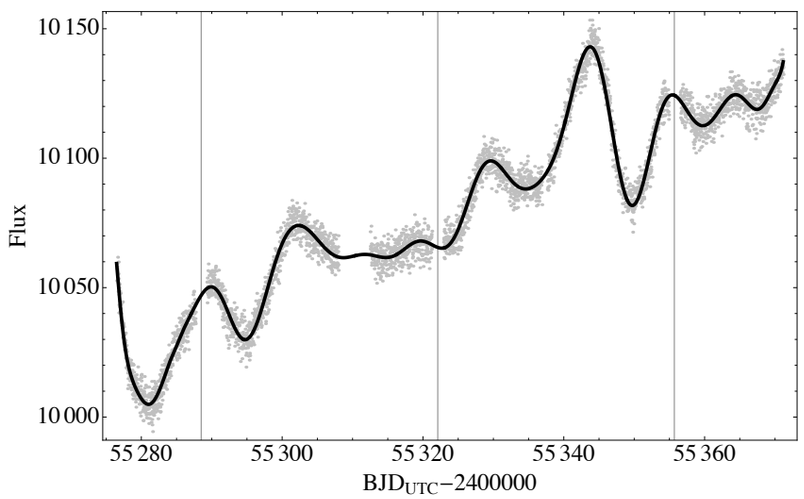
(b) Quarter 2



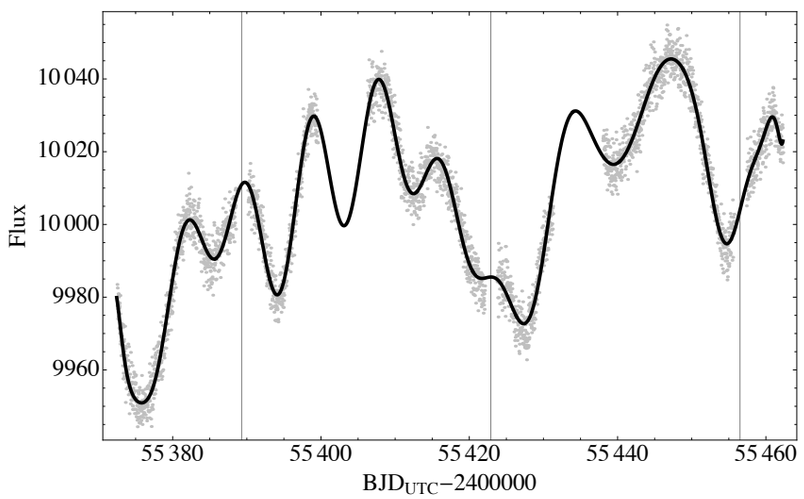
(c) Quarter 3



(d) Quarter 4



(e) Quarter 5



(f) Quarter 6

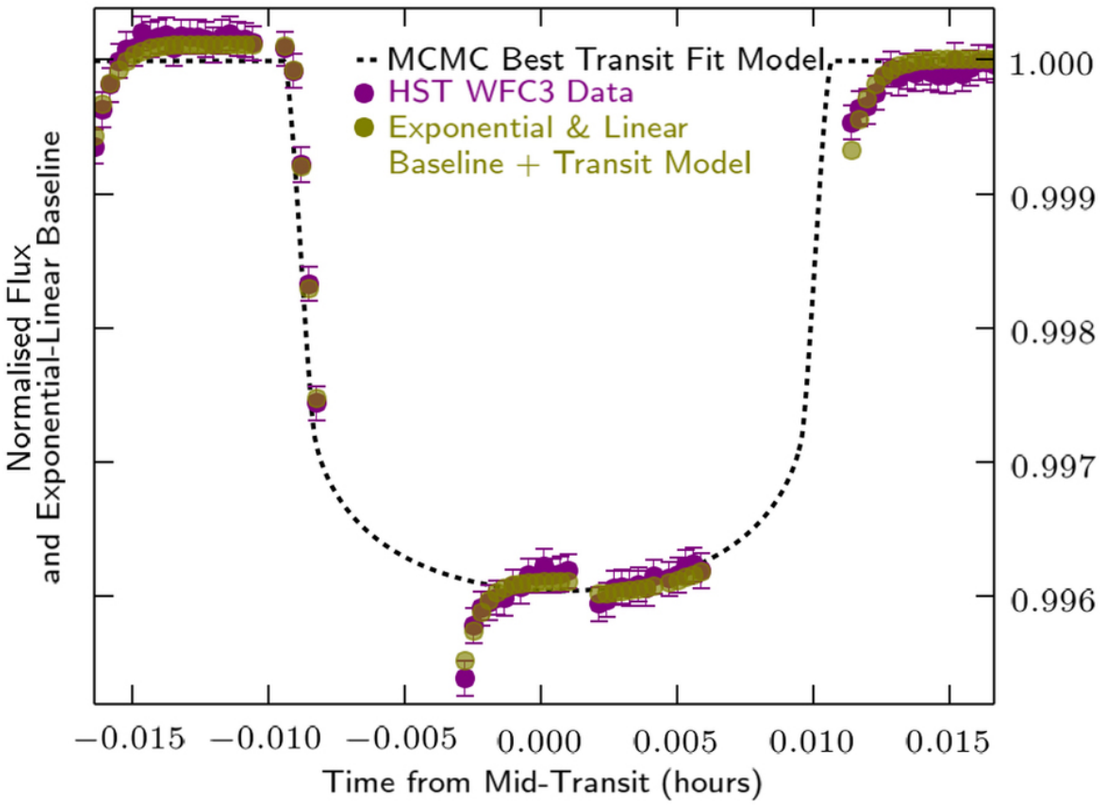
transits inform $\theta_{interest}$ representing <0.1% of all data

out-of-transit inform give $\theta_{nuisance}$ representing >99.9% of all data

under these conditions, two-stage analysis is OK but still requires we can specify a “good” model to describe the trends

limited training data

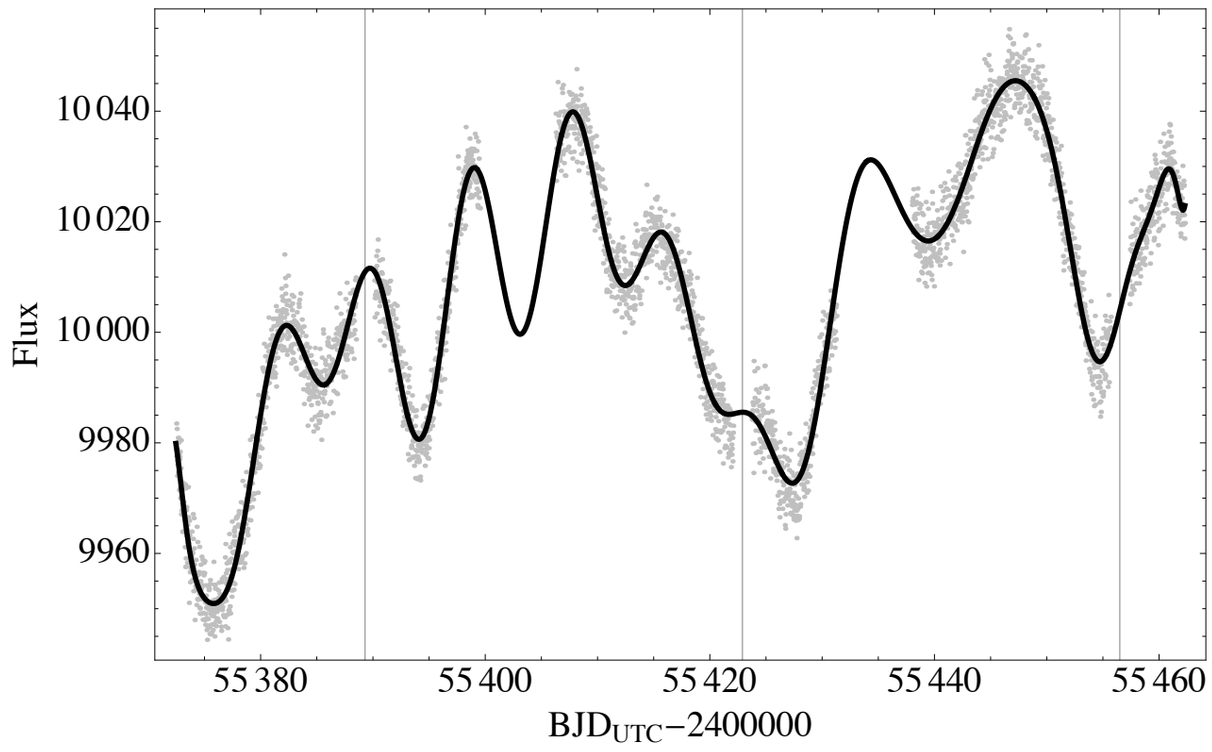
$\theta_{interest}$ and $\theta_{nuisance}$ have to be inferred together in a single, joint fit



very robust, but increases dimensionality and thus CPU cost of model a lot

crap-loads of training data

$\theta_{nuisance}$ can be inferred first, then used to detrend data and infer $\theta_{interest}$



we end up having to interpolate the training model into a data gap

if we mis-specify the nuisance/trend model, you are screwed either way

parametric detrending models

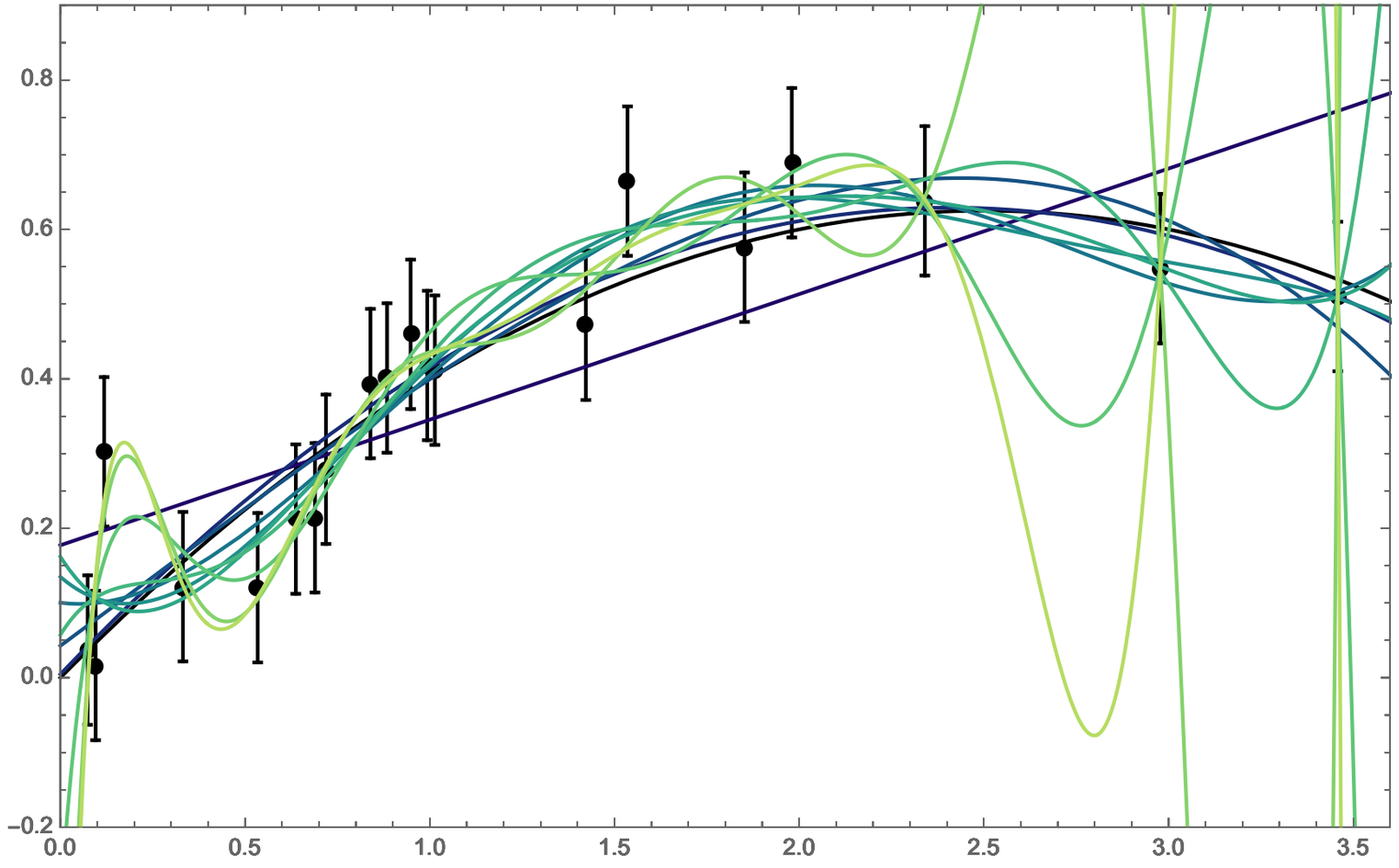
parametric => a model which depends on some parameters, $\theta_{nuisance}$

often, we don't really know how the instrument/astrophysics behaves so we have come up with some general form able to reasonably explain the data

typical choices might be a polynomial, but beware that high order polynomials (>5) become unstable

if polynomial order = # of data fits, line will go through every point (!) but will be awful at prediction

for now, choose the simplest which looks reasonable! (we'll discuss formal model selection methods later)



parametric detrending models

typical choices might be...

polynomial
polynomial in cosines
exponentials (for ramps)

non-parametric detrending models

non-parametric => not well-defined, but typically just a very large number of latent parameters

typical choices might be...

moving median

Gaussian process

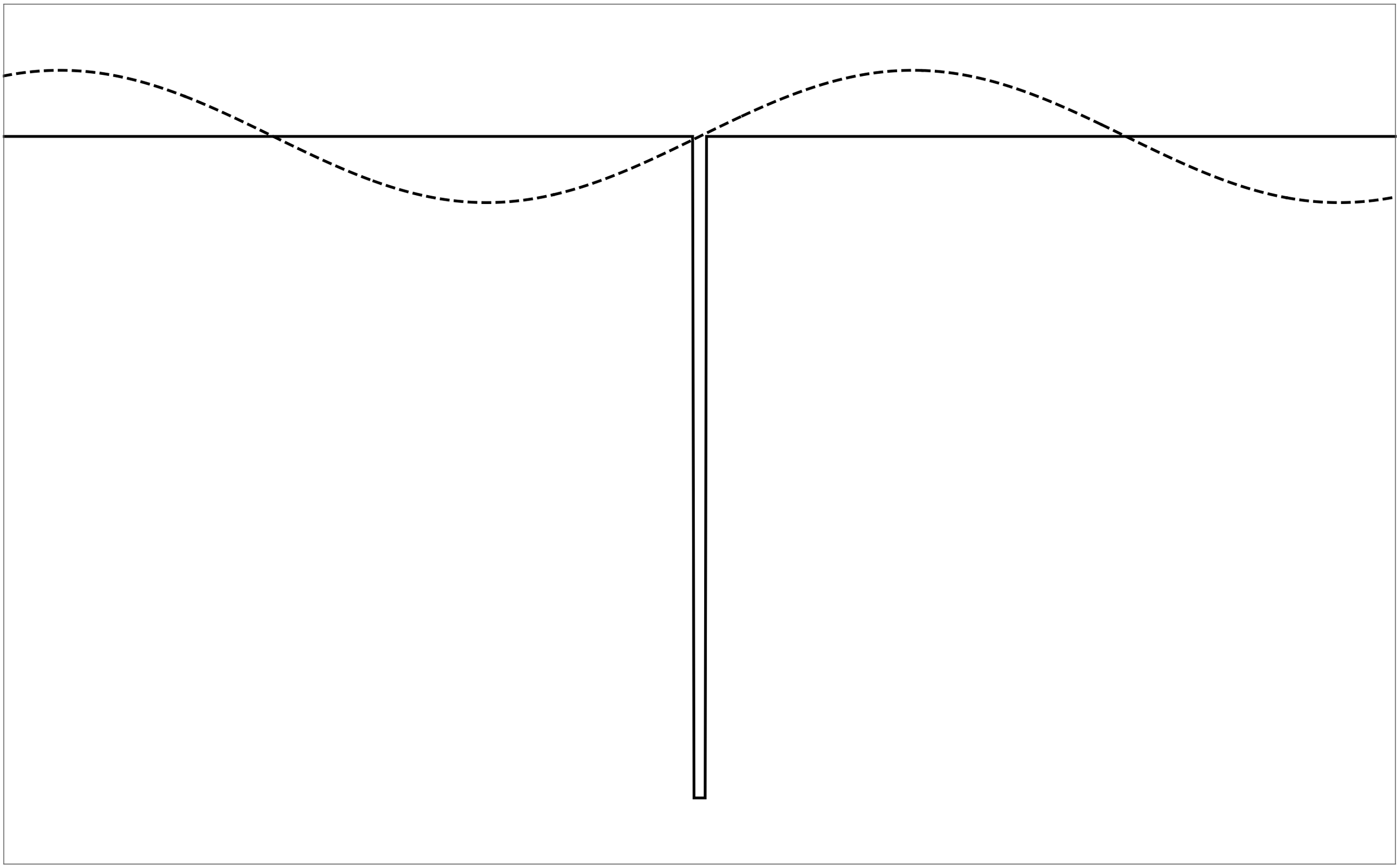
moving average/median discussed earlier, but for GPs too advanced for today

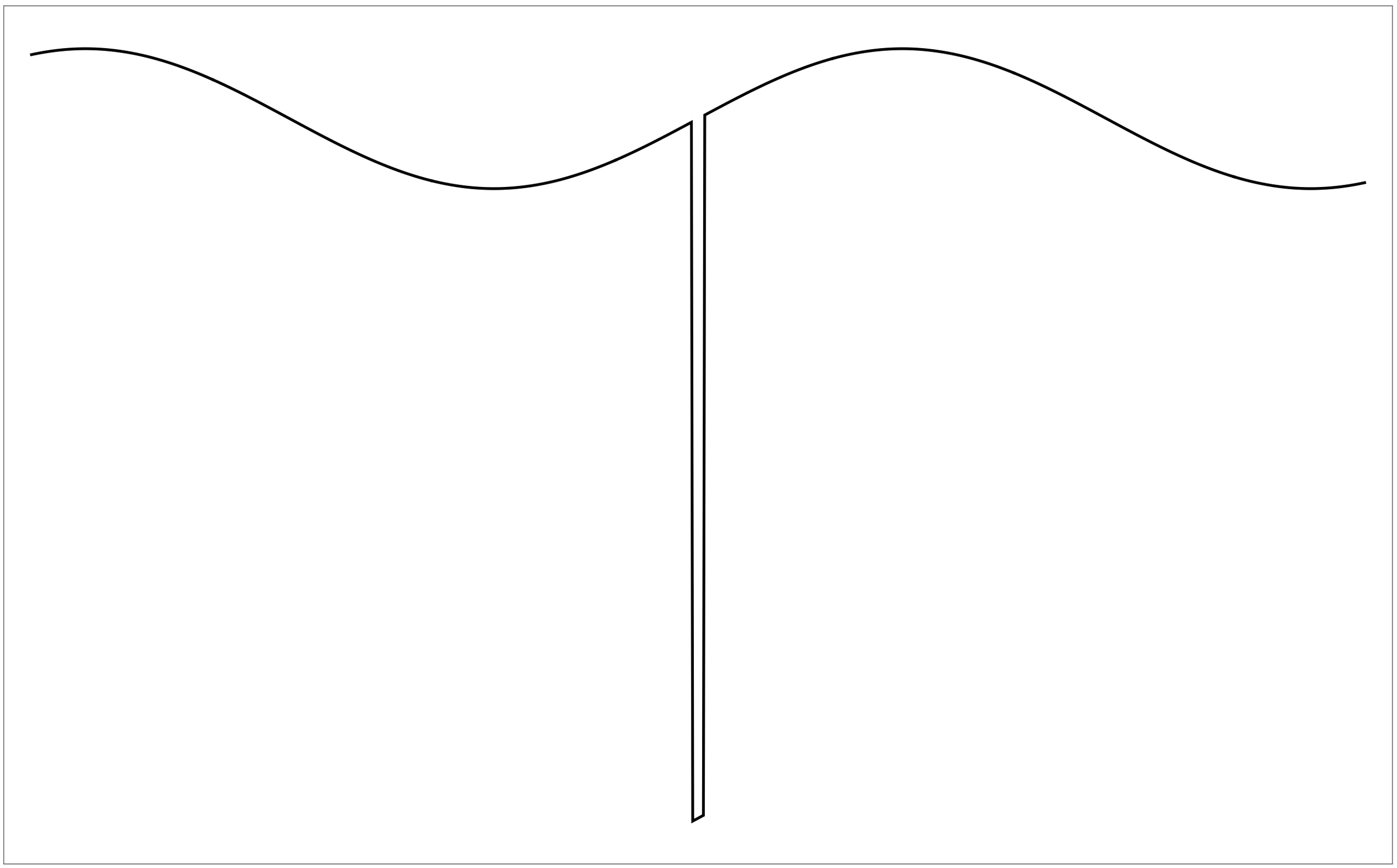
non-parametric detrending models

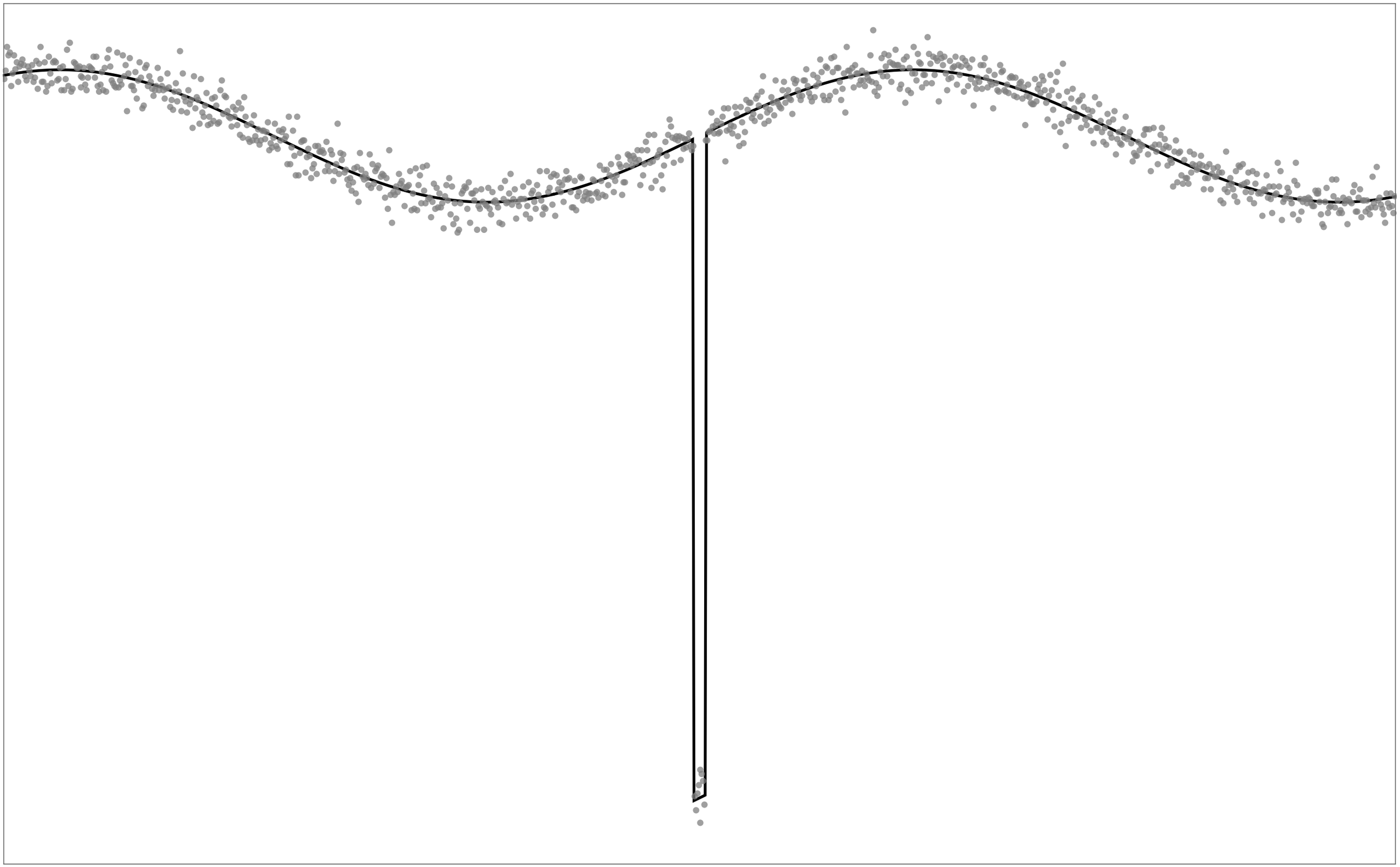
moving median example (assuming outliers have been removed)

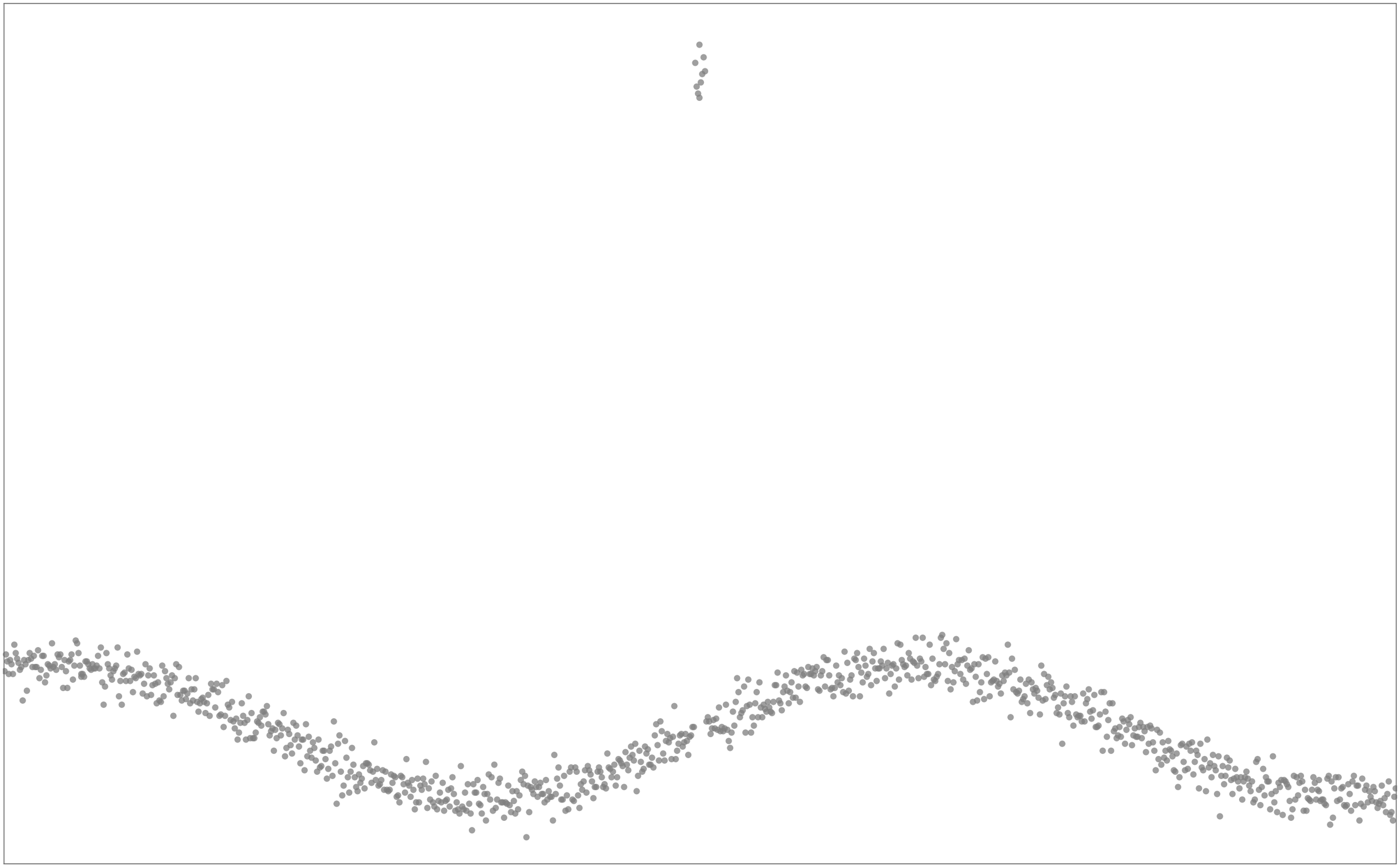
note that the model is not able to interpolate/extrapolate, so we can't omit the data which gives $\theta_{interest}$, we have to apply to all data

the choice of window size affects the results







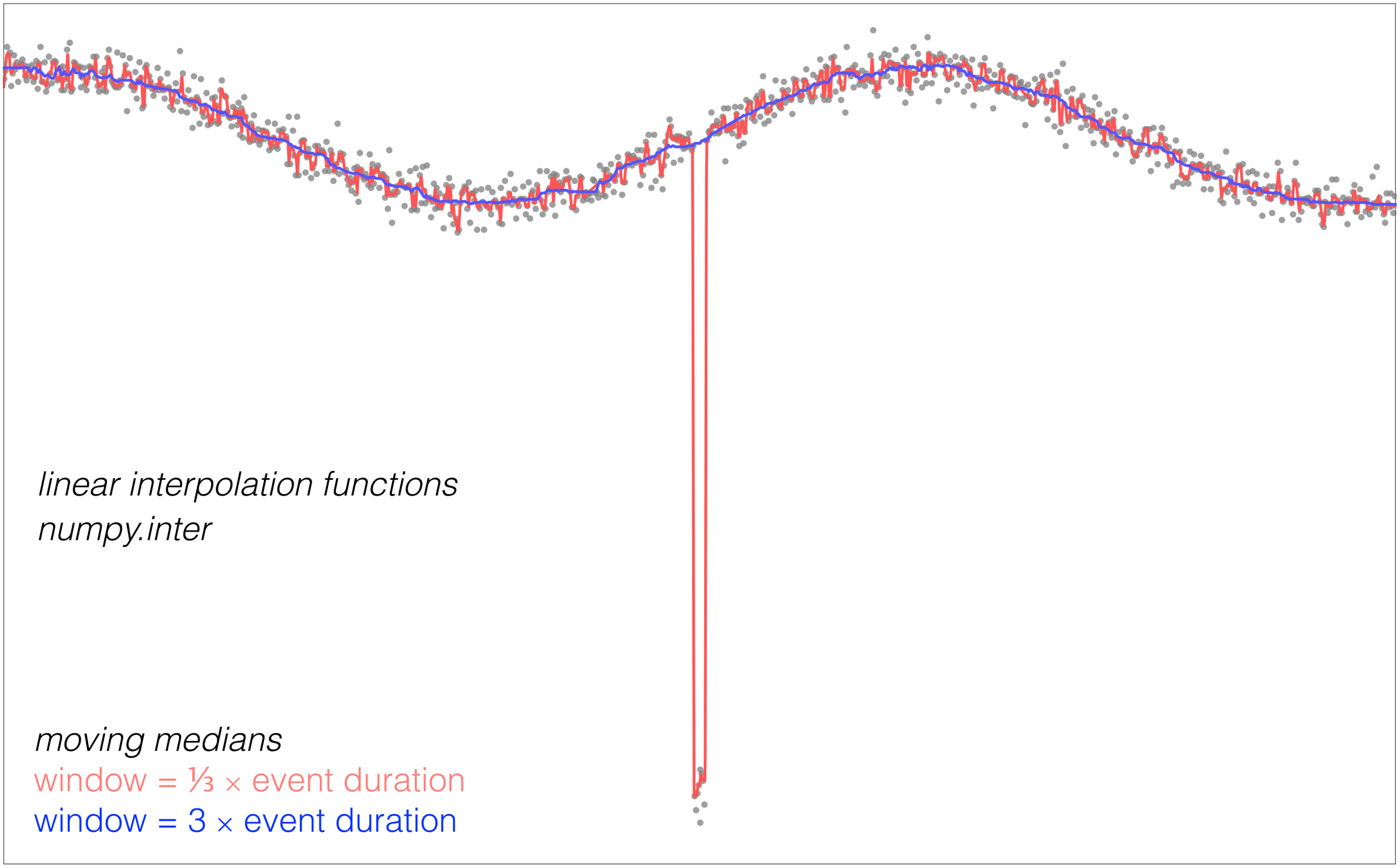


moving medians

window = ⅓ × event duration

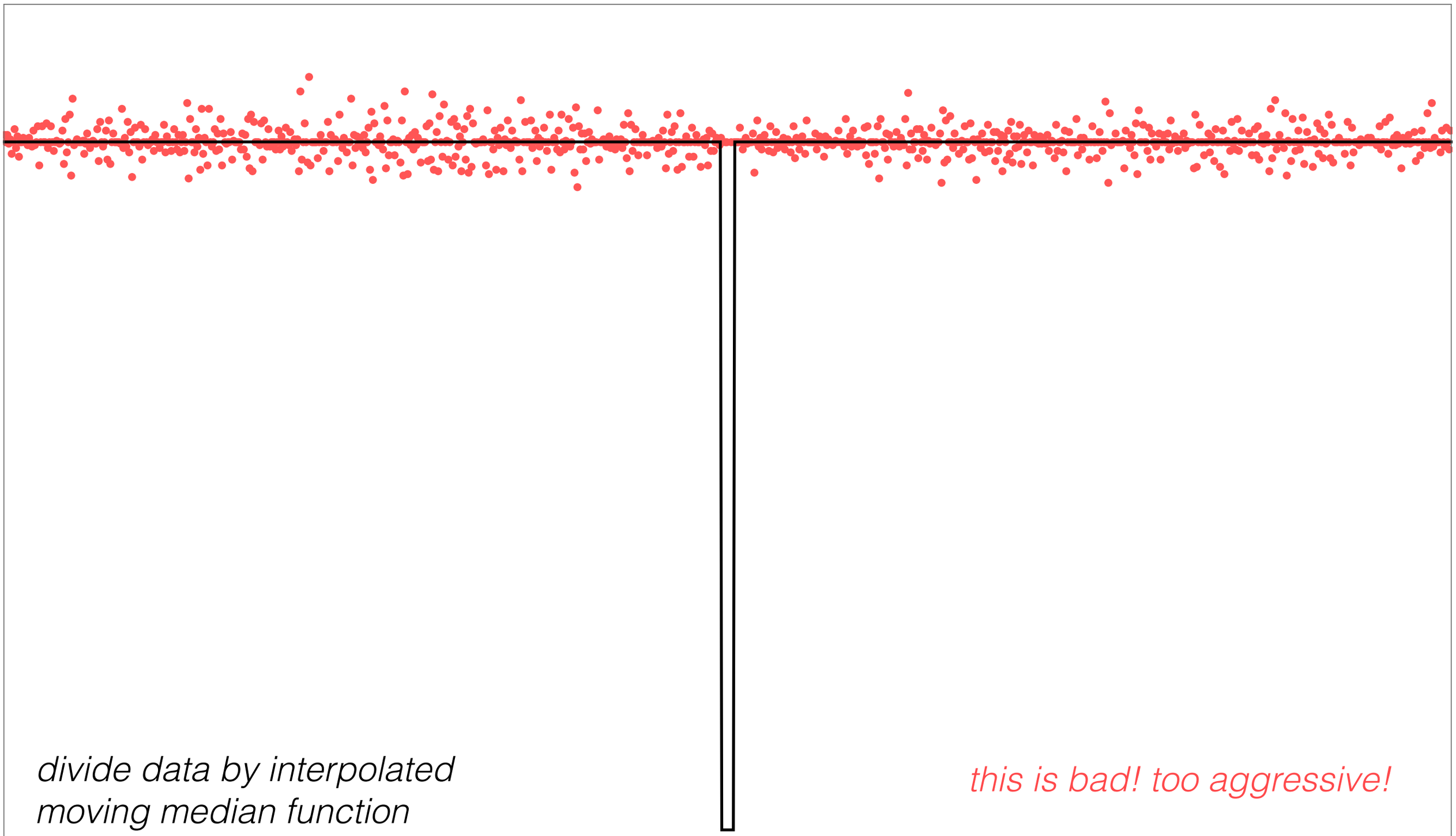
window = 3 × event duration





linear interpolation functions
numpy.inter

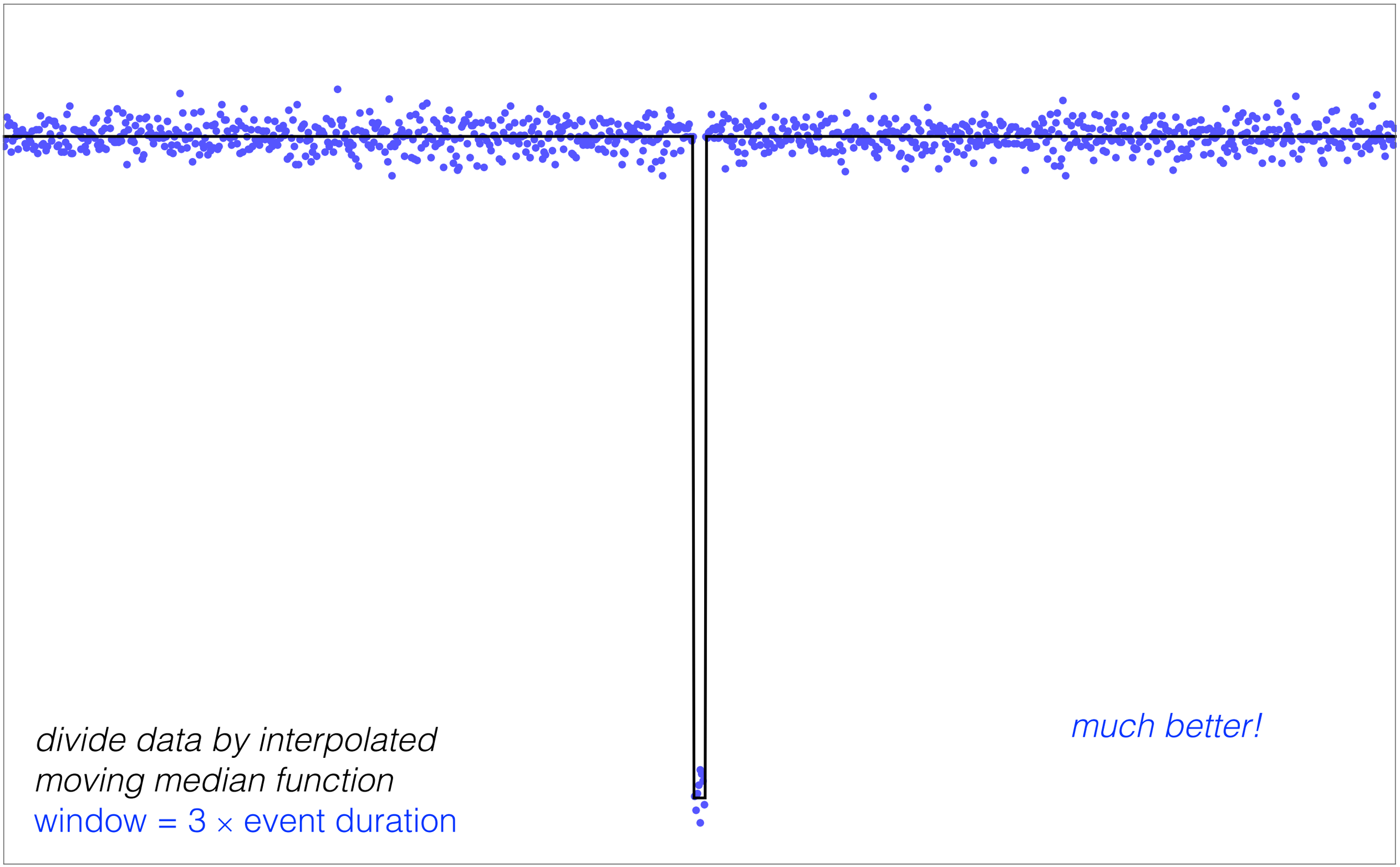
moving medians
window = ⅓ × event duration
window = 3 × event duration



divide data by interpolated moving median function

window = ⅓ × event duration

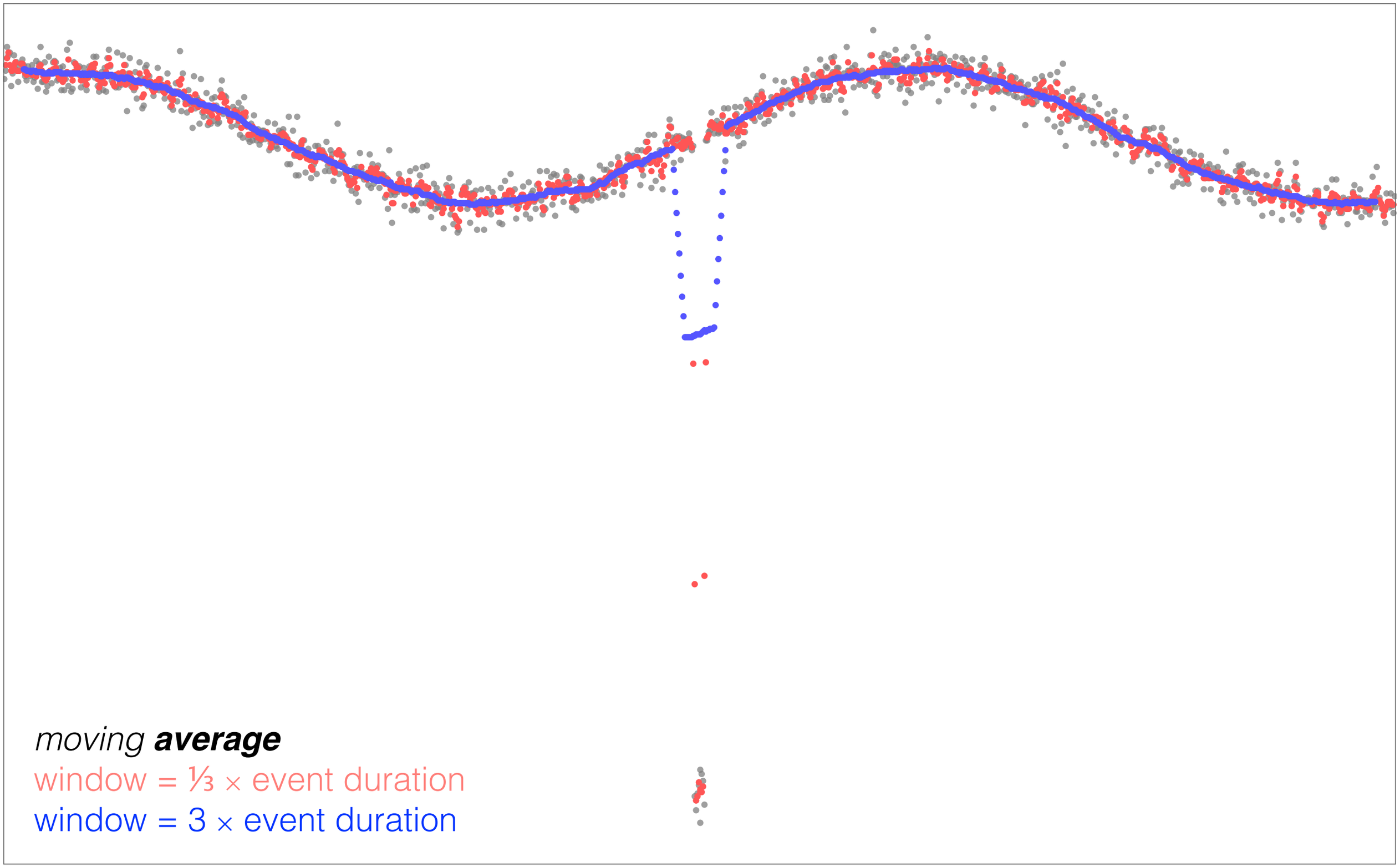
this is bad! too aggressive!



divide data by interpolated moving median function

window = 3 × event duration

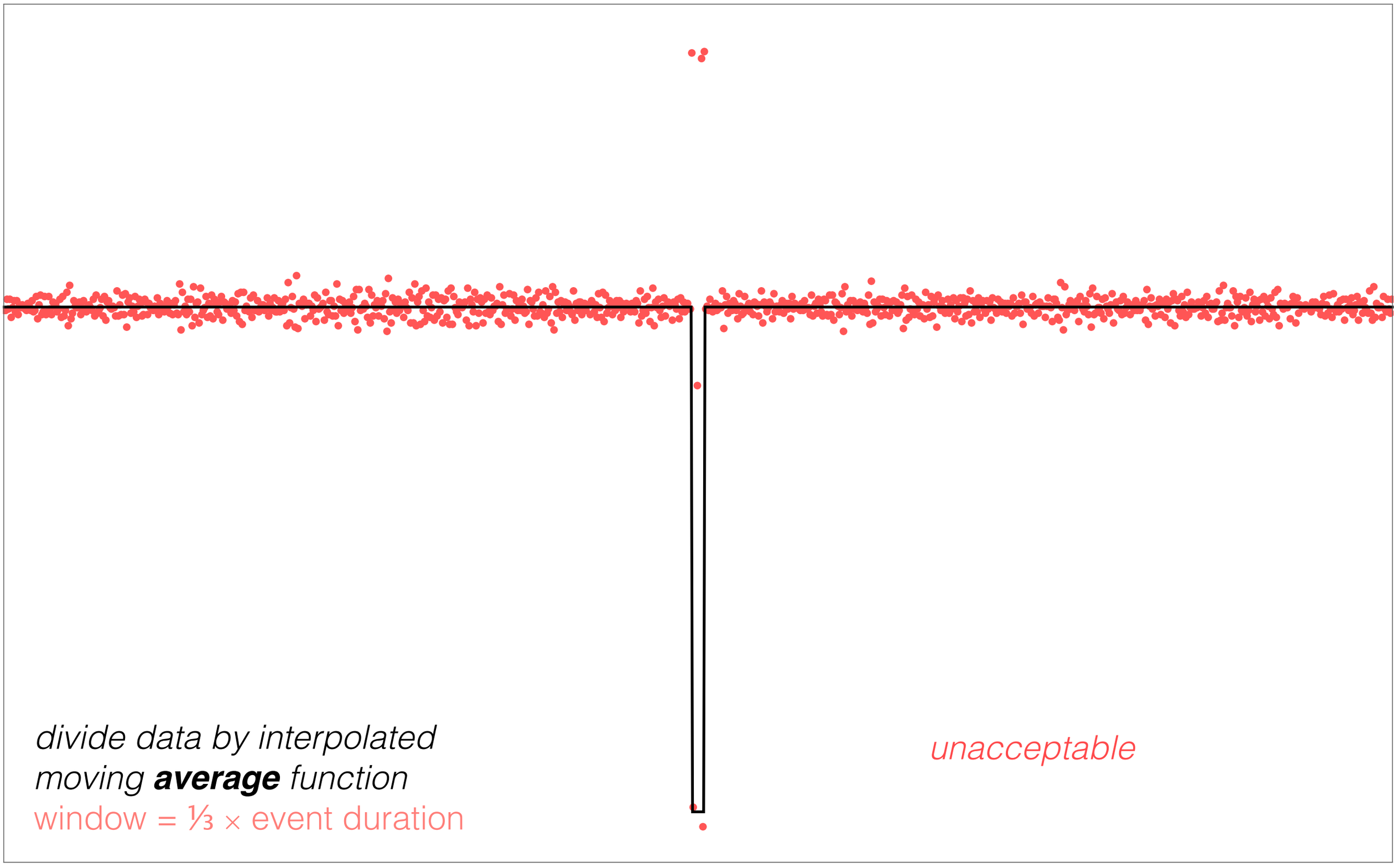
much better!



moving ***average***

window = ⅓ × event duration

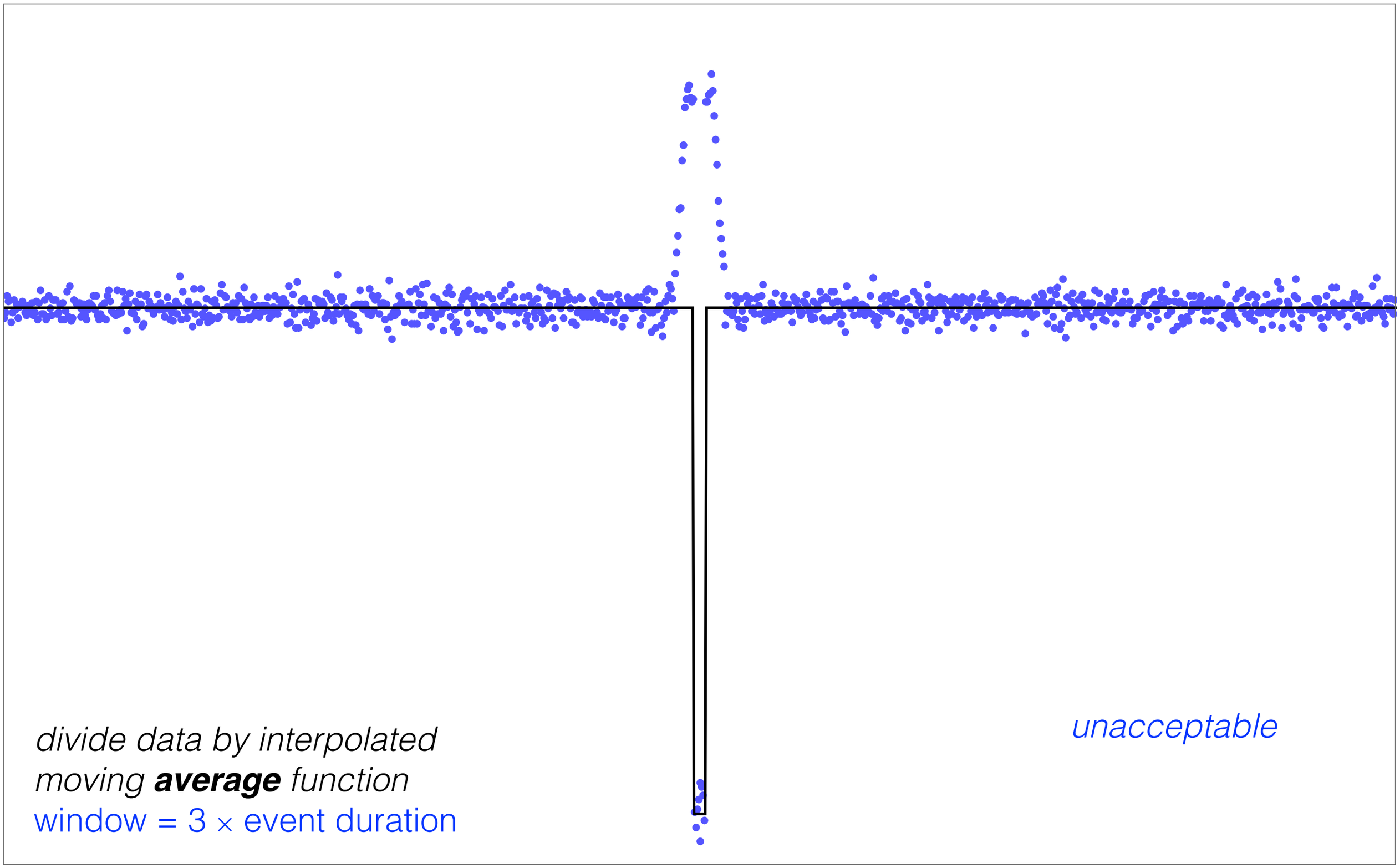
window = 3 × event duration



*divide data by interpolated moving **average** function*

window = ⅓ × event duration

unacceptable



divide data by interpolated
*moving **average** function*
window = 3 × event duration

unacceptable

take-away...

- moving averages are sensitive to features of interest, so can't be used for detrending
- moving medians can work, but ensure window is much longer than timescale of interest

but remember…

- short (but not too short!) window moving medians are used for finding outliers: traces out full signal
- long window moving medians are used as a low-cut filter, removes slow variations